DIPLOMATURA EN DATA SCIENCE · ACÁMICA · PROYECTO PROPERATI – PARTE 1

Predicción de precios de propiedades en Capital Federal

Análisis exploratorio, limpieza de outliers, preprocesamiento y comparación de modelos de regresión (lineal, KNN, árbol de decisión, polinómica con regularización y XGBoost)

Autor: Gerardo Masucci · **Versión:** V1 (noviembre 2026)

Contexto: En el notebook este trabajo se menciona como “proyecto 2”; el “proyecto 1” es el modelo base con el que se compara.

Repositorio: https://github.com/FreeAlejandria/Acamica-Proyecto_Properati_Part1

Stack: Python · pandas · NumPy · scikit-learn · XGBoost · matplotlib / seaborn

Resumen del trabajo

- **Objetivo:** estimar el precio en dólares de propiedades en venta (departamentos, PH y casas) de Capital Federal a partir del dataset de publicaciones de Properati (146.660 registros).
- **EDA y limpieza:** análisis de distribución por tipo de propiedad y zona, corrección de registros con superficie cubierta mayor a la total y eliminación escalonada de outliers por rango intercuartil (IQR) sobre superficies, precio y ambientes. Dataset final: 50.112 propiedades.
- **Preprocesamiento:** tratamiento de nulos, escalado con StandardScaler y one-hot encoding del tipo de propiedad.
- **Modelos simples:** frente al modelo del proyecto anterior, la mejora en la preparación de datos redujo el RMSE **más de un 65%** (KNN y árbol de decisión, hiperparámetros elegidos con curvas de validación).
- **Modelos avanzados:** regresión lineal con atributos polinómicos y regularización Lasso/Ridge (resultados pobres) y XGBoost con GridSearchCV y validación cruzada. El mejor desempeño se obtuvo con **XGBoost usando todos los atributos**, incluido el tipo de propiedad; la superficie cubierta fue el atributo de mayor peso.

> Este documento muestra el notebook completo tal como fue desarrollado (código, salidas y análisis). Se omitieron solo advertencias técnicas de librerías y celdas vacías, para facilitar la lectura. El código fuente y los datasets están en el repositorio indicado. El dataset original (Properati.csv) se descarga desde el enlace indicado en el README del repositorio. Las salidas de las secciones 3.2.1.1.2 y 3.2.1.1.3 se habían perdido en el archivo original y se re-ejecutaron con el mismo código y datos (ver nota en esa sección).

Índice

1 **Introduccion**

2 **EDA / Preprocesamiento**

2.1 Analisis Inicial de Datos

2.2 Detección y eliminación de Outliers y valores erroneos

2.2.1 Errores en valores de atributos surface_total o surface_covered

2.2.2 Eliminacion de Outliers escalonada

2.3 Valores nulos

2.4 Imputacion de Valores Faltantes

2.4 Escalado de Datos

2.5 Encoding

2.7 Comparacion con resultados de modelos del proyecto 1

2.7.1 Proyecto 1 y 2 / Arbol de Regresion y K Vecinos

2.7.2 DF Proyecto 2 Reescalado/ Arbol de Regresion y K Vecinos

2.7.3 Conclusiones/Resultados

3 **Modelos Avanzados**

3.1 Modelo Avanzado 1 - Regresión lineal con atributos polinómicos y regularización

3.1.1 Regresion Lineal

3.1.1.2 Regresion lineal con atributos polinomicos

3.1.1.3 Regresion lineal con Regularizacion Ridge

3.1.1.4 Regresion lineal con Regularizacion Lasso

3.1.1.5 Regresion lineal con atributos categoricos "property_type" diferenciados (departamento/ph y Casa)

3.1.2 Conclusiones/Resultados

3.2 Modelo Avanzado 2 - XGBoostRegression

3.2.1 XGBoostRegression con Gredsearch y CV

3.2.1.1 XGBoostRegression con Gredsearch con atributos categoricos "property_type" diferenciados (departamento/ph y Casa)

3.2.2 Conclusiones/Resultados

4 **Conclusiones Generales**

5 **Para seguir investigando**

- Glosario de Dataframes

- **properati:** df original con toda la info que esta en el dataset DS_Proyecto_01_Datos_Properati.csv
- **properati1:** df con **unicamente** todas las instancias que tiene un valor de surface_covered mayor al de surface_total.
- **properati_Capital:** df con unicamente todas las instancias que tienen l2= Capital y property_type=Ph o Departamento o Casa.
- **properati_BsAsNorte:** df con unicamente todas las instancias que tienen l2= BsAsNorte y property_type=Ph o Departamento o Casa.
- **properati_BsAsOeste:** df con unicamente todas las instancias que tienen l2= BsAsOeste y property_type=Ph o Departamento o Casa.
- **properati_BsAsSur:** df con unicamente todas las instancias que tienen l2= BsAsSur y property_type=Ph o Departamento o Casa.
- **properati_Capital1:** df con unicamente todas las instancias que tiene el df properati_Capital menos las instancias del df properati1.
- **new_properati_Capital1_SC:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre el atributo durface_covered.
- **new_properati_Capital1_ST:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre el atributo durface_total.
- **new_properati_Capital1_P:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre el atributo price.
- **new_properati_Capital1_SC_ST:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos surface_covered y surface_total.
- **new_properati_Capital1_SC_P:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos durface_covered y price.
- **new_properati_Capital1_ST_P:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos durface_total y price.
- **new_properati_Capital1_ST_SC_P:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos durface_covered, surface_total y price.
- **new_properati_Capital1_ST_SC_P:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos durface_covered, surface_total y price.
- **new_properati_Capital1_ST_P_B_R:** df con unicamente todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos surface_total, price, bathrooms y rooms.
- **new_properati_Capital1_All_sin_rooms:** df con todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos surface_total, price, bathrooms, bedrooms y surface_covered.
- **new_properati_Capital1_All_sin_bedrooms:** df df con todas las instancias que tiene el df properati_Capital1 menos las instancias que estan por fuera de las vallas que nos indica el rango intercuartil (IQR) calculado sobre los atributos surface_total, price, bathrooms, surface_covered y rooms.

- **new_properati_Capital1_All_sin_bedrooms_clean:** Viene de (**new_properati_Capital1_All_sin_bedrooms**)) pero sin los atributos que no voy a utilizar (l1,l2,start_date,end_date,created_on,lat,lon,currency, title, description, operation_type).
- **new_properati_Capital1_All_sin_bedrooms_clean_Scalet:** Viene de df (**new_properati_Capital1_All_sin_bedrooms_clean**) pero reescalado por el metodo Standard_Scaler.
- **new_properati_Capital2_All_sin_bedrooms_clean_Scalet:** Viene del df (**new_properati_Capital1_All_sin_bedrooms_clean_Scalet:**). Es un arreglo para tener un df mas ordenado en nombres de atributos luego del reescalado.
- **data_premiun:** Es un df creado del **new_properati_Capital2_All_sin_bedrooms_clean_Scalet**, solo con la utilidad de a partir de ahora, escribir menos.
- **df_proyecto1_clean**: Es el df del proyecto 1.

1 · Introduccion

> Tomando en cuenta las variables que hasta el momento tengo en el dataset (DS_Proyecto_01_Datos_Properati.csv); mas adelante mejoraria el analisis con la totalidad o parte de la siguiente informacion.

- Desde la vision de las **Caracteristicas de la Zona de la Propiedad**:

Esta informacion extra la tendria antes que el vendedor tome contacto con el cliente. Sera utilizada para definir un valor (moneda dolar), de la propiedad.

> - Seguridad efectiva (robos violentos, asesinatos).
> - Seguridad percibida (villas, carceles).
> - Estudio (jardines de Infantes, Colegios primarios y secundarios, Universidades).
> - Esparcimiento (gastronomia, Cine, Teatro, plazas).
> - Transporte publico (lineas de colectivos, trenes y subtes).
> - Salud (Clubes, Gimnasios, Hopitales, Clinicas).
> - Aglomeracion temporal de personas. Ej: Estadios que impliquen mucho transito de gente en eventos deportivos o musicales.
> - Aglomeracion diaria de personas.
> - Aglomeracion de negocios de nivel alto o bajo.
> - Compras diarias (Supermercados grandes y chicos).

- Desde la vision de las **Caracteristicas del Cliente**:

Con esta nueva informacion podria calcular un porcentaje de "Valor Extra" (positivo o negativo), de la propiedad. Para esto hace falta la capacidad del vendedor para conseguir informacion sobre el cliente. Imagino una app que tendria el vendedor con posibilidad de cargar info del cliente en vivo, y tener un precio de venta "actualizado", ajustando el analisis que ya realizo el modelo que calculo solo por las caracteristicas por Zona.

> - Edad.
> - Familia (parejas, hijo/s).
> - Estudia? En que Institucion? Tanto el/ella, como tambien sus hijos.
> - Ingresos.
> - Compra a realizar en Efectivo o Credito?
>
> Ej: Intuyendo causalidad, pero teniendo que reforzar encontrando correlacion; antes y despues de la compra.
>
> Un hombre solo que viene del interior para estudiar; tal vez estaria dispuesto a pagar mas por una misma propiedad si esta se encuentra mas cerca de su Universidad. Tal vez no le interesaria tanto la seguridad de la zona.
>
> Una familia con 2 hijos; tal vez este muy interesada por la seguridad de la zona y tener cerca el colegio de sus hijos.

2 · EDA / Preprocesamiento

2.1 · Analisis Inicial de Datos

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

properati= pd.read_csv("Properati.csv")
```

- **Para este analisis NO voy a utilizar las siguientes variables:**

-title y description.
-lat y lon: Las podre utilizar en otra etapa del trabajo para graficar con un mapa (aplicacion).
-start-date, end_date y created_on.

- **Las variables que SI voy a utilizar para este analisis son:**

surface_total, **surface_covered**, **bathrooms**, **rooms** (hace referencia a las cantidad de ambientes- utilizado en la Argentina), **bedrooms** (hace referencia a la cantidad de dormitorios - utilizado en el resto de Latinoamerica), **price**, **property_type**, **currency**, **l1**, **l2**, **l3**, **operation_type**

```
display(properati.currency.unique())
display(properati.operation_type.unique())
```

```
array(['USD'], dtype=object)
array(['Venta'], dtype=object)
```

Ya sabemos que todas las publicaciones en el dataset son en USD y por Venta. Informacion fundamental para ordenar el analisis.

```
display(np.shape(properati))
display(properati.head(5))
```

```
(146660, 19)
```

	start_date	end_date	created_on	lat	lon	l1	l2	l3	rooms	bedrooms	...	description	property_type	operation_type
0	2019-10-17	2019-12-23	2019-10-17	-34.605880	-58.384949	Argentina	Capital Federal	San Cristobal	7.0	7.0	...	DESCRIPCION DE LA PROPIEDAD: Departamento de 1...	Departamento	Venta
1	2019-10-17	2019-11-21	2019-10-17	-34.624056	-58.412110	Argentina	Capital Federal	Boedo	2.0	1.0	...	PH reciclado en Boedo a una cuadra de la plaz...	PH	Venta
2	2019-10-17	2019-11-01	2019-10-17	-34.593569	-58.427474	Argentina	Capital Federal	Palermo	2.0	1.0	...	2 ambienets amplio , excelente estado , patio ...	PH	Venta
3	2019-10-17	2019-12-23	2019-10-17	-34.581294	-58.436754	Argentina	Capital Federal	Palermo	2.0	1.0	...	HERMOSO PH EN PALERMO!!!2 AMBIENTES TOTALMENTE...	PH	Venta
4	2019-10-17	2020-03-11	2019-10-17	-34.914194	-57.938219	Argentina	Bs.As. G.B.A. Zona Sur	La Plata	2.0	1.0	...	58 entre 1 y 2 Venta de departamento en PH. P...	PH	Venta

```
properati.describe().round(2)
```

	lat	lon	rooms	bedrooms	bathrooms	surface_total	surface_covered	price
count	136735.00	136701.00	146660.00	146660.00	140703.00	126133.00	125046.00	146660.00
mean	-34.60	-58.47	3.08	1.98	1.60	216.87	112.82	241221.08
std	0.47	0.49	1.42	1.15	0.89	2037.61	916.86	318519.46
min	-35.32	-180.00	1.00	0.00	1.00	10.00	1.00	5500.00
25%	-34.63	-58.50	2.00	1.00	1.00	52.00	47.00	111000.00
50%	-34.60	-58.44	3.00	2.00	1.00	78.00	68.00	166000.00
75%	-34.57	-58.41	4.00	3.00	2.00	140.00	108.00	265000.00
max	85.05	-57.81	35.00	15.00	14.00	193549.00	126062.00	32434232.00

Del dataset properati lo que podemos observar a primera vista, y que llama mas la atencion, es:

> - Lo grande de la desviacion standar en comparacion con la media en surface_total y surface_covered.
> - Los minimos en bedrooms, surface_total y surface_covered.
> - Los maximos en rooms, bedrooms, bathrooms, surface_total, surface_covered y price.

```
properati.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 146660 entries, 0 to 146659
Data columns (total 19 columns):
 #   Column           Non-Null Count   Dtype  
---  ------           --------------   -----  
 0   start_date       146660 non-null  object 
 1   end_date         146660 non-null  object 
 2   created_on       146660 non-null  object 
 3   lat              136735 non-null  float64
 4   lon              136701 non-null  float64
 5   l1               146660 non-null  object 
 6   l2               146660 non-null  object 
 7   l3               146660 non-null  object 
 8   rooms            146660 non-null  float64
 9   bedrooms         146660 non-null  float64
 10  bathrooms        140703 non-null  float64
 11  surface_total    126133 non-null  float64
 12  surface_covered  125046 non-null  float64
 13  price            146660 non-null  float64
 14  currency         146660 non-null  object 
 15  title            146660 non-null  object 
 16  description      146660 non-null  object 
 17  property_type    146660 non-null  object 
 18  operation_type   146660 non-null  object 
dtypes: float64(8), object(11)
memory usage: 21.3+ MB
```

> -Veo la cantidad de nulos en cada atributo.

```
properati.isnull().sum()
```

```
start_date            0
end_date              0
created_on            0
lat                9925
lon                9959
l1                    0
l2                    0
l3                    0
rooms                 0
bedrooms              0
bathrooms          5957
surface_total     20527
surface_covered   21614
price                 0
currency              0
title                 0
description           0
property_type         0
operation_type        0
dtype: int64
```

```
print("Tipo de propiedades: ",properati.property_type.unique())
```

```
Tipo de propiedades:  ['Departamento' 'PH' 'Casa' 'Lote' 'Casa de campo' 'Oficina' 'Otro'
 'Local comercial' 'Cochera' 'Depósito']
```

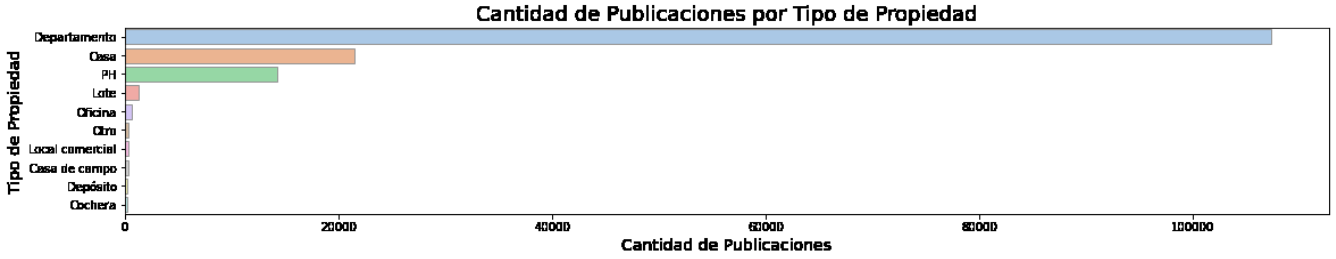
```
properati["property_type"].value_counts()
```

```
Departamento       107326
Casa                21521
PH                  14298
Lote                 1312
Oficina               658
Otro                  374
Local comercial       325
Casa de campo         322
Depósito              265
Cochera               259
Name: property_type, dtype: int64
```

> - De todos los tipos de propiedades publicadas las 3 mas numerosas son Departamentos, Casas y PH.

```
plt.subplots(figsize=(19, 3))
sns.countplot(y="property_type", data=properati, palette="pastel", edgecolor=".6",
              order=properati["property_type"]. value_counts().index)
plt.xlabel('Cantidad de Publicaciones',size=14)
plt.ylabel('Tipo de Propiedad',size=14)
plt.title('Cantidad de Publicaciones por Tipo de Propiedad',size=18)
```

```
Text(0.5, 1.0, 'Cantidad de Publicaciones por Tipo de Propiedad')
```




```
print("El pais es: ", properati.l1.unique())
print()
print("Las Provincias/Regiones son: ", properati.l2.unique())
print()
print("Las Ciudades son: ", properati.l3.unique())
```

```
El pais es:  ['Argentina']

Las Provincias/Regiones son:  ['Capital Federal' 'Bs.As. G.B.A. Zona Sur' 'Bs.As. G.B.A. Zona Norte'
 'Bs.As. G.B.A. Zona Oeste']

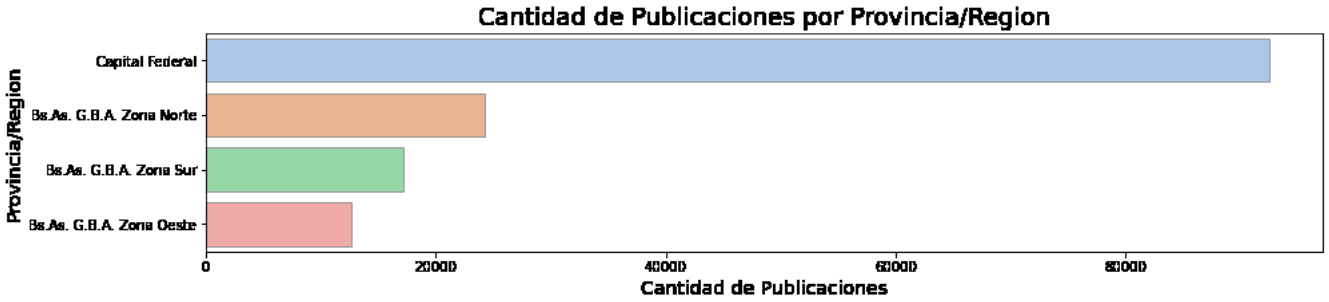
Las Ciudades son:  ['San Cristobal' 'Boedo' 'Palermo' 'La Plata' 'Villa Crespo'
 'Parque Patricios' 'Villa Pueyrredón' 'Pilar' 'Tigre' 'Abasto' 'Flores'
 'Morón' 'Nuñez' 'Almagro' 'Belgrano' 'Floresta' 'Recoleta' 'Balvanera'
 'Caballito' 'Chacarita' 'San Telmo' 'Colegiales' 'La Matanza'
 'San Isidro' 'San Miguel' 'Villa Luro' 'Barrio Norte' 'Las Cañitas'
 'San Nicolás' 'Villa Urquiza' 'Vicente López' 'Parque Centenario'
 'General San Martín' 'Villa General Mitre' 'Lanús' 'Ituzaingó' 'Boca'
 'Once' 'Retiro' 'Monserrat' 'Avellaneda' 'San Fernando' 'Puerto Madero'
 'Villa Ortuzar' 'Parque Chacabuco' 'Centro / Microcentro'
 'Tres de Febrero' 'Liniers' 'Coghlan' 'Congreso' 'Paternal' 'Saavedra'
 'Villa Santa Rita' 'Barracas' 'Villa Lugano' 'Constitución' 'Mataderos'
 'Villa Devoto' 'Parque Avellaneda' 'Villa del Parque'
 'Esteban Echeverría' 'Lomas de Zamora' 'Almirante Brown' 'Ezeiza'
 'San Vicente' 'Berazategui' 'Quilmes' 'Malvinas Argentinas' 'José C Paz'
 'Escobar' 'Moreno' 'General Rodríguez' 'Monte Castro' 'Pompeya' 'Merlo'
 'Marcos Paz' 'Florencio Varela' 'Agronomía' 'Presidente Perón'
 'Tribunales' 'Velez Sarsfield' 'Parque Chas' 'Hurlingham' 'Villa Real'
 'Versalles' 'Villa Riachuelo' 'Cañuelas' 'Villa Soldati' 'Catalinas']
```

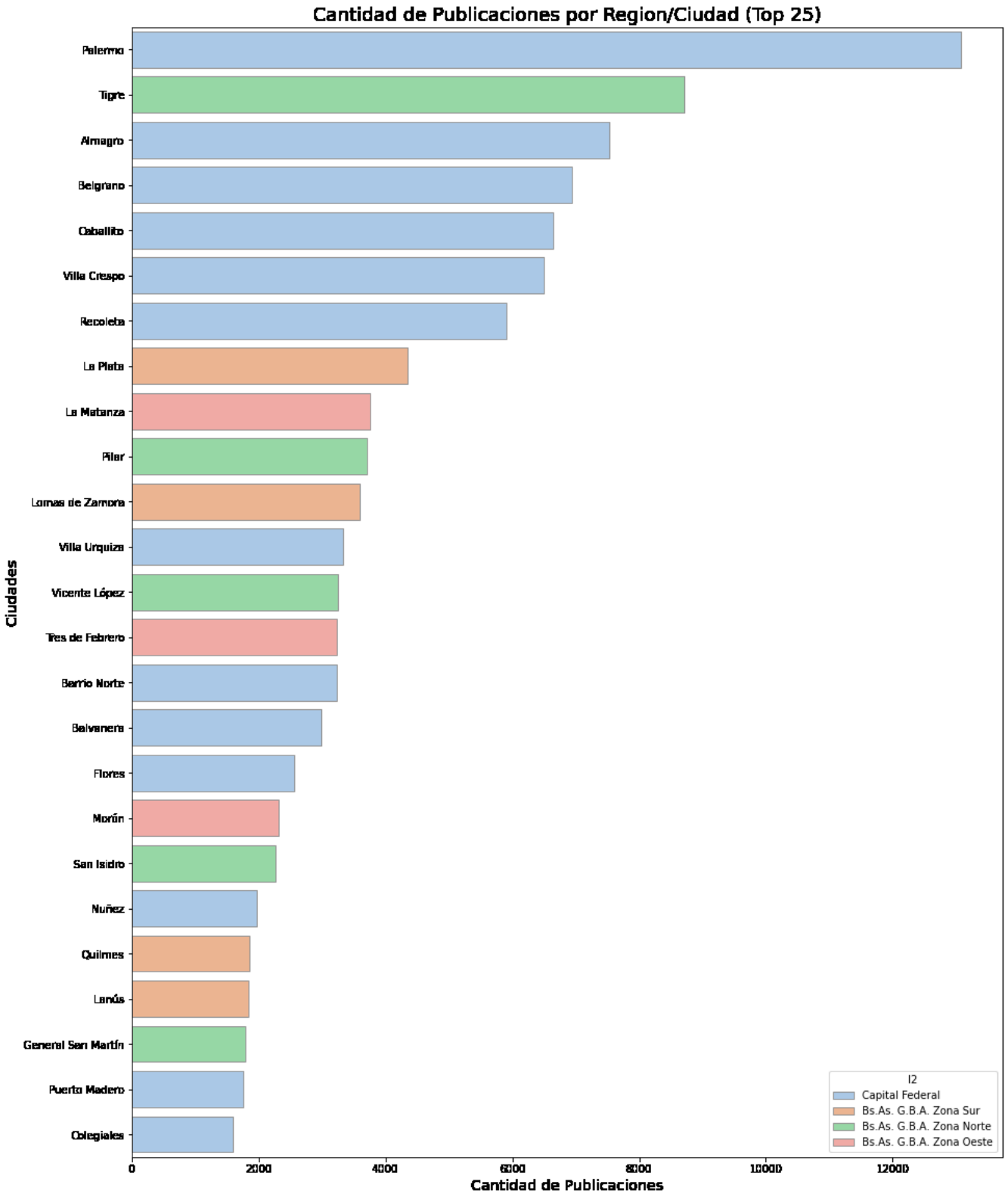
```
plt.subplots(figsize=(15, 3))
sns.countplot(y="l2", data=properati, palette="pastel", edgecolor=".6",
              order=properati["l2"]. value_counts().index)
plt.xlabel('Cantidad de Publicaciones',size=14)
plt.ylabel('Provincia/Region',size=14)
plt.title('Cantidad de Publicaciones por Provincia/Region',size=18)

plt.subplots(figsize=(15, 20))
```

```
sns.countplot(y="l3", data=properati, hue="l2", palette="pastel", edgecolor=".6", dodge=None,ax=None,
              order=properati["l3"]. value_counts().iloc[:25].index)
plt.xlabel('Cantidad de Publicaciones',size=14)
plt.ylabel('Ciudades',size=14)
plt.title("Cantidad de Publicaciones por Region/Ciudad (Top 25)" ,size=18)
```

```
Text(0.5, 1.0, 'Cantidad de Publicaciones por Region/Ciudad (Top 25)')
```





La Ciudades con mas publicaciones por Region son:

- Palermo por Capital federal.
- La Plata por BsAs Zona Sur.
- Tigre por BsAs Zona Norte.
- La Matanza por BsAs Zona Oeste.

- Creo 4 dataframe para ordenar mejor las publicaciones de Departamentos, Casas y PH en las 4 Regiones principales que son Capital Federal, BsAs Sur, BsAs Norte y BsAs Oeste.

```
print("Cantidad de columnas y filas por Region:\n")

properati_Capital=properati[(properati.l2=="Capital Federal")&
                            ((properati.property_type=="Departamento")|
                             (properati.property_type=="Casa")|
                             (properati.property_type=="PH"))]

print("Capital",properati_Capital.shape)

properati_BsAsSur=properati[(properati.l2=="Bs.As. G.B.A. Zona Sur")&
                            ((properati.property_type=="Departamento")|
                             (properati.property_type=="Casa")|
                             (properati.property_type=="PH"))]

print("BsAsSur",properati_BsAsSur.shape)

properati_BsAsOeste=properati[(properati.l2=="Bs.As. G.B.A. Zona Oeste")&
                            ((properati.property_type=="Departamento")|
                             (properati.property_type=="Casa")|
                             (properati.property_type=="PH"))]

print("BsAsOeste",properati_BsAsOeste.shape)

properati_BsAsNorte=properati[(properati.l2=="Bs.As. G.B.A. Zona Norte")&
                            ((properati.property_type=="Departamento")|
                             (properati.property_type=="Casa")|
                             (properati.property_type=="PH"))]

print("BsAsNorte",properati_BsAsNorte.shape)
```

```
Cantidad de columnas y filas por Region:

Capital (91485, 19)
BsAsSur (15814, 19)
BsAsOeste (12149, 19)
BsAsNorte (23697, 19)
```

```
print("La cantidad de publicaciones de las 3 principales propiedades en Capital Federal son:\n")
print(properati_Capital.property_type.value_counts())
print()
print("en BsAsSur son: ")
print(properati_BsAsSur.property_type.value_counts())
print()
print("en BsAsOeste son: ")
print(properati_BsAsOeste.property_type.value_counts())
print()
print("y en BsAsNorte son ")
print(properati_BsAsNorte.property_type.value_counts())
```

```
La cantidad de publicaciones de las 3 principales propiedades en Capital Federal son:

Departamento    79029
PH               9764
Casa             2692
Name: property_type, dtype: int64

en BsAsSur son:
Casa            7377
Departamento    7041
PH              1396
Name: property_type, dtype: int64

en BsAsOeste son:
Departamento    5588
Casa            5013
PH              1548
Name: property_type, dtype: int64

y en BsAsNorte son
Departamento    15668
Casa             6439
PH               1590
Name: property_type, dtype: int64
```

- Grafico de las 10 ciudades con mas publicaciones segun cada Region.

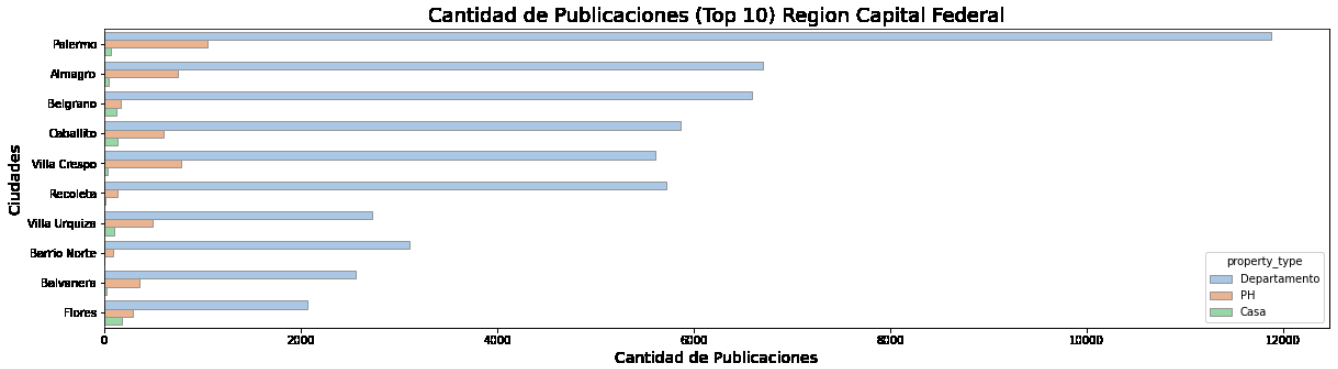
```
plt.subplots(figsize=(20, 23))
plt.subplot(4,1,4)
sns.countplot(y="l3", data=properati_Capital, hue="property_type",
              order=properati_Capital.l3.value_counts().iloc[:10].index,palette="pastel", edgecolor=".6")
plt.xlabel('Cantidad de Publicaciones',size=14)
plt.ylabel('Ciudades',size=14)
plt.title("Cantidad de Publicaciones (Top 10) Region Capital Federal" ,size=18)

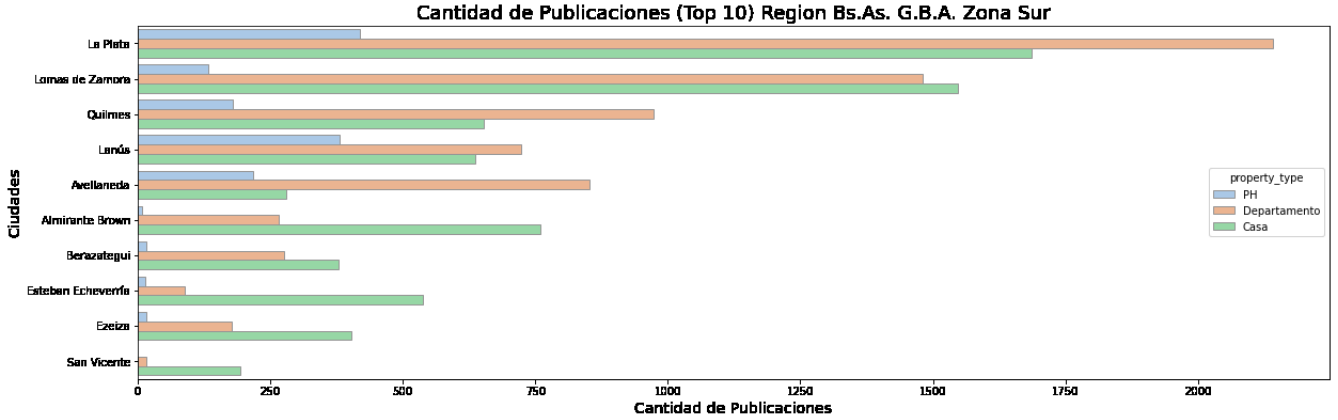
plt.subplots(figsize=(20, 28))
plt.subplot(4,1,1)
sns.countplot(y="l3", data=properati_BsAsSur,hue="property_type",
              order= properati_BsAsSur.l3.value_counts().iloc[:10].index,palette="pastel", edgecolor=".6")
plt.xlabel('Cantidad de Publicaciones',size=14)
plt.ylabel('Ciudades',size=14)
plt.title("Cantidad de Publicaciones (Top 10) Region Bs.As. G.B.A. Zona Sur" ,size=18)

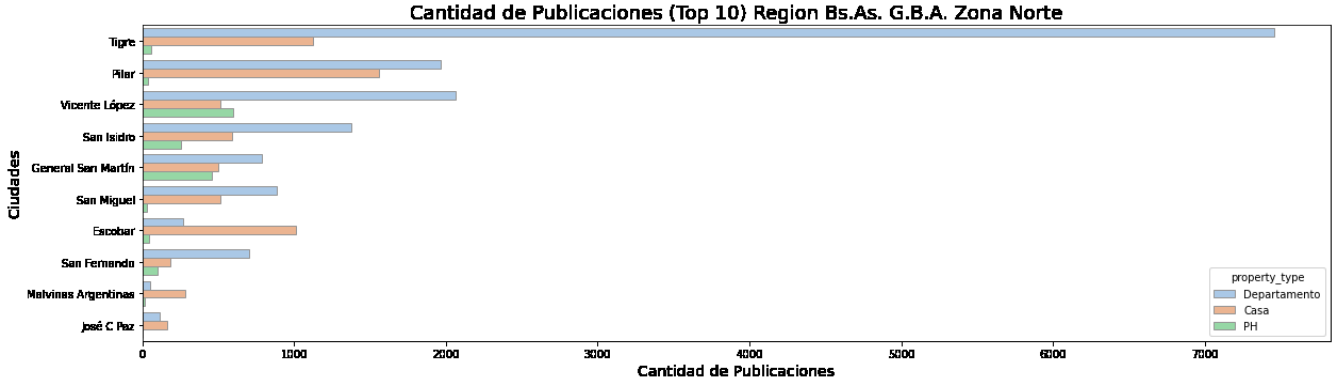
plt.subplots(figsize=(20, 25))
plt.subplot(4,1,2)
sns.countplot(y="l3", data=properati_BsAsNorte, hue="property_type",
              order=properati_BsAsNorte.l3.value_counts().iloc[:10].index,palette="pastel",
edgecolor=".6")
plt.xlabel('Cantidad de Publicaciones',size=14)
plt.ylabel('Ciudades',size=14)
plt.title("Cantidad de Publicaciones (Top 10) Region Bs.As. G.B.A. Zona Norte" ,size=18)

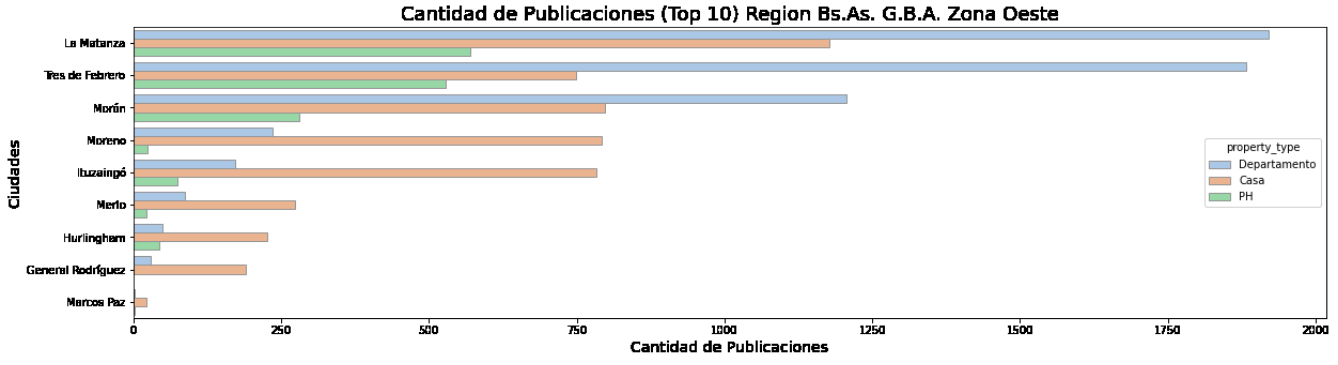
plt.subplots(figsize=(20, 23))
plt.subplot(4,1,3)
sns.countplot(y="l3", data=properati_BsAsOeste, hue="property_type",
              order=properati_BsAsOeste.l3.value_counts().iloc[:10].index,palette="pastel",
edgecolor=".6")
plt.xlabel('Cantidad de Publicaciones',size=14)
plt.ylabel('Ciudades',size=14)
plt.title("Cantidad de Publicaciones (Top 10) Region Bs.As. G.B.A. Zona Oeste" ,size=18)
```

```
Text(0.5, 1.0, 'Cantidad de Publicaciones (Top 10) Region Bs.As. G.B.A. Zona Oeste')
```









- Grafico de los promedios de precios por Ciudad; de Departamentos, Ph y Casas; diferenciando por Region.

```
plt.figure(figsize = (30,40))
def show_values_on_bars(axs, h_v="v", space=0.4, fontsize=30):
    def _show_on_single_plot(ax):
        if h_v == "v":
            for p in ax.patches:
                _x = p.get_x() + p.get_width() / 2
                _y = p.get_y() + p.get_height()
                value = int(p.get_height())
                ax.text(_x, _y, value, ha="center")
        elif h_v == "h":
            for p in ax.patches:
                _x = p.get_x() + p.get_width() + float(space)
```

```
                _y = p.get_y() + p.get_height()
                value = int(p.get_width())
                ax.text(_x, _y, value, ha="left")
    if isinstance(axs, np.ndarray):
        for idx, ax in np.ndenumerate(axs):
            _show_on_single_plot(ax)
    else:
        _show_on_single_plot(axs)

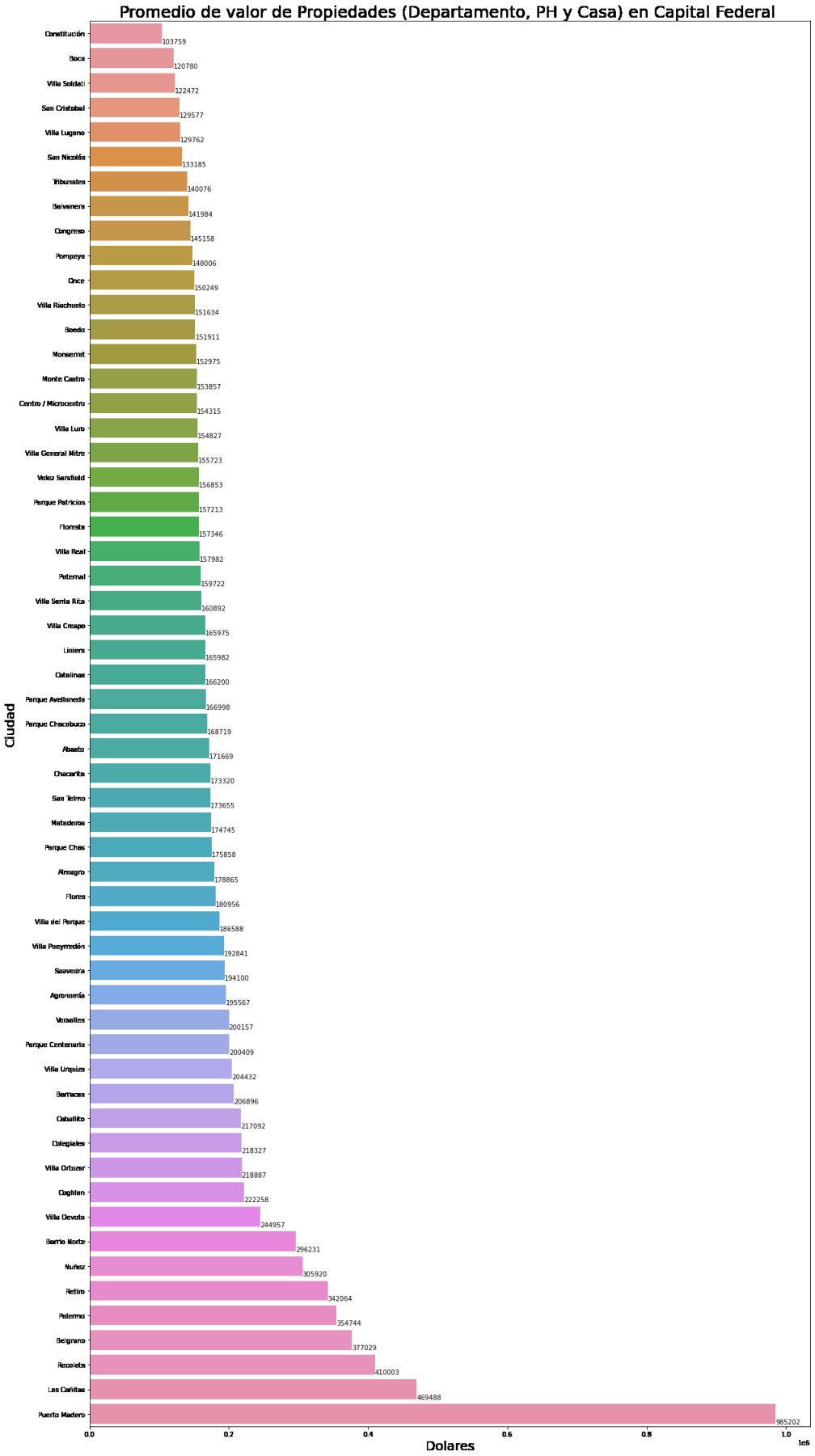
plt.figure(figsize = (20,40))
result_group = properati_Capital.groupby(["l3"]).mean()[["price"]].sort_values("price")
result_group['l3'] = result_group.index
ax = sns.barplot(x='price', y="l3", orient="h", data=result_group)
show_values_on_bars(ax, "h", 1)
plt.xlabel('Dolares',size=20)
plt.ylabel('Ciudad',size=20)
plt.title("Promedio de valor de Propiedades (Departamento, PH y Casa) en Capital Federal " ,size=25)
plt.show()

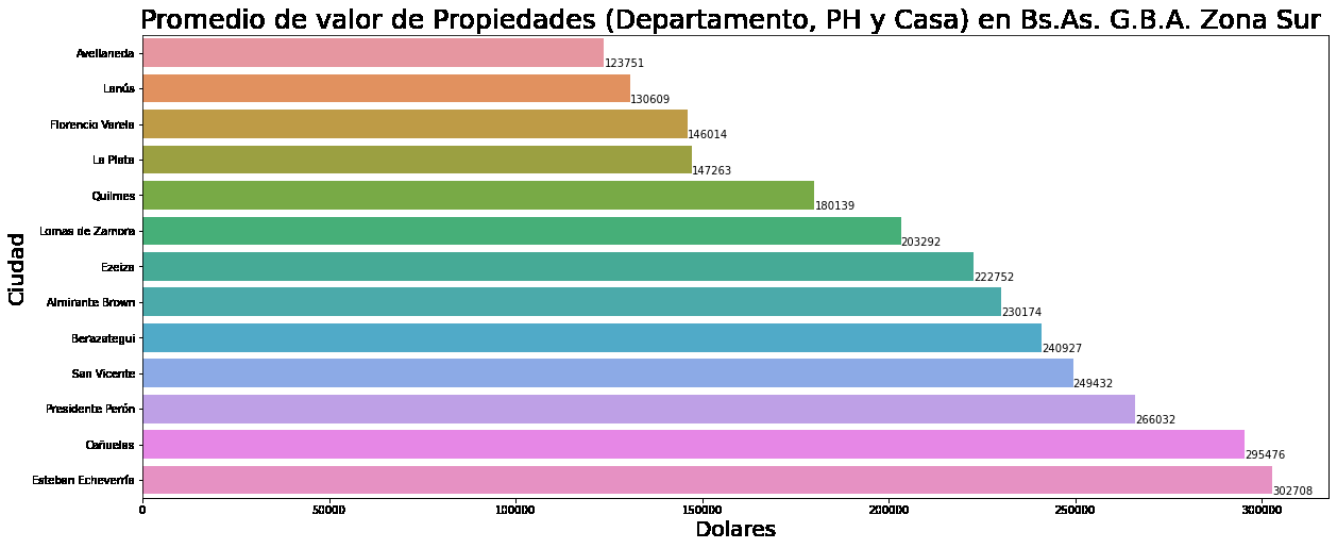

plt.figure(figsize = (20,8))
result_group = properati_BsAsSur.groupby(["l3"]).mean()[["price"]].sort_values("price")
result_group['l3'] = result_group.index
ax = sns.barplot(x='price', y="l3", orient="h", data=result_group)
show_values_on_bars(ax, "h", 1)
plt.xlabel('Dolares',size=20)
plt.ylabel('Ciudad',size=20)
plt.title("Promedio de valor de Propiedades (Departamento, PH y Casa) en Bs.As. G.B.A. Zona Sur "
,size=25)
plt.show()

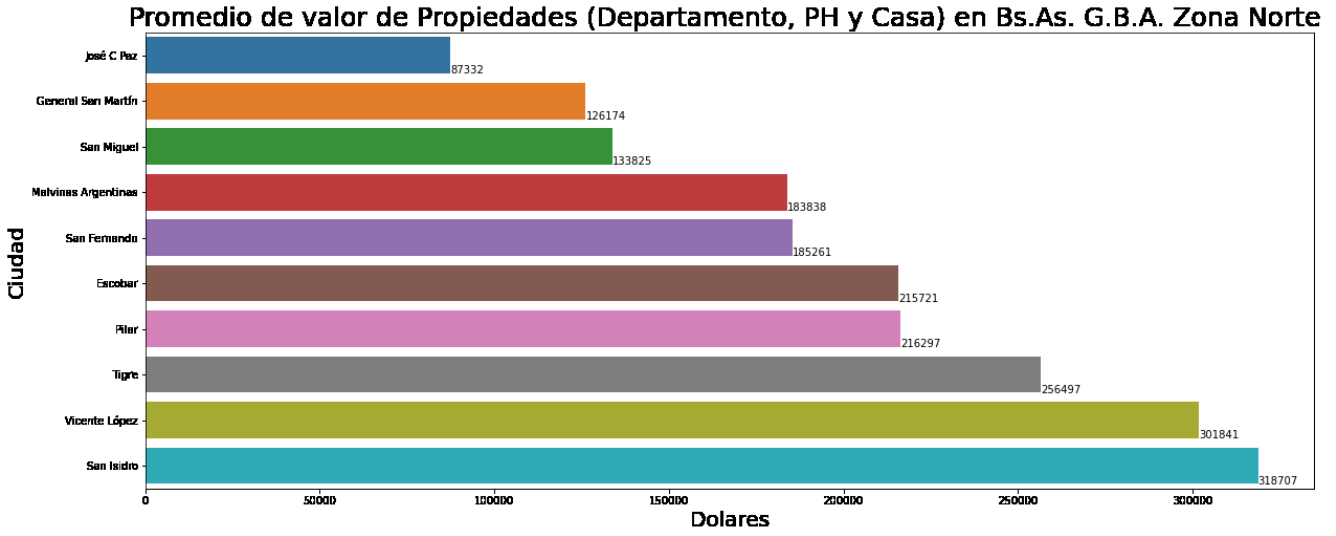
plt.figure(figsize = (20,8))
result_group = properati_BsAsNorte.groupby(["l3"]).mean()[["price"]].sort_values("price")
result_group['l3'] = result_group.index
ax = sns.barplot(x='price', y="l3", orient="h", data=result_group)
show_values_on_bars(ax, "h", 1)
plt.xlabel('Dolares',size=20)
plt.ylabel('Ciudad',size=20)
plt.title("Promedio de valor de Propiedades (Departamento, PH y Casa) en Bs.As. G.B.A. Zona Norte "
,size=25)
plt.show()

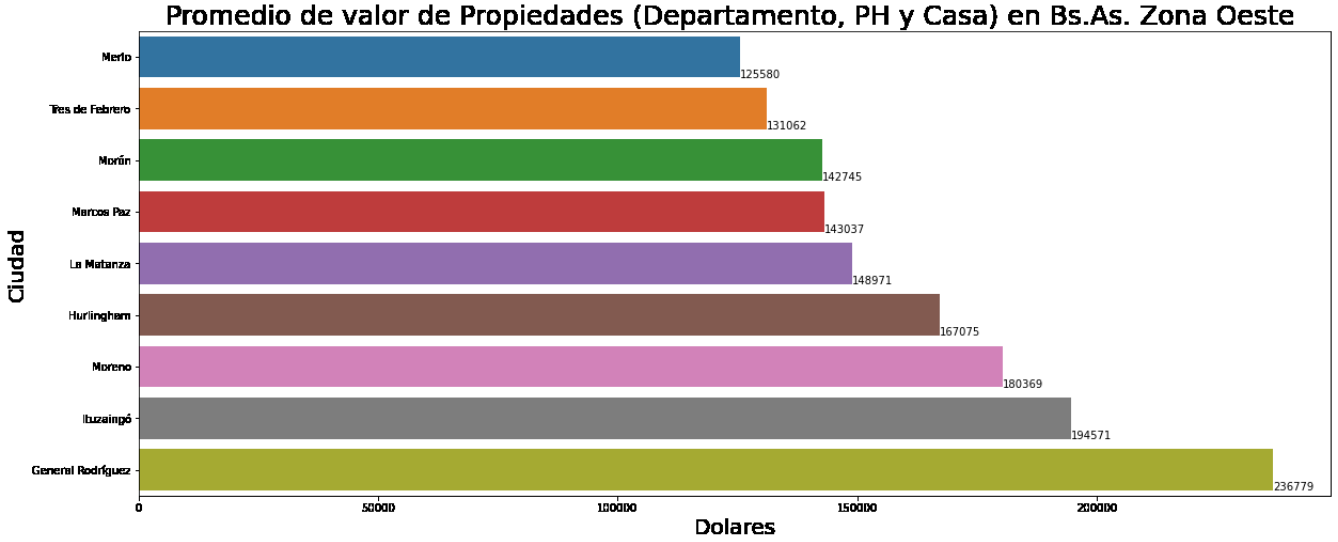
plt.figure(figsize = (20,8))
result_group = properati_BsAsOeste.groupby(["l3"]).mean()[["price"]].sort_values("price")
result_group['l3'] = result_group.index
ax = sns.barplot(x='price', y="l3", orient="h", data=result_group)
show_values_on_bars(ax, "h", 1)
plt.xlabel('Dolares',size=20)
plt.ylabel('Ciudad',size=20)
plt.title("Promedio de valor de Propiedades (Departamento, PH y Casa) en Bs.As. Zona Oeste " ,size=25)
plt.show()
```

```
<Figure size 2160x2880 with 0 Axes>
```


Promedio de valor de Propiedades (Departamento, PH y Casa) en Capital Federal
Ciudad
Dolares
1e6
Constitución 103759
Boca 120780
Villa Soldati 122472
San Cristobal 129577
Villa Lugano 129762
San Nicolás 133185
Tribunales 140076
Belvanera 141984
Congreso 145158
Pompeya 148006
Once 150249
Villa Riachuelo 151634
Boedo 151911
Monserrat 152975
Monte Castro 153857
Centro / Microcentro 154315
Villa Luro 154827
Villa General Mitre 155723
Velez Sarsfield 156853
Parque Patricios 157213
Floresta 157346
Villa Real 157982
Paternal 159722
Villa Santa Rita 160892
Villa Crespo 165975
Liniers 165982
Caballnas 166200
Parque Avellaneda 166998
Parque Chacabuco 168719
Abasto 171669
Chacarita 173320
San Telmo 173655
Mataderos 174745
Parque Chas 175858
Almagro 178865
Flores 180956
Villa del Parque 186588
Villa Pueyrredón 192841
Saavedra 194100
Agronomía 195567
Versalles 200157
Parque Centenario 200409
Villa Urquiza 204432
Barracas 206896
Caballito 217092
Colegiales 218327
Villa Ortuzar 218887
Coghlan 222258
Villa Devoto 244957
Barrio Norte 296231
Nuñez 305920
Retiro 342064
Palermo 354744
Belgrano 377029
Recoleta 410003
Las Cañitas 469488
Puerto Madero 985202
0.0
0.2
0.4
0.6
0.8
1.0







- Grafico de los promedios de precios en **Region Capital**; diferenciando por tipo de propiedad.

```
import time

t = time.process_time()

properati_Capital_Depto=properati[(properati.l2=="Capital Federal")&
                                  (properati.property_type=="Departamento")]

properati_Capital_PH=properati[(properati.l2=="Capital Federal")&
                               (properati.property_type=="PH")]
```

```
properati_Capital_Casa=properati[(properati.l2=="Capital Federal")&
                                 (properati.property_type=="Casa")]


plt.figure(figsize = (30,40))
def show_values_on_bars(axs, h_v="v", space=0.4, fontsize=30):
    def _show_on_single_plot(ax):
        if h_v == "v":
            for p in ax.patches:
                _x = p.get_x() + p.get_width() / 2
                _y = p.get_y() + p.get_height()
                value = int(p.get_height())
                ax.text(_x, _y, value, ha="center")
        elif h_v == "h":
            for p in ax.patches:
                _x = p.get_x() + p.get_width() + float(space)
                _y = p.get_y() + p.get_height()
                value = int(p.get_width())
                ax.text(_x, _y, value, ha="left")
    if isinstance(axs, np.ndarray):
        for idx, ax in np.ndenumerate(axs):
            _show_on_single_plot(ax)
    else:
        _show_on_single_plot(axs)


plt.figure(figsize = (20,40))
result_group = properati_Capital_Depto.groupby(["l3"]).mean()[["price"]].sort_values("price")
result_group['l3'] = result_group.index
ax = sns.barplot(x='price', y="l3", orient="h", data=result_group)
show_values_on_bars(ax, "h", 1)
plt.xlabel('Dolares',size=20)
plt.ylabel('Ciudad',size=20)
plt.title("Promedio de valor de Propiedades (Departamento) en Region Capital Federal " ,size=25)
plt.show()

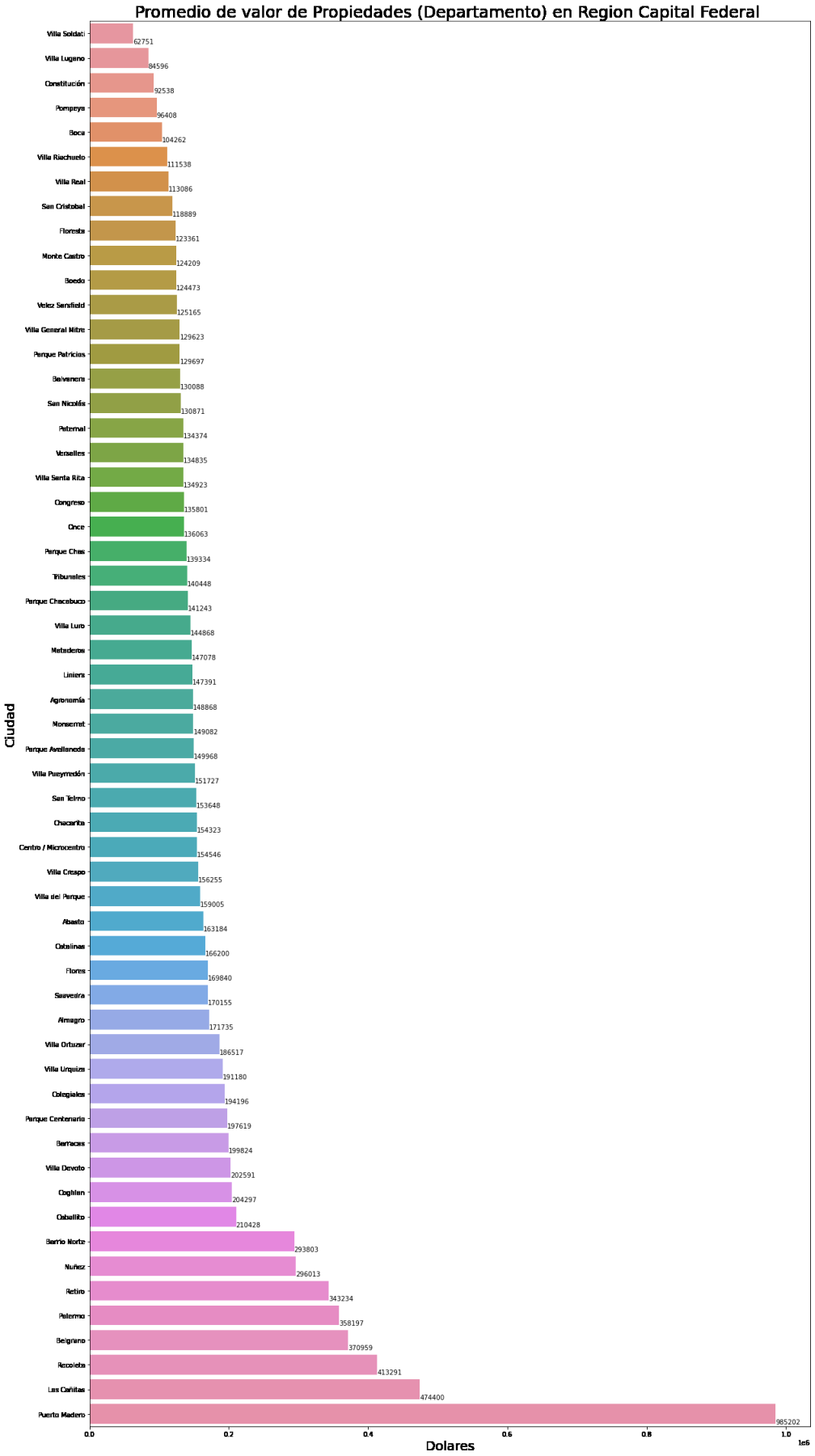
plt.figure(figsize = (20,40))
result_group = properati_Capital_PH.groupby(["l3"]).mean()[["price"]].sort_values("price")
result_group['l3'] = result_group.index
ax = sns.barplot(x='price', y="l3", orient="h", data=result_group)
show_values_on_bars(ax, "h", 1)
plt.xlabel('Dolares',size=20)
plt.ylabel('Ciudad',size=20)
plt.title("Promedio de valor de Propiedades (PH) en Region Capital Federal " ,size=25)
plt.show()

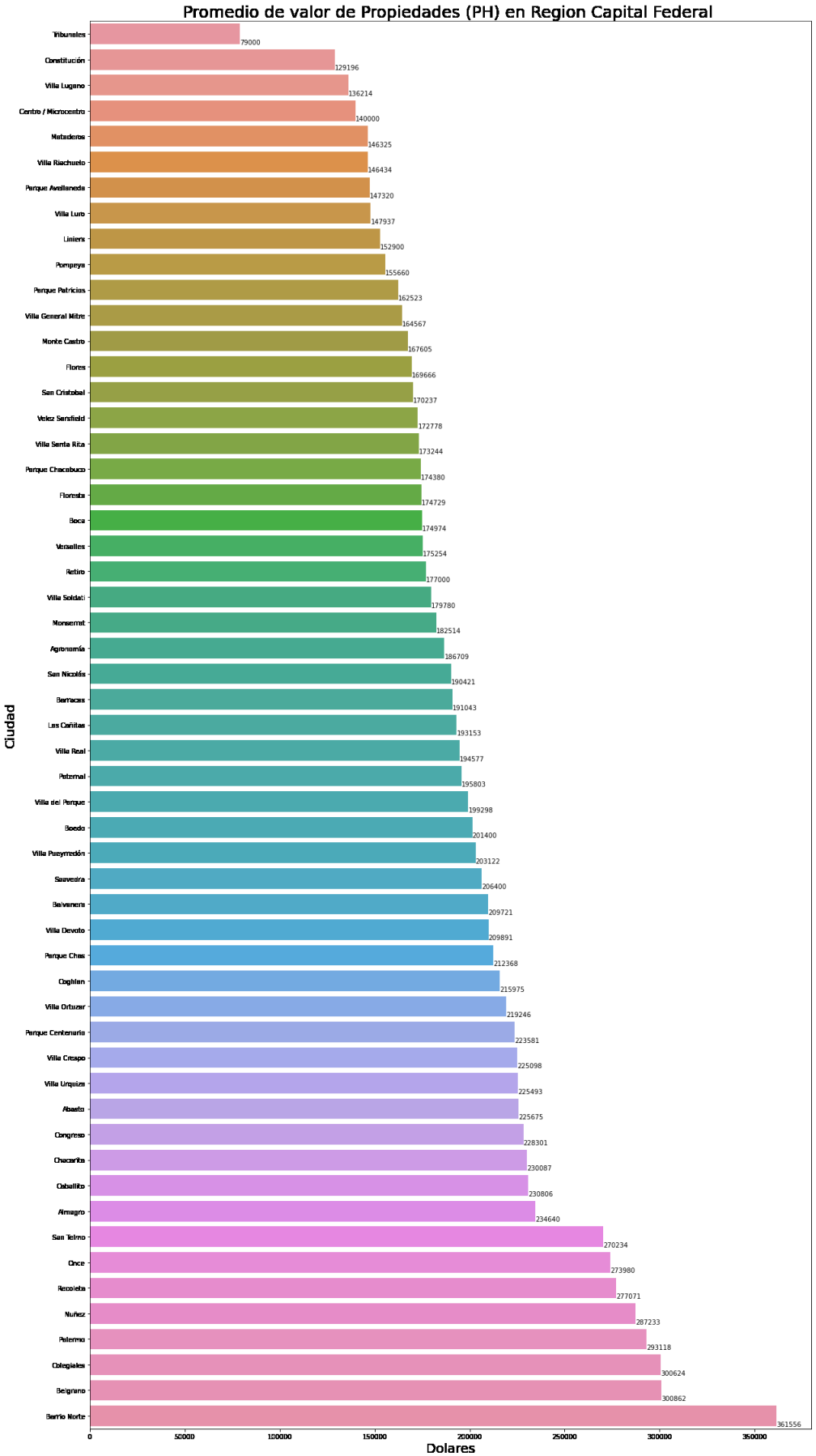
plt.figure(figsize = (20,40))
result_group =properati_Capital_Casa.groupby(["l3"]).mean()[["price"]].sort_values("price")
result_group['l3'] = result_group.index
ax = sns.barplot(x='price', y="l3", orient="h", data=result_group)
show_values_on_bars(ax, "h", 1)
plt.xlabel('Dolares',size=20)
plt.ylabel('Ciudad',size=20)
plt.title("Promedio de valor de Propiedades (Casa) en Region Capital Federal " ,size=25)
plt.show()

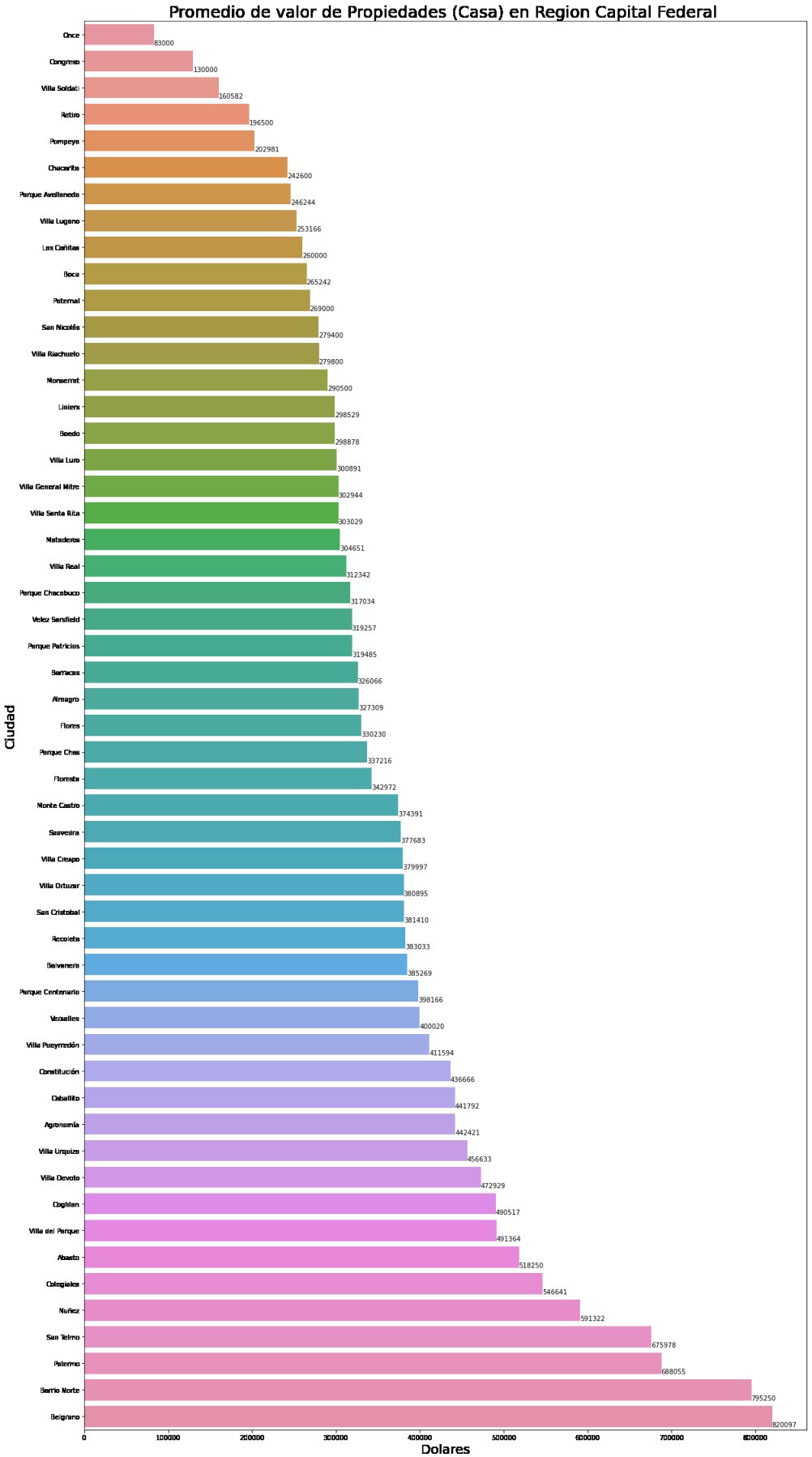

elapsed_time = time.process_time() - t

print(elapsed_time)
```

```
<Figure size 2160x2880 with 0 Axes>
```


Promedio de valor de Propiedades (Departamento) en Region Capital Federal
Villa Soldati 62751
Villa Lugano 84596
Constitución 92538
Pompeya 96408
Boca 104262
Villa Riachuelo 111538
Villa Real 113086
San Cristobal 118889
Floresta 123361
Monte Castro 124209
Boedo 124473
Velez Sarsfield 125165
Villa General Mitre 129623
Parque Patricios 129697
Belvanera 130088
San Nicolás 130871
Paternal 134374
Versalles 134835
Villa Santa Rita 134923
Congreso 135801
Once 136063
Parque Chas 139334
Tribunales 140448
Parque Chacabuco 141243
Villa Luro 144868
Mataderos 147078
Liniers 147391
Agronomía 148868
Monserrat 149082
Parque Avellaneda 149968
Villa Pueyrredón 151727
San Telmo 153648
Chacarita 154323
Centro / Microcentro 154546
Villa Crespo 156255
Villa del Parque 159005
Abasto 163184
Caballnas 166200
Flores 169840
Saavedra 170155
Almagro 171735
Villa Ortuzar 186517
Villa Urquiza 191180
Colegiales 194196
Parque Centenario 197619
Barracas 199824
Villa Devoto 202591
Coghlan 204297
Caballito 210428
Barrio Norte 293803
Nuñez 296013
Retiro 343234
Palermo 358197
Belgrano 370959
Recoleta 413291
Las Cañitas 474400
Puerto Madero 985202
Ciudad
0.0
0.2
0.4
0.6
0.8
1.0
1e6
Dolares


Promedio de valor de Propiedades (PH) en Region Capital Federal
Ciudad
Dolares
Tribunales 79000
Constitución 129196
Villa Lugano 136214
Centro / Microcentro 140000
Mataderos 146325
Villa Riachuelo 146434
Parque Avellaneda 147320
Villa Luro 147937
Liniers 152900
Pompeya 155660
Parque Patricios 162523
Villa General Mitre 164567
Monte Castro 167605
Flores 169666
San Cristobal 170237
Velez Sarsfield 172778
Villa Santa Rita 173244
Parque Chacabuco 174380
Floresta 174729
Boca 174974
Versalles 175254
Retiro 177000
Villa Soldati 179780
Monserrat 182514
Agronomía 186709
San Nicolás 190421
Barracas 191043
Las Cañitas 193153
Villa Real 194577
Paternal 195803
Villa del Parque 199298
Boedo 201400
Villa Pueyrredón 203122
Saavedra 206400
Belvanera 209721
Villa Devoto 209891
Parque Chas 212368
Coghlan 215975
Villa Ortuzar 219246
Parque Centenario 223581
Villa Crespo 225098
Villa Urquiza 225493
Abasto 225675
Congreso 228301
Chacarita 230087
Caballito 230806
Almagro 234640
San Telmo 270234
Once 273980
Recoleta 277071
Nuñez 287233
Palermo 293118
Colegiales 300624
Belgrano 300862
Barrio Norte 361556
0
50000
100000
150000
200000
250000
300000
350000




```
4.828125
```

> - Interesante observar que en las Cañitas, Recoleta y Puerto Madero no hay publicadas Casas ni Ph; solo Departamentos. Y son las zonas con promedios de precios mas altos de toda Capital

Federal.

- Quiero ver la correlacion de los atributos por zona, entre los atributos surface_covered, surface_total y price.

```
print("Corrrelacion Capital")
display(properati_Capital[["surface_total","surface_covered","price"]].corr().round(3))
print("Correlacion BsAs Sur")
display(properati_BsAsSur[["surface_total","surface_covered","price"]].corr().round(2))
print("Correlacion BsAs Oeste")
display(properati_BsAsOeste[["surface_total","surface_covered","price"]].corr().round(3))
print("Correlacion BsAs Norte")
display(properati_BsAsNorte[["surface_total","surface_covered","price"]].corr().round(3))
```

```
Corrrelacion Capital
```

	surface_total	surface_covered	price
surface_total	1.000	0.771	0.049
surface_covered	0.771	1.000	0.055
price	0.049	0.055	1.000

```
Correlacion BsAs Sur
```

	surface_total	surface_covered	price
surface_total	1.00	0.11	0.15
surface_covered	0.11	1.00	0.07
price	0.15	0.07	1.00

```
Correlacion BsAs Oeste
```

	surface_total	surface_covered	price
surface_total	1.000	0.315	0.060
surface_covered	0.315	1.000	0.035
price	0.060	0.035	1.000

```
Correlacion BsAs Norte
```

	surface_total	surface_covered	price
surface_total	1.000	0.308	0.072
surface_covered	0.308	1.000	0.073
price	0.072	0.073	1.000

- Comparando las correlaciones entre todo el dataset y por zona geografica; viendo solo los atributos surface_total, surface_covered y price; solo se destaca que la mayor correlacion es entre surface_total y surface_covered en zona Capital Federal, siendo del 0.77. Todas las otras correlaciones son muy bajas.

```
print("La cantidad de nulos de las 3 principales propiedades en Capital Federal son:\n")
print(properati_Capital[["surface_covered"]].isnull().sum())
print(properati_Capital [["surface_total"]].isnull().sum())
print(properati_Capital[["bathrooms"]].isnull().sum())
print()
print("La cantidad de nulos de las 3 principales propiedades en BsAsSur son:")
print(properati_BsAsSur[["surface_covered"]].isnull().sum())
print(properati_BsAsSur [["surface_total"]].isnull().sum())
print(properati_BsAsSur[["bathrooms"]].isnull().sum())
print()
print("La cantidad de nulos de las 3 principales propiedades en BsAsOeste son:")
print(properati_BsAsOeste[["surface_covered"]].isnull().sum())
print(properati_BsAsOeste [["surface_total"]].isnull().sum())
print(properati_BsAsOeste[["bathrooms"]].isnull().sum())
print()
print("La cantidad de nulos de las 3 principales propiedades en BsAsNorte son:")
print(properati_BsAsNorte[["surface_covered"]].isnull().sum())
```

```
print(properati_BsAsNorte[["surface_total"]].isnull().sum())
print(properati_BsAsNorte[["bathrooms"]].isnull().sum())
```

```
La cantidad de nulos de las 3 principales propiedades en Capital Federal son:

surface_covered    9235
dtype: int64
surface_total    8917
dtype: int64
bathrooms    2171
dtype: int64

La cantidad de nulos de las 3 principales propiedades en BsAsSur son:
surface_covered    4638
dtype: int64
surface_total    4659
dtype: int64
bathrooms    832
dtype: int64

La cantidad de nulos de las 3 principales propiedades en BsAsOeste son:
surface_covered    3339
dtype: int64
surface_total    3343
dtype: int64
bathrooms    785
dtype: int64

La cantidad de nulos de las 3 principales propiedades en BsAsNorte son:
surface_covered    1697
dtype: int64
surface_total    1704
dtype: int64
bathrooms    423
dtype: int64
```

> Importante: A partir de aca voy a trabajar con el Dataframe properati_Capital. Voy a trabajar con esta informacion porque es la que mas publicaciones posee y hay muchas diferencias con las otras 3 regiones. Algunas intuitivas porque conozco las zonas, como seguridad, institutos educativos, bienestar (disponibilidad de servicios basicos), etc; otras objetivas como los promedios de precios. Capital es la region con promedios de precios mas elevados de toda la Argentina.

2.2 · Detección y eliminación de Outliers y valores erroneos

2.2.1 · Errores en valores de atributos surface_total o surface_covered

- Voy a detectar si hay valores de surface_covered mayores a los de surface_total para la misma instancia. Esto claramente se trataria de un error. A partir de ahi ver que estrategia seguir para resolver el problema.

```
properati1=  properati_Capital[(properati_Capital.surface_total<properati_Capital.surface_covered)]
display(properati1.shape)
properati1.head(2)
```

```
(760, 19)
```

	start_date	end_date	created_on	lat	lon	l1	l2	l3	rooms	bedrooms	...	description	property_type	operation_type
446	2019-03-04	2019-08-17	2019-03-04	-34.613369	-58.488637	Argentina	Capital Federal	Villa Santa Rita	5.0	4.0	...	Corredor Responsable: Mónica Silvia Martinuzzi...	Casa	Venta
859	2019-04-25	2019-05-21	2019-04-25	-34.561805	-58.459579	Argentina	Capital Federal	Belgrano	3.0	2.0	...	Corredor Responsable: Gustavo Guastello - C.U....	Departamento	Venta

```
properati1[["surface_total","surface_covered","bathrooms"]].isnull().sum()
```

```
surface_total      0
surface_covered    0
bathrooms          8
dtype: int64
```

- Comparo el df properati_Capital con df properati1.

```
print("Dataframe
properati_Capital\n",properati_Capital[["surface_total","surface_covered"]].describe().round(2))
print()
print("Dataframe properati1\n", properati1[["surface_total","surface_covered"]].describe().round(2))
print()
print("Dataframe
properati_Capital\n",properati_Capital[["surface_total","surface_covered"]].corr().round(3))
print()
print("Dataframe properati1\n",properati1[["surface_total","surface_covered"]].corr().round(3))
print()
print("Dataframe properati_Capital\n",properati_Capital[["surface_total","surface_covered"]].shape)
print()
print("Dataframe properati1\n",properati1[["surface_total","surface_covered"]].shape)
```

```
Dataframe properati_Capital
        surface_total  surface_covered
count       82568.00         82250.00
mean          112.63            94.97
std           980.71           778.52
min            10.00             1.00
25%            49.00            44.00
50%            71.00            63.00
75%           111.00            96.00
max        126062.00        126062.00

Dataframe properati1
        surface_total  surface_covered
count         760.00           760.00
mean           98.47           392.77
std           147.04          1456.95
min            10.00            16.00
25%            40.00            44.00
50%            61.00            72.50
75%           112.00           177.50
max          3335.00         17220.00

Dataframe properati_Capital
                  surface_total  surface_covered
surface_total             1.000            0.771
surface_covered           0.771            1.000

Dataframe properati1
                  surface_total  surface_covered
surface_total             1.000            0.143
surface_covered           0.143            1.000

Dataframe properati_Capital
 (91485, 2)

Dataframe properati1
 (760, 2)
```

> Conclusiones sobre errores en valores de surface_total o surface_covered.
>
> - Si comparamos las correlaciones entre el df properati_Capital y el df properati1 (que es el que posee solo las instancias con errores en los dos atributos en cuestion); vemos que la correlacion es de 0.77 en el properati_Capital y por debajo de 0.2 en el properati1.
> - Si comparamos los valores de descripcion general vemos que el promedio de surface_covered es 4 veces mayor y la desviacion standar 2 veces mayor en properati1 en relacion con properati_Capital.
> - No puedo saber si el error esta en surface_total, en surface_covered o en ambos.
>
> **- Por todas estas conclusiones que no me permiten decidir algun tipo de imputacion o correccion de valores, y considerando que esas instancias son menos de un 1% del total de las instancias del dataset; las voy a eliminar.**

```
properati_Capital1= properati_Capital.drop(properati1 .index, axis=0)
```

```
display (properati_Capital.shape)
display(properati1.shape)
properati_Capital1.shape
```

```
(91485, 19)
(760, 19)
(90725, 19)
```

```
91485-760 #Comprobacion.
```

```
90725
```

2.2.2 · Eliminacion de Outliers escalonada

> Lo que conviene es comenzar a sacar outliers desde el atributo que tenga mas nulos. De esa manera cuando saco los outliers voy a tener mayor probabilidades de tambien sacar nulos. Y luego voy a tener menos nulos para reemplazar por la media o la estrategia que decida utilizar para resolverlos. Tambien tengo cierta probabilidad de que se reduzca la cantidad de outliers de otros atributos. De este modo, sin ir a sacar outliers de todos los atributos que graficando considere; lo hacemos escalonado, con la intencion de sacar la menor cantidad de instancias posible.

- Vamos a ver primero la graficacion de boxplot para entender como es la situacion con los outliers.

```
plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.surface_total,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('surface_total',size=14)
plt.title("surface_total" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.surface_covered,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('surface_covered',size=14)
plt.title("surface_covered" ,size=18)

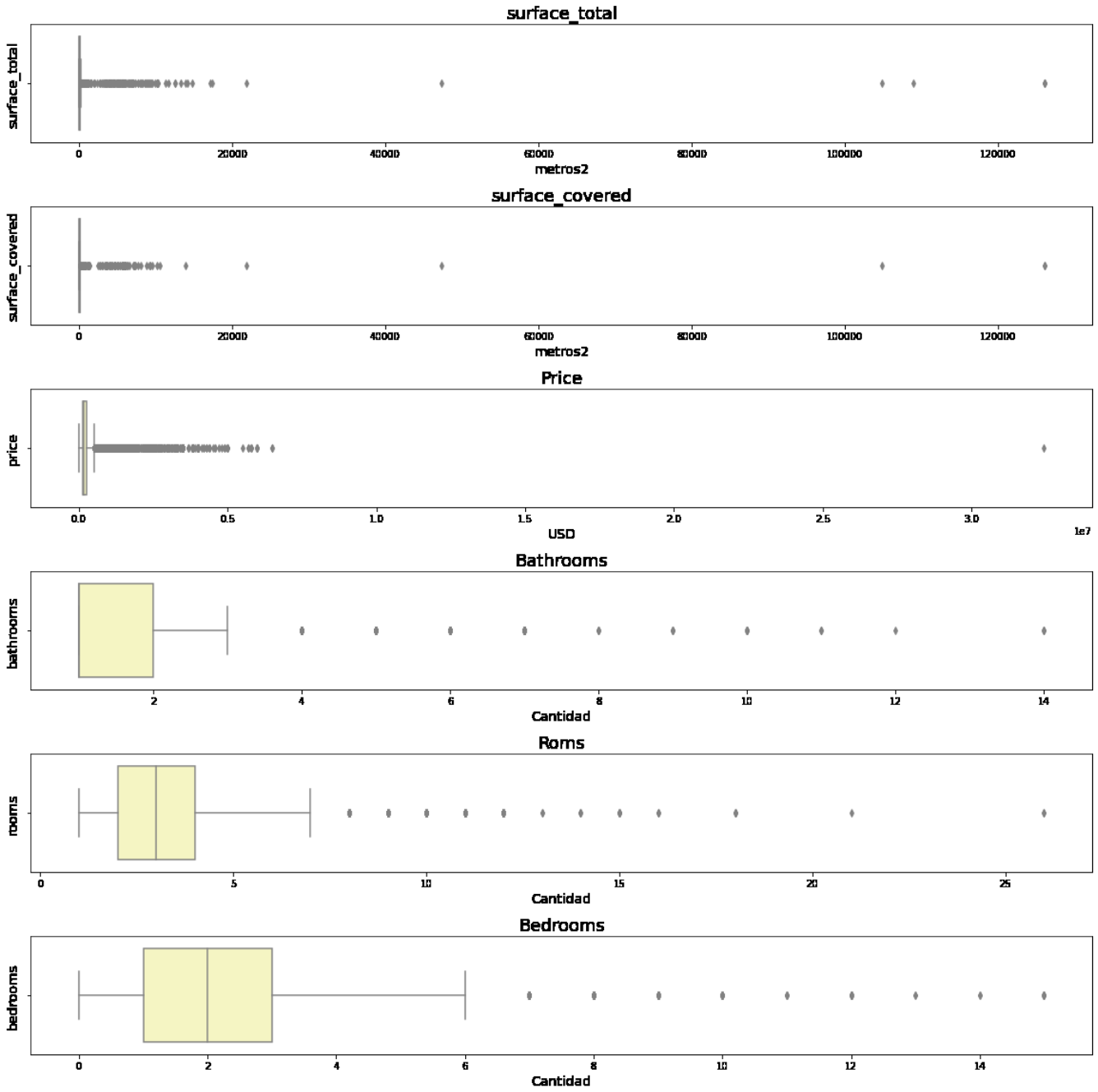
plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.price,palette="Spectral",orient="h")
plt.xlabel('USD',size=14)
plt.ylabel('price',size=14)
plt.title("Price" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.bathrooms,palette="Spectral",orient="h")
plt.xlabel('Cantidad',size=14)
plt.ylabel("bathrooms",size=14)
plt.title("Bathrooms" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.rooms,palette="Spectral",orient="h")
plt.xlabel('Cantidad',size=14)
plt.ylabel('rooms',size=14)
plt.title("Roms" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.bedrooms,palette="Spectral",orient="h")
plt.xlabel('Cantidad',size=14)
plt.ylabel('bedrooms',size=14)
plt.title("Bedrooms" ,size=18)
```

```
Text(0.5, 1.0, 'Bedrooms')
```




```
properati_Capital1[["surface_total","surface_covered","bathrooms"]].isnull().sum()
```

```
surface_total      8917
surface_covered    9235
bathrooms          2163
dtype: int64
```

```
properati_Capital1.corr()
```

	lat	lon	rooms	bedrooms	bathrooms	surface_total	surface_covered	price
lat	1.000000	-0.064840	0.003918	-0.007641	0.098400	-0.002226	-0.000049	0.171750
lon	-0.064840	1.000000	0.064975	0.049852	0.083147	0.000121	0.007947	0.158821
rooms	0.003918	0.064975	1.000000	0.926979	0.601782	0.064767	0.066940	0.449173
bedrooms	-0.007641	0.049852	0.926979	1.000000	0.602327	0.064343	0.066233	0.413548
bathrooms	0.098400	0.083147	0.601782	0.602327	1.000000	0.056131	0.058554	0.587096
surface_total	-0.002226	0.000121	0.064767	0.064343	0.056131	1.000000	0.783820	0.048763
surface_covered	-0.000049	0.007947	0.066940	0.066233	0.058554	0.783820	1.000000	0.054697
price	0.171750	0.158821	0.449173	0.413548	0.587096	0.048763	0.054697	1.000000

- Los 6 atributos tienen outliers, pero los 3 que parecen tener mas son surface_total, surface_covered y price.

```
Q1_surface_total=properati_Capital1.surface_total.quantile(0.25)
Q3_surface_total=properati_Capital1.surface_total.quantile(0.75)

Q1_price=properati_Capital1.price.quantile(0.25)
Q3_price=properati_Capital1.price.quantile(0.75)

Q1_surface_covered=properati_Capital1.surface_covered.quantile(0.25)
Q3_surface_covered=properati_Capital1.surface_covered.quantile(0.75)

Q1_bathrooms=properati_Capital1.bathrooms.quantile(0.25)
Q3_bathrooms=properati_Capital1.bathrooms.quantile(0.75)

Q1_rooms=properati_Capital1.rooms.quantile(0.25)
Q3_rooms=properati_Capital1.rooms.quantile(0.75)

Q1_bedrooms=properati_Capital1.bedrooms.quantile(0.25)
Q3_bedrooms=properati_Capital1.bedrooms.quantile(0.75)

IQR_surface_total=Q3_surface_total-Q1_surface_total
IQR_price=Q3_price-Q1_price
IQR_surface_covered=Q3_surface_covered-Q1_surface_covered
IQR_bathrooms=Q3_bathrooms-Q1_bathrooms
IQR_bedrooms=Q3_bedrooms-Q1_bedrooms
IQR_rooms=Q3_rooms-Q1_rooms

rango_surface_total=IQR_surface_total*1.5
rango_price=IQR_price*1.5
rango_surface_covered=IQR_surface_covered*1.5
rango_bathrooms=IQR_bathrooms*1.5
rango_rooms=IQR_rooms*1.5
rango_bedrooms=IQR_bedrooms*1.5

valla_inferior_price=-(Q1_price-rango_price)
valla_superior_price=Q3_price+rango_price
valla_inferior_surface_total=-(Q1_surface_total-rango_surface_total)
valla_superior_surface_total=Q3_surface_total+rango_surface_total
valla_inferior_surface_covered=-(Q1_surface_covered-rango_surface_covered)
valla_superior_surface_covered=Q3_surface_covered+rango_surface_covered
valla_superior_bathrooms=Q3_bathrooms+rango_bathrooms
valla_inferior_bathrooms=-(Q1_bathrooms-rango_bathrooms)
valla_superior_rooms=Q3_rooms+rango_rooms
valla_inferior_rooms=-(Q1_rooms-rango_rooms)
valla_superior_bedrooms=Q3_bedrooms+rango_bedrooms
valla_inferior_bedrooms=-(Q1_bedrooms-rango_bedrooms)

print("Rango de price: ",valla_inferior_price,valla_superior_price)
print("Rango de surface_total: ",valla_inferior_surface_total,valla_superior_surface_total)
print("Rango de surface_covered: ",valla_inferior_surface_covered,valla_superior_surface_covered)
print("Rango de bathrooms: ",valla_inferior_bathrooms,valla_superior_bathrooms)
print("Rango de rooms: ",valla_inferior_rooms,valla_superior_rooms)
print("Rango de bedrooms: ",valla_inferior_bedrooms,valla_superior_bedrooms)
```

```
Rango de price:  114412.5 515047.5
Rango de surface_total:  44.0 204.0
Rango de surface_covered:  34.0 174.0
Rango de bathrooms:  0.5 3.5
Rango de rooms:  1.0 7.0
Rango de bedrooms:  2.0 6.0
```

- Con el codigo escrito del metodo IQR armo 7 df con las distintas combinaciones posibles. Puedo decidir eliminar outliers solo de uno de los atributos; de dos, de tres, etc.

- En el caso de "bathrooms" redondeo los limites porque tiene que ser un numero entero.

```
new_properati_Capital1_SC=properati_Capital1[(properati_Capital1.surface_covered>valla_inferior_surface_co
vered)&

(properati_Capital1.surface_covered<valla_superior_surface_covered)]

new_properati_Capital1_P=properati_Capital1[(properati_Capital1.price>valla_inferior_price)&
                                        (properati_Capital1.price<valla_superior_price)]
```

```
new_properati_Capital1_ST=properati_Capital1[(properati_Capital1.surface_total>valla_inferior_surface_tota
l)&
                                        (properati_Capital1.surface_total<valla_superior_surface_total)]

new_properati_Capital1_SC_P=properati_Capital1[(properati_Capital1.surface_covered>valla_inferior_surface_
covered)&

(properati_Capital1.surface_covered<valla_superior_surface_covered)&
                                          (properati_Capital1.price>valla_inferior_price)&
                                        (properati_Capital1.price<valla_superior_price)  ]

new_properati_Capital1_SC_ST=properati_Capital1[(properati_Capital1.surface_covered>valla_inferior_surface
_covered)&

(properati_Capital1.surface_covered<valla_superior_surface_covered)&
                                        (properati_Capital1.surface_total>valla_inferior_surface_total)&
                                        (properati_Capital1.surface_total<valla_superior_surface_total) ]

new_properati_Capital1_ST_P=properati_Capital1[(properati_Capital1.price>valla_inferior_price)&
                                        (properati_Capital1.price<valla_superior_price)&
                                        (properati_Capital1.surface_total>valla_inferior_surface_total)&
                                        (properati_Capital1.surface_total<valla_superior_surface_total) ]

new_properati_Capital1_SC_P_ST=properati_Capital1[(properati_Capital1.surface_covered>valla_inferior_surfa
ce_covered)&

(properati_Capital1.surface_covered<valla_superior_surface_covered)&
                                        (properati_Capital1.price>valla_inferior_price)&
                                        (properati_Capital1.price<valla_superior_price) &
                                        (properati_Capital1.surface_total>valla_inferior_surface_total)&
                                        (properati_Capital1.surface_total<valla_superior_surface_total) ]

new_properati_Capital1_ST_P_B_R=properati_Capital1[(properati_Capital1.price>valla_inferior_price)&
                                        (properati_Capital1.price<valla_superior_price)&
                                        (properati_Capital1.surface_total>valla_inferior_surface_total)&
                                        (properati_Capital1.surface_total<valla_superior_surface_total)&
                                        (properati_Capital1.bathrooms>=1)&
                                        (properati_Capital1.bathrooms<=4)&
                                        (properati_Capital1.rooms>=1)&
                                        (properati_Capital1.rooms<=7)]

new_properati_Capital1_All_sin_rooms=properati_Capital1[(properati_Capital1.price>valla_inferior_price)&
                                        (properati_Capital1.price<valla_superior_price)&
                                        (properati_Capital1.surface_total>valla_inferior_surface_total)&
                                        (properati_Capital1.surface_total<valla_superior_surface_total)&

(properati_Capital1.surface_covered>valla_inferior_surface_covered)&

(properati_Capital1.surface_covered<valla_superior_surface_covered)&
                                        (properati_Capital1.bathrooms>=1)&
                                        (properati_Capital1.bathrooms<=4)&
                                        (properati_Capital1.bedrooms>=2)&
                                        (properati_Capital1.bedrooms<=6)]


new_properati_Capital1_All_sin_bedrooms=properati_Capital1[(properati_Capital1.price>valla_inferior_price)
&
                                        (properati_Capital1.price<valla_superior_price)&
                                        (properati_Capital1.surface_total>valla_inferior_surface_total)&
                                        (properati_Capital1.surface_total<valla_superior_surface_total)&

(properati_Capital1.surface_covered>valla_inferior_surface_covered)&

(properati_Capital1.surface_covered<valla_superior_surface_covered)&
                                        (properati_Capital1.bathrooms>=1)&
                                        (properati_Capital1.bathrooms<=4)&
                                        (properati_Capital1.rooms>=1)&
                                        (properati_Capital1.rooms<=7)]

new_properati_Capital1_All=properati_Capital1[(properati_Capital1.price>valla_inferior_price)&
                                        (properati_Capital1.price<valla_superior_price)&
                                        (properati_Capital1.surface_total>valla_inferior_surface_total)&
```

```
                                        (properati_Capital1.surface_total<valla_superior_surface_total)&

(properati_Capital1.surface_covered>valla_inferior_surface_covered)&

(properati_Capital1.surface_covered<valla_superior_surface_covered)&
                                        (properati_Capital1.bathrooms>=1)&
                                        (properati_Capital1.bathrooms<=4)&
                                        (properati_Capital1.rooms>=1)&
                                        (properati_Capital1.rooms<=7)&
                                        (properati_Capital1.bedrooms>=2)&
                                        (properati_Capital1.bedrooms<=6)]
```

- Veo las columnas que tiene cada uno de estos df.

```
print("properati_Capital1",properati_Capital1.shape)
print("new_properati_Capital1_SC",new_properati_Capital1_SC.shape)
print("new_properati_Capital1_P",new_properati_Capital1_P.shape)
print("new_properati_Capital1_ST",new_properati_Capital1_ST.shape)
print("new_properati_Capital1_SC_P",new_properati_Capital1_SC_P.shape)
print("new_properati_Capital1_SC_ST",new_properati_Capital1_SC_ST.shape)
print("new_properati_Capital1_ST_P",new_properati_Capital1_ST_P.shape)
print("new_properati_Capital1_SC_P_ST",new_properati_Capital1_SC_P_ST.shape)
print("new_properati_Capital1_ST_P_B_R",new_properati_Capital1_ST_P_B_R.shape)
print("new_properati_Capital1_All_sin_rooms",new_properati_Capital1_All_sin_rooms.shape)
print("new_properati_Capital1_All_sin_bedrooms",new_properati_Capital1_All_sin_bedrooms.shape)
print("new_properati_Capital1_All",new_properati_Capital1_All.shape)
```

```
properati_Capital1 (90725, 19)
new_properati_Capital1_SC (69276, 19)
new_properati_Capital1_P (64495, 19)
new_properati_Capital1_ST (60662, 19)
new_properati_Capital1_SC_P (55340, 19)
new_properati_Capital1_SC_ST (58972, 19)
new_properati_Capital1_ST_P (51566, 19)
new_properati_Capital1_SC_P_ST (50665, 19)
new_properati_Capital1_ST_P_B_R (50921, 19)
new_properati_Capital1_All_sin_rooms (37204, 19)
new_properati_Capital1_All_sin_bedrooms (50112, 19)
new_properati_Capital1_All (37178, 19)
```

- Veo los nulos que cada df tiene.

```
print("Los nulos en cada df son los siguientes:\n")
print("properati_Capital1\n",properati_Capital1[["surface_total","surface_covered","bathrooms"]].isnull().
sum())
print("new_properati_Capital1_SC\n",new_properati_Capital1_SC[["surface_total","surface_covered","bathroom
s"]].isnull().sum())
print("new_properati_Capital1_P\n",new_properati_Capital1_P[["surface_total","surface_covered","bathrooms"
]].isnull().sum())
print("new_properati_Capital1_ST\n",new_properati_Capital1_ST[["surface_total","surface_covered","bathroom
s"]].isnull().sum())
print("new_properati_Capital1_SC_P\n",new_properati_Capital1_SC_P[["surface_total","surface_covered","bath
rooms"]].isnull().sum())
print("new_properati_Capital1_SC_ST\n",new_properati_Capital1_SC_ST[["surface_total","surface_covered","ba
throoms"]].isnull().sum())
print("new_properati_Capital1_ST_P\n",new_properati_Capital1_ST_P[["surface_total","surface_covered","bath
rooms"]].isnull().sum())
print("new_properati_Capital1_SC_P_ST\n",new_properati_Capital1_SC_P_ST[["surface_total","surface_covered"
,"bathrooms"]].isnull().sum())
print("new_properati_Capital1_ST_P_B_R\n",new_properati_Capital1_ST_P_B_R[["surface_total","surface_covere
d","bathrooms"]].isnull().sum())
print("new_properati_Capital1_All_sin_rooms\n",new_properati_Capital1_All_sin_rooms[["surface_total","surf
ace_covered","bathrooms"]].isnull().sum())
print("new_properati_Capital1_All_sin_bedrooms\n",

new_properati_Capital1_All_sin_bedrooms[["surface_total","surface_covered","bathrooms"]].isnull().sum())
print("new_properati_Capital1_All\n",new_properati_Capital1_All[["surface_total","surface_covered","bathro
oms"]].isnull().sum())
```

```
Los nulos en cada df son los siguientes:

properati_Capital1
 surface_total      8917
surface_covered    9235
bathrooms          2163
dtype: int64
new_properati_Capital1_SC
 surface_total      125
surface_covered      0
bathrooms          676
dtype: int64
new_properati_Capital1_P
 surface_total      5790
surface_covered    5971
bathrooms          1326
dtype: int64
new_properati_Capital1_ST
 surface_total        0
surface_covered    366
bathrooms          583
dtype: int64
new_properati_Capital1_SC_P
 surface_total       91
surface_covered      0
bathrooms          501
dtype: int64
new_properati_Capital1_SC_ST
 surface_total        0
surface_covered      0
…  (salida recortada para el PDF)  …
new_properati_Capital1_All_sin_bedrooms
 surface_total      0
surface_covered    0
bathrooms          0
dtype: int64
new_properati_Capital1_All
 surface_total      0
surface_covered    0
bathrooms          0
dtype: int64
```

> Fui probando desde el df que menos instancias elimina y fui revisando comportamiento de nulos y distribucion de todos los boxplots antes de seguir con el siguiente df.
>
> - Me quede con el df que tiene la mejor combinacion entre:

- La menor **perdida de instancias** posible.
- La mayor **perdida de nulos** posible.
- **Graficas** de boxplots y distplot que permita ver una razonable comportamiento.

> - Es muy significativa la perdida de instancias que se da cuando se sacan los outliers calculados por el IQR incluyendo todas las variables numericas CON "bedrooms"; eso se ve en la comparacion entre los df (new_properati_Capital1_All_sin_rooms) y (new_properati_Capital1_All_sin_bedrooms).

- En el df (new_properati_Capital1_All_sin_rooms) la **perdida de instancias** con respecto a (properati_Capital1) es de un **55%.**
- En el df (new_properati_Capital1_All_sin_bedrooms) la **perdida de instancias** con respecto a (properati_Capital1) es de un **41%.**
- En ambos df con la eliminacion de los outliers tambien estamos eliminando **TODOS los nulos.**

- Por todo esto el df que selecciono es (new_properati_Capital1_All_sin_bedrooms).

```
plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.surface_total,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('surface_total',size=14)
plt.title("surface_total" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.surface_covered,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('surface_covered',size=14)
plt.title("surface_covered" ,size=18)

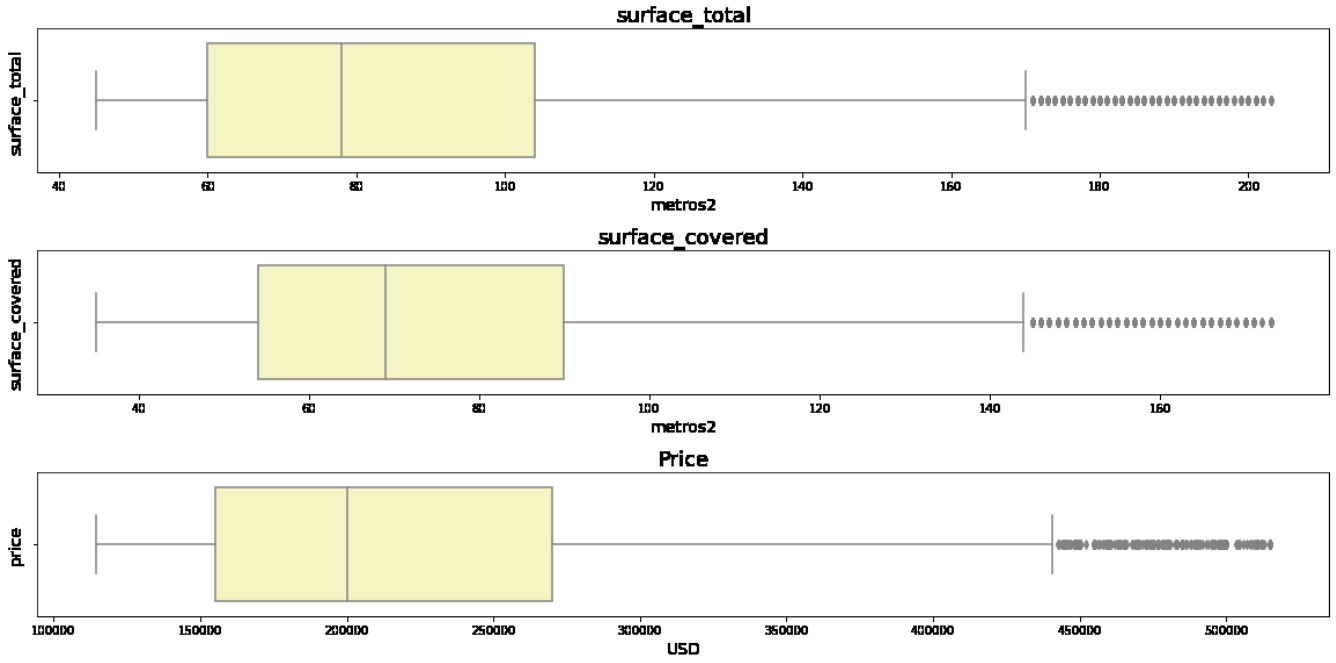
plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.price,palette="Spectral",orient="h")
plt.xlabel('USD',size=14)
plt.ylabel('price',size=14)
plt.title("Price" ,size=18)

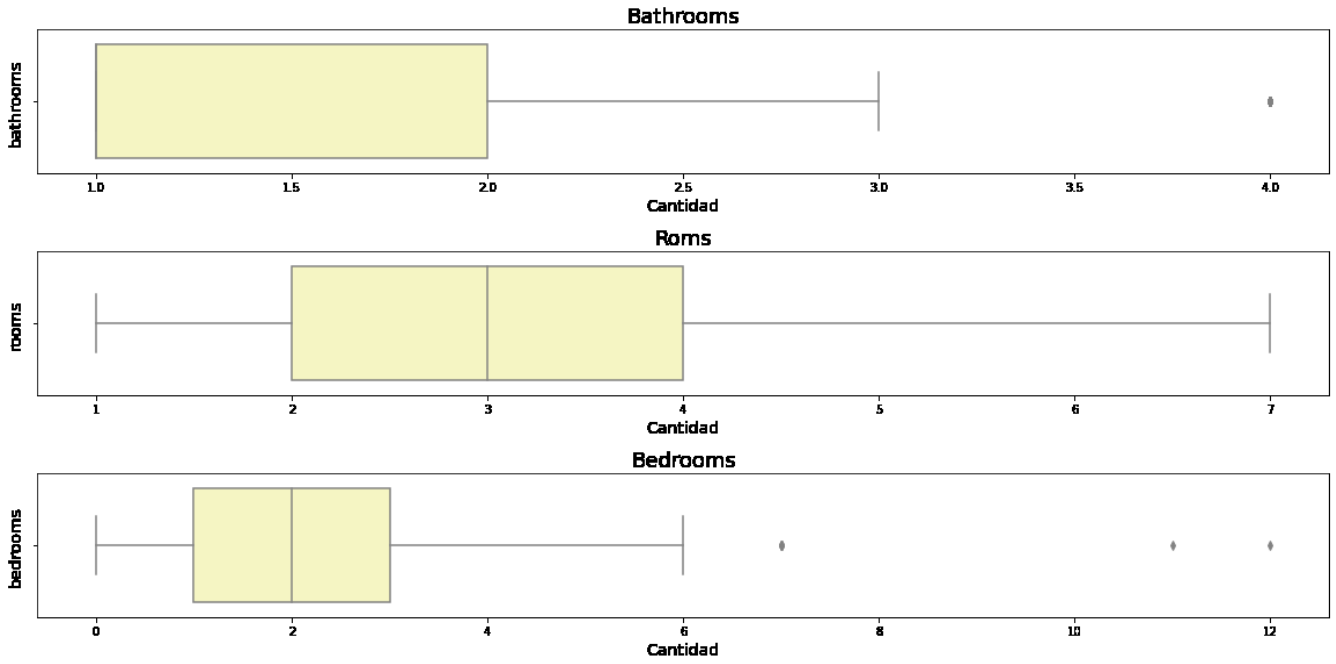
plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.bathrooms,palette="Spectral",orient="h")
plt.xlabel('Cantidad',size=14)
plt.ylabel("bathrooms",size=14)
plt.title("Bathrooms" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.rooms,palette="Spectral",orient="h")
plt.xlabel('Cantidad',size=14)
plt.ylabel('rooms',size=14)
plt.title("Roms" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.bedrooms,palette="Spectral",orient="h")
plt.xlabel('Cantidad',size=14)
plt.ylabel('bedrooms',size=14)
plt.title("Bedrooms" ,size=18)
```

```
Text(0.5, 1.0, 'Bedrooms')
```



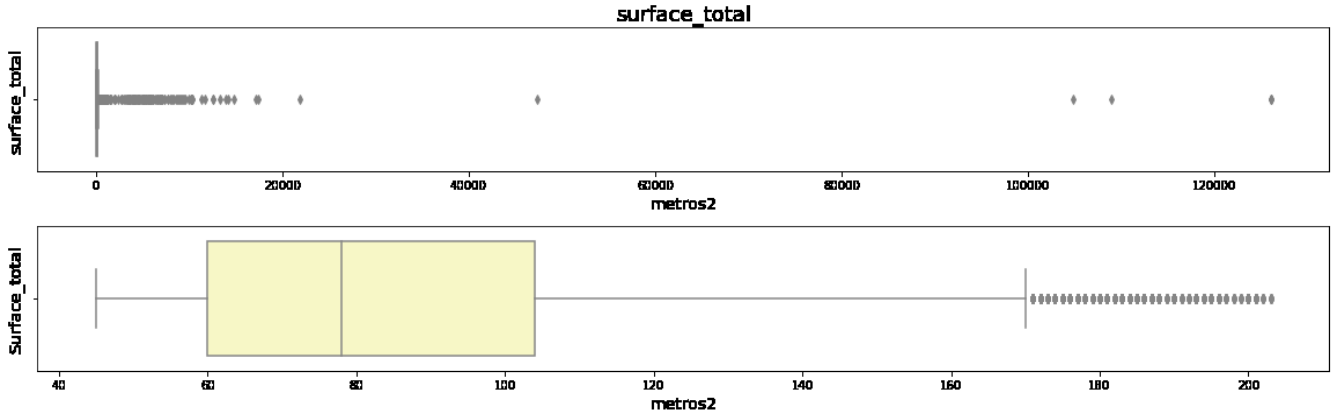


- Grafico para que se vea mas claramente la mejora en la distribucion luego de eliminar outliers.

```
plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.surface_total,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('surface_total',size=14)
plt.title("surface_total" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,2)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.surface_total,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('Surface_total',size=14)
```

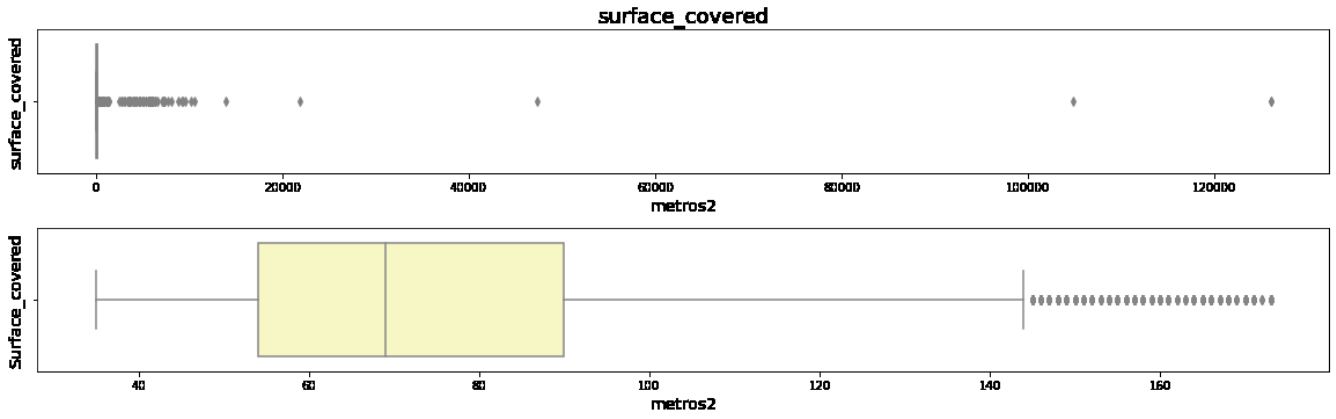
```
Text(0, 0.5, 'Surface_total')
```




```
plt.subplots(figsize=(20, 5))
plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.surface_covered,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('surface_covered',size=14)
plt.title("surface_covered" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,2)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.surface_covered,palette="Spectral",orient="h")
plt.xlabel('metros2',size=14)
plt.ylabel('Surface_covered',size=14)
```

```
Text(0, 0.5, 'Surface_covered')
```

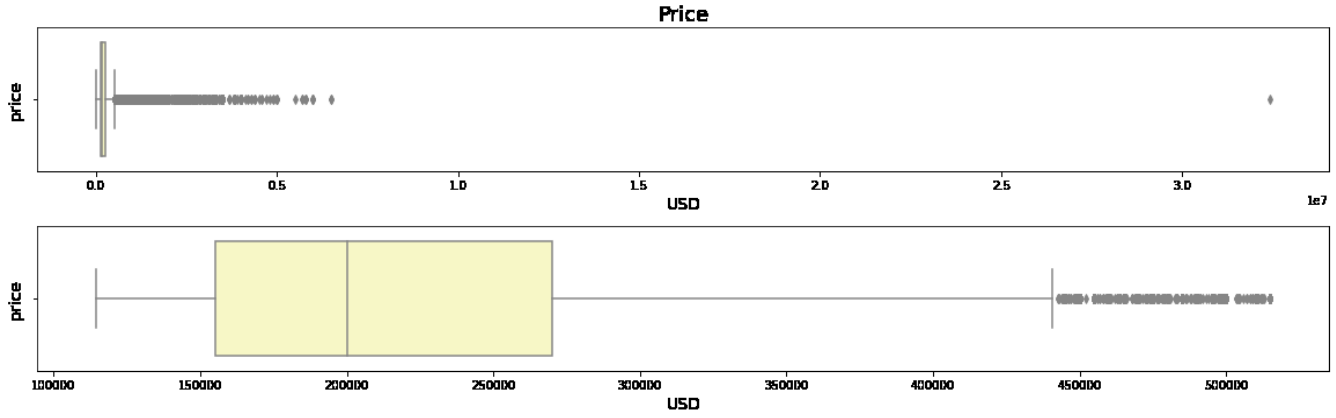



```
plt.subplots(figsize=(20, 5))

plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.price,palette="Spectral",orient="h")
plt.ylabel("price",size=14)
plt.xlabel('USD',size=14)
plt.title("Price" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,2)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.price,palette="Spectral",orient="h")
plt.xlabel('USD',size=14)
plt.ylabel('price',size=14)
```

```
Text(0, 0.5, 'price')
```

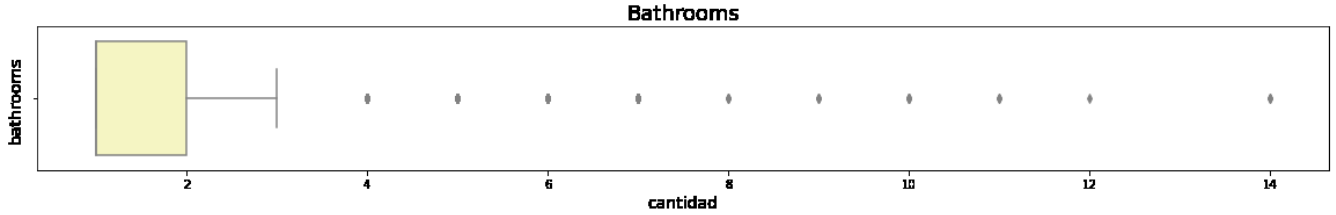


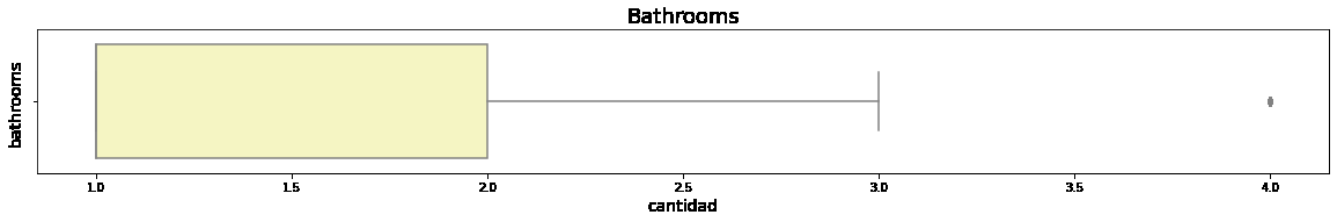

```
plt.subplots(figsize=(20, 5))

plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.bathrooms,palette="Spectral",orient="h")
plt.ylabel("bathrooms",size=14)
plt.xlabel('cantidad',size=14)
plt.title("Bathrooms" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,2)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.bathrooms,palette="Spectral",orient="h")
plt.ylabel("bathrooms",size=14)
plt.xlabel('cantidad',size=14)
plt.title("Bathrooms" ,size=18)
```

```
Text(0.5, 1.0, 'Bathrooms')
```



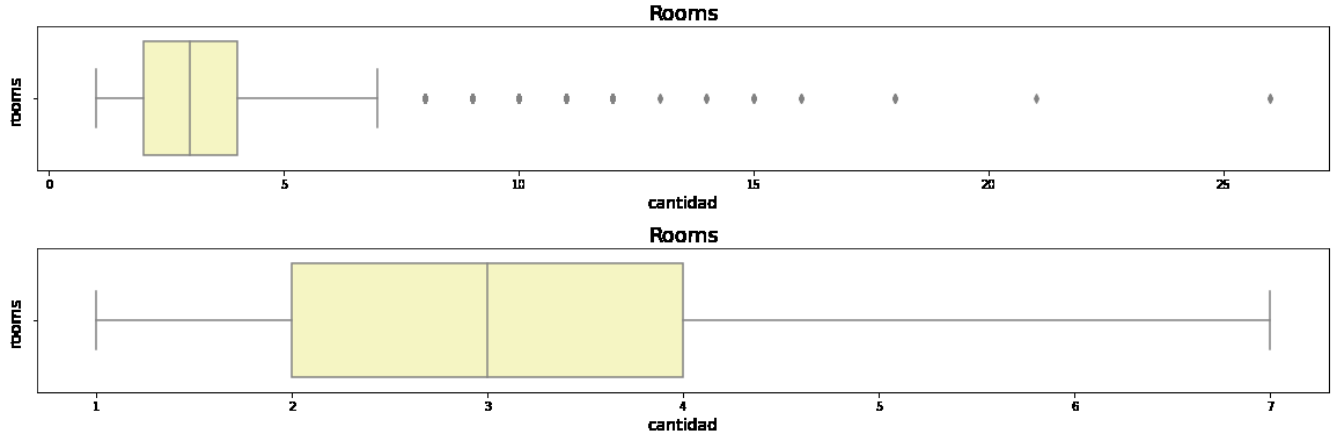



```
plt.subplots(figsize=(20, 5))

plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.rooms,palette="Spectral",orient="h")
plt.ylabel("rooms",size=14)
plt.xlabel('cantidad',size=14)
plt.title("Rooms" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,2)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.rooms,palette="Spectral",orient="h")
plt.ylabel("rooms",size=14)
plt.xlabel('cantidad',size=14)
plt.title("Rooms" ,size=18)
```

```
Text(0.5, 1.0, 'Rooms')
```

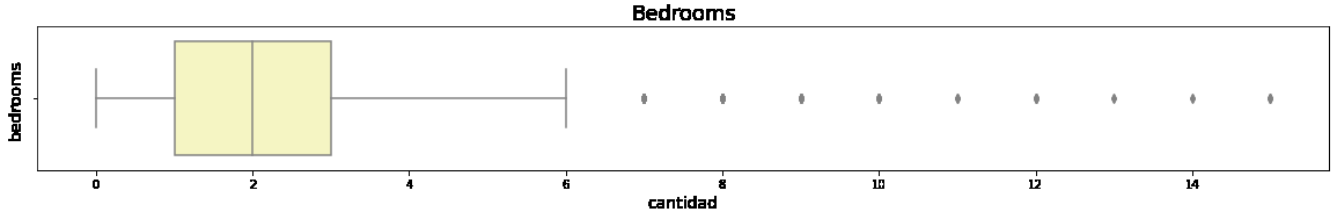


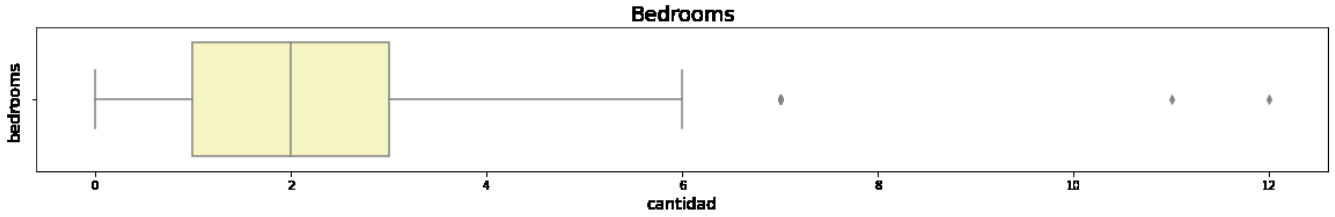

```
plt.subplots(figsize=(20, 5))

plt.subplot(2,1,1)
sns.boxplot(properati_Capital1.bedrooms,palette="Spectral",orient="h")
plt.ylabel("bedrooms",size=14)
plt.xlabel('cantidad',size=14)
plt.title("Bedrooms" ,size=18)

plt.subplots(figsize=(20, 5))
plt.subplot(2,1,2)
sns.boxplot(new_properati_Capital1_All_sin_bedrooms.bedrooms,palette="Spectral",orient="h")
plt.ylabel("bedrooms",size=14)
plt.xlabel('cantidad',size=14)
plt.title("Bedrooms" ,size=18)
```

```
Text(0.5, 1.0, 'Bedrooms')
```





2.3 · Valores nulos

- No tengo nulos para eliminar.

2.4 · Imputacion de Valores Faltantes

- Al no tener nulos no tengo valores faltantes a imputar.

- Vemos la distribucion de los atributos (new_properati_Capital1_All_sin_bedrooms).

```
plt.subplots(figsize=(20,17))

plt.subplot(6,1,1)
plt.title("surface_total" ,size=18)
sns.distplot(new_properati_Capital1_All_sin_bedrooms ["surface_total"],bins=100)
plt.xlabel('metros2',size=14)
plt.ylabel('surface_total',size=14)

plt.subplots(figsize=(20,17))
plt.subplot(6,1,2)
plt.title("surface_covered" ,size=18)
sns.distplot(new_properati_Capital1_All_sin_bedrooms["surface_covered"],bins=100)
plt.xlabel('metros2',size=14)
plt.ylabel('surface_covered',size=14)

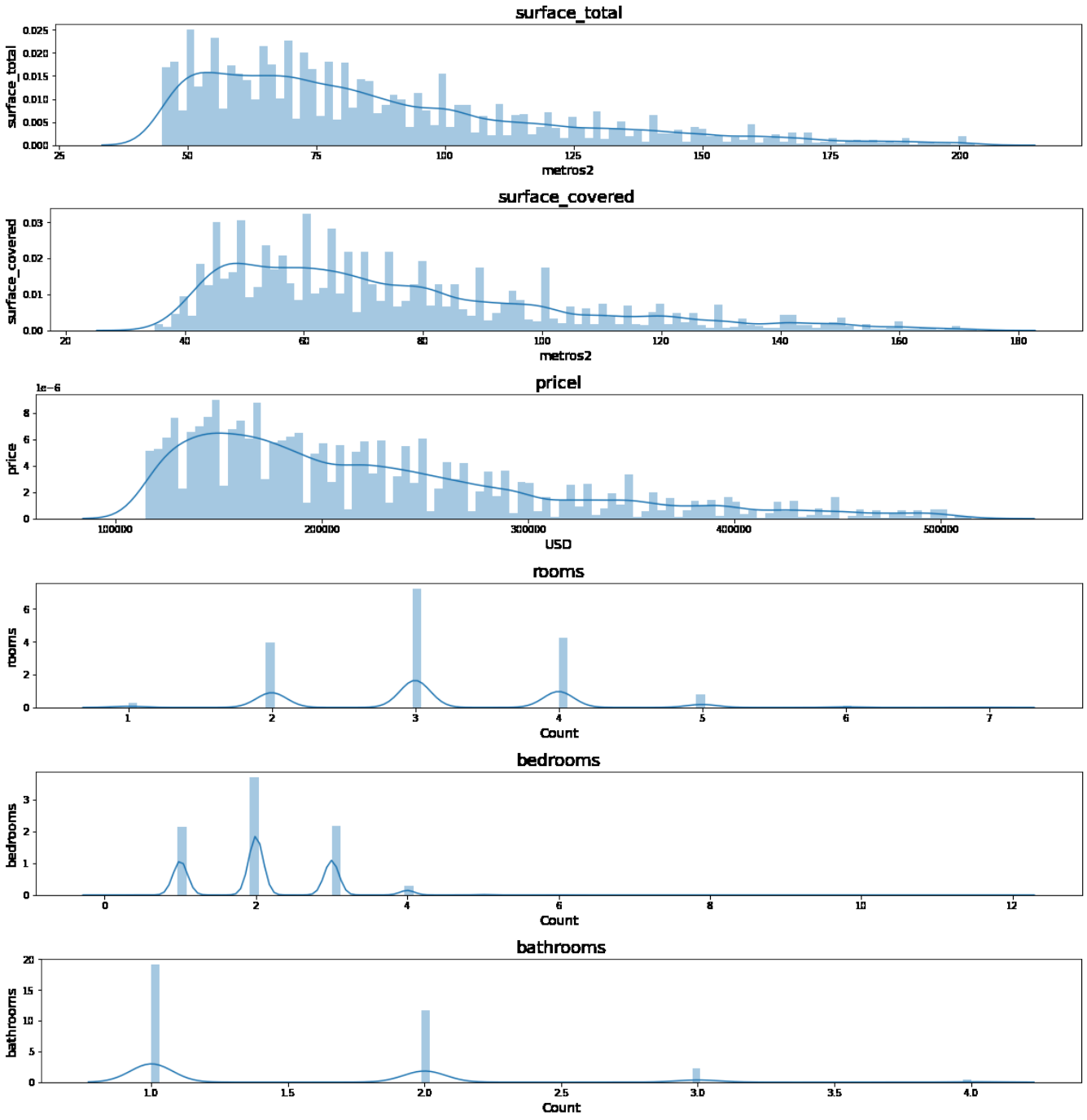
plt.subplots(figsize=(20,17))
plt.subplot(6,1,3)
plt.title("pricel" ,size=18)
sns.distplot(new_properati_Capital1_All_sin_bedrooms ["price"],bins=100)
plt.xlabel('USD',size=14)
plt.ylabel('price',size=14)

plt.subplots(figsize=(20,17))
plt.subplot(6,1,4)
plt.title("rooms" ,size=18)
sns.distplot(new_properati_Capital1_All_sin_bedrooms["rooms"],bins=100)
plt.xlabel('Count',size=14)
plt.ylabel('rooms',size=14)

plt.subplots(figsize=(20,17))
plt.subplot(6,1,5)
plt.title("bedrooms" ,size=18)
sns.distplot(new_properati_Capital1_All_sin_bedrooms["bedrooms"],bins=100)
plt.xlabel('Count',size=14)
plt.ylabel('bedrooms',size=14)

plt.subplots(figsize=(20,17))
plt.subplot(6,1,6)
plt.title("bathrooms" ,size=18)
sns.distplot(new_properati_Capital1_All_sin_bedrooms["bathrooms"],bins=100)
plt.xlabel('Count',size=14)
plt.ylabel('bathrooms',size=14)
```

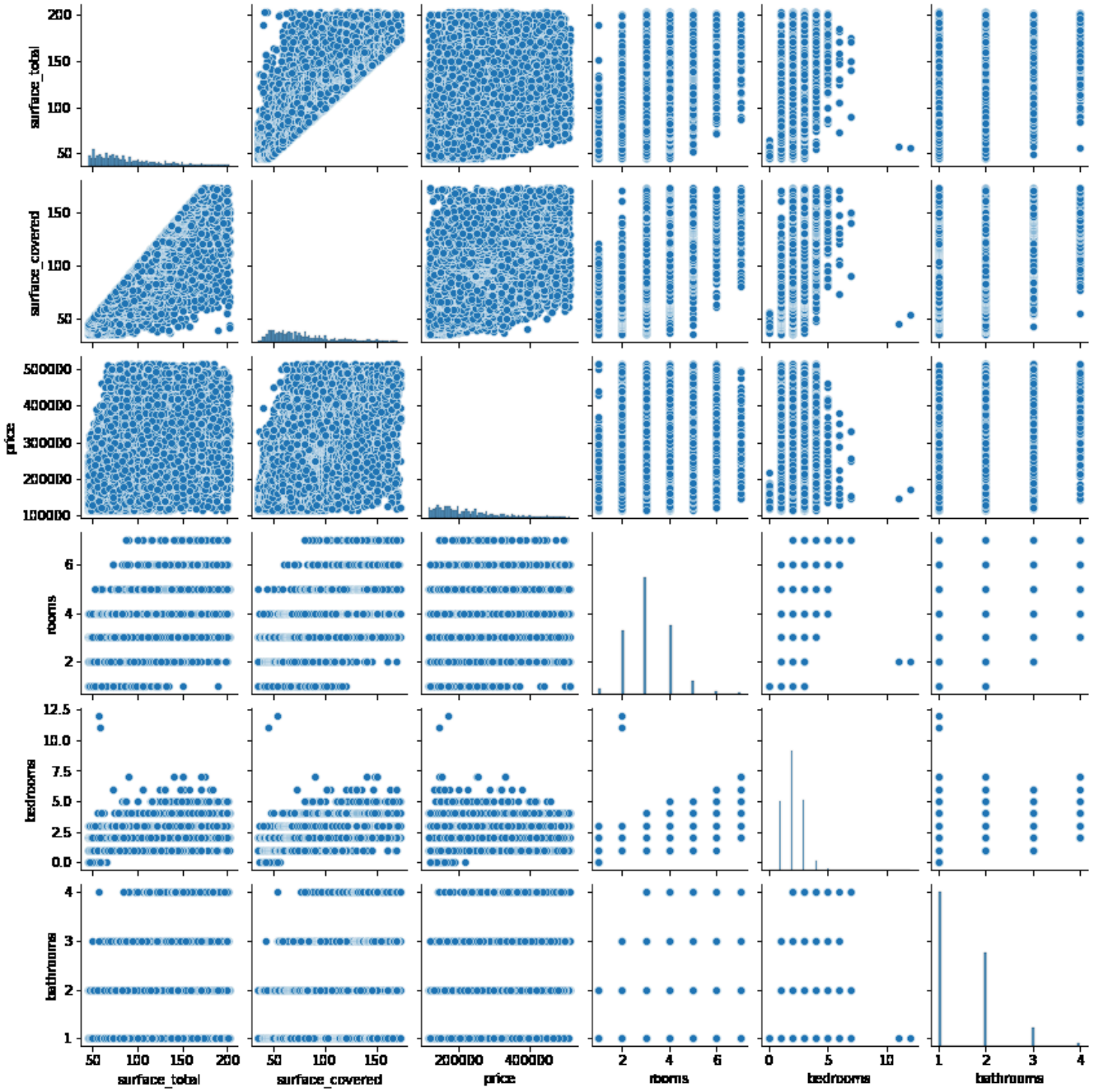
```
Text(0, 0.5, 'bathrooms')
```



- Las distribuciones son normales, asimetricas y positivas en las variables surface_total, surface_covered y price.

```
sns.pairplot(new_properati_Capital1_All_sin_bedrooms[["surface_total","surface_covered","price","rooms","bedrooms", "bathrooms"]],height=2)
```

```
<seaborn.axisgrid.PairGrid at 0x182367e2f40>
```




```
new_properati_Capital1_All_sin_bedrooms[["surface_total","surface_covered",
"bathrooms","price","rooms","bedrooms"]].corr().round(2)
```

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms
surface_total	1.00	0.89	0.52	0.62	0.67	0.64
surface_covered	0.89	1.00	0.57	0.66	0.74	0.73
bathrooms	0.52	0.57	1.00	0.55	0.45	0.45
price	0.62	0.66	0.55	1.00	0.43	0.41
rooms	0.67	0.74	0.45	0.43	1.00	0.94
bedrooms	0.64	0.73	0.45	0.41	0.94	1.00

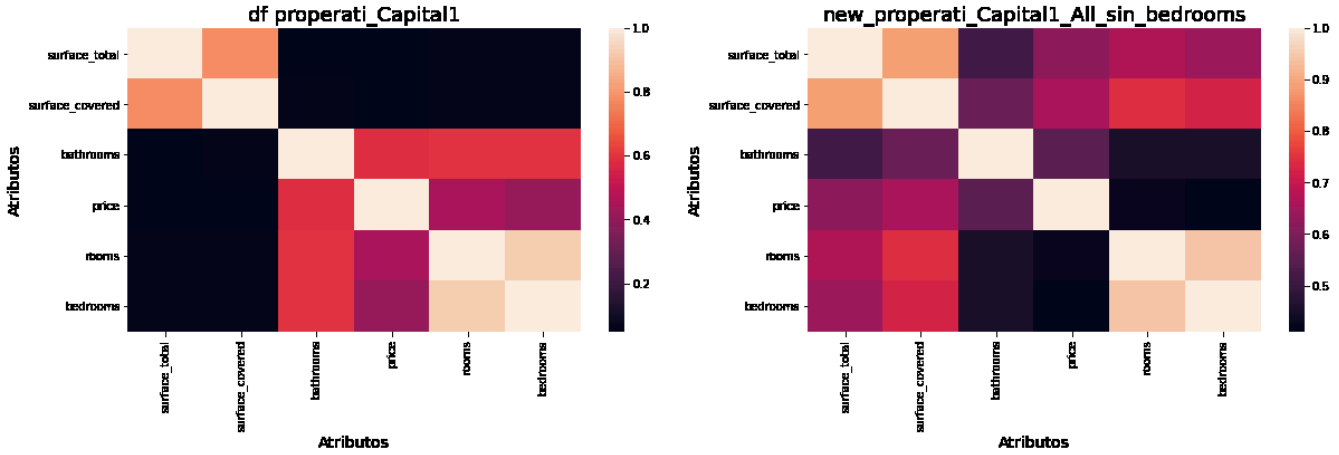
```
plt.subplots(figsize=(20,5))

plt.subplot(1,2,1)
sns.heatmap(properati_Capital1[["surface_total","surface_covered",
"bathrooms","price","rooms","bedrooms"]].corr())
plt.ylabel("Atributos",size=14)
plt.xlabel("Atributos",size=14)
plt.title("df properati_Capital1" ,size=18)

#plt.subplots(figsize=(5,5))
```

```
plt.subplot(1,2,2)
sns.heatmap(new_properati_Capital1_All_sin_bedrooms[["surface_total","surface_covered",
"bathrooms","price","rooms","bedrooms"]].corr())
plt.xlabel("Atributos",size=14)
plt.ylabel("Atributos",size=14)
plt.title("new_properati_Capital1_All_sin_bedrooms" ,size=18)
```

```
Text(0.5, 1.0, 'new_properati_Capital1_All_sin_bedrooms')
```



- Mejoraron mucho los valores de correlaccion despues de eliminar los outliers.

- rooms/surface_total de 0.06 a 0.67
- rooms/surface_covered de 0.07 a 0.74
- bedrooms/surface_total de 0.06 a 0.65
- bedrooms/surface_covered de 0.07 a 0.73
- bathrooms/surface_total de 0.06 a 0.52
- bathrooms/surface_covered de 0.06 a 0.56
- surface_total/surface_covered de 0.78 a 0.89
- surface_total/price de 0.05 a 0,62
- surface_covered/price de 0.05 a 0.66

-Las variables con mayor correlacion **en general** son de mayor a menor:

> 1- surface_total/surface_covered 0.89
> 2- rooms/bedrooms 0.94
> 3- rooms/surface_covered 0.74

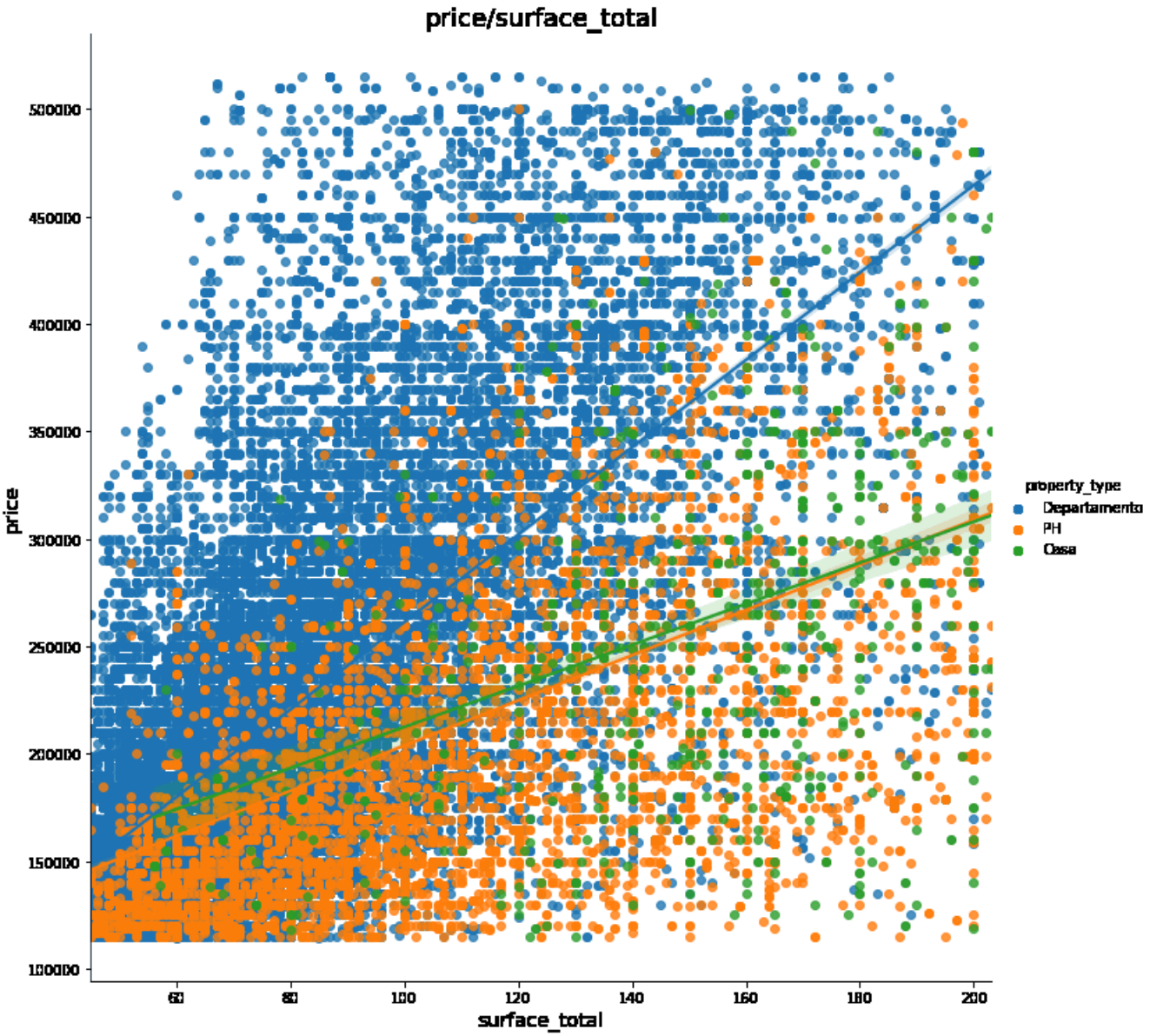
-Las variables con mayor correlacion con la **variable price** son de mayor a menor:

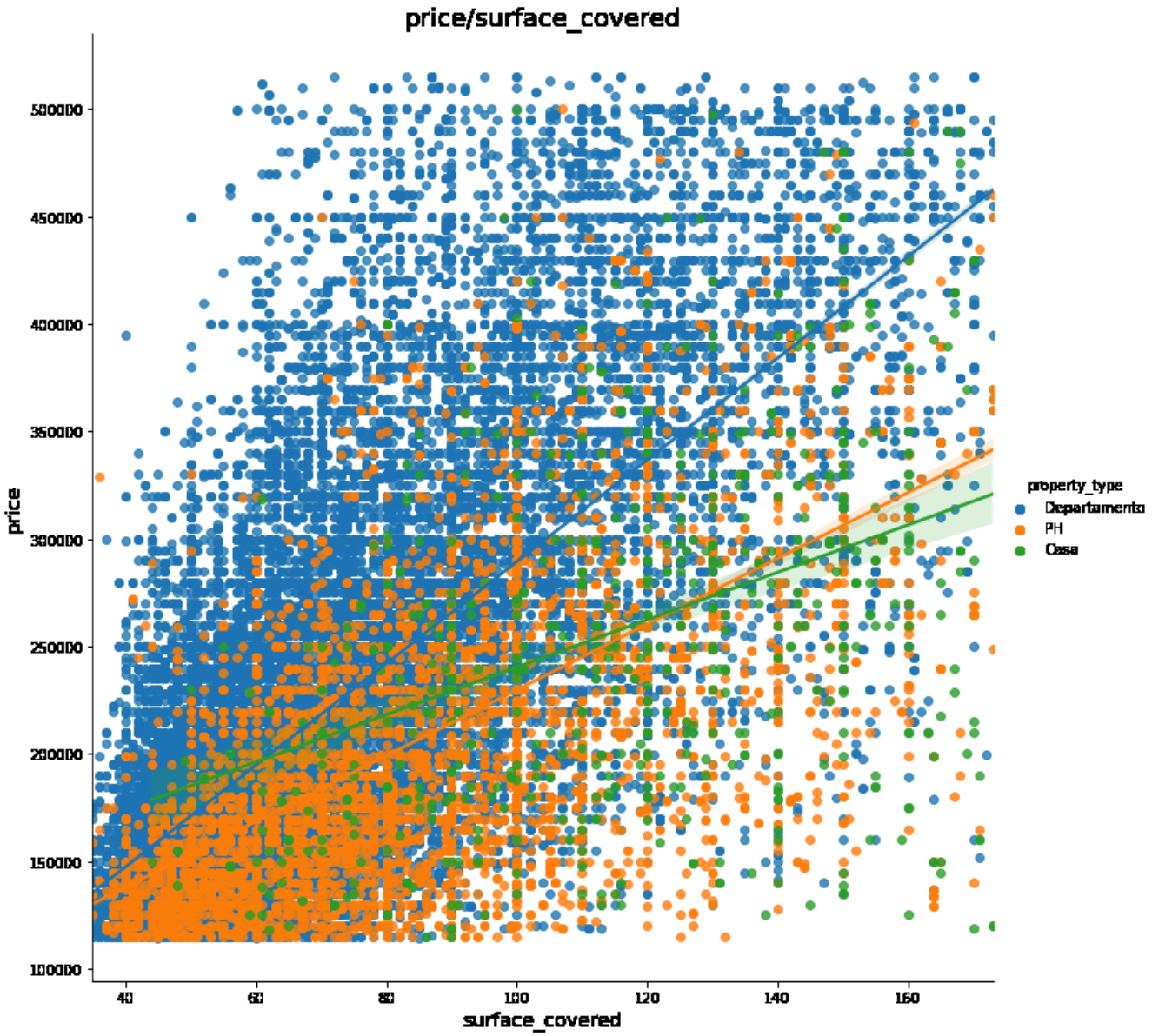
> 1- surface_covered/price 0,66
> 2- surface_total/price 0.62

```
g=sns.lmplot(x="surface_total", y="price",
data=new_properati_Capital1_All_sin_bedrooms,height=10,hue="property_type")
plt.ylabel("price",size=14)
plt.xlabel('surface_total',size=14)
plt.title("price/surface_total" ,size=18)

g=sns.lmplot(x="surface_covered", y="price",
data=new_properati_Capital1_All_sin_bedrooms,height=10,hue="property_type")
plt.ylabel("price",size=14)
plt.xlabel('surface_covered',size=14)
plt.title("price/surface_covered" ,size=18)
```

```
Text(0.5, 1.0, 'price/surface_covered')
```


price/surface_total
price
surface_total
500000
450000
400000
350000
300000
250000
200000
150000
100000
60
80
100
120
140
160
180
200
property_type
Departamento
PH
Casa



-En el grafico **surface_total/price** la regresion lineal del tipo de propiedad Casa y PH son identicas, tanto en pendiente como en ordenada al origen. La ordenada al origen de las tres lineas practicamente coinciden.

-En el grafico **surface_covered/price** la regresion lineal del tipo de propiedad Casa y PH son muy similares. Las pendientes de PH y Casa son similares. La de departamentos tiene una pendiente mayor. La ordenada al origen es practicamente identica entre departamento y Ph; siendo mas elevada en tipo de propiedad Casa.

2.4 · Escalado de Datos

```
#new_properati_Capital1_All_sin_bedrooms.to_csv('new_properati_Capital1_All_sin_bedrooms.csv')
```

- Primero voy a limpiar el df (new_properati_Capital1_All_sin_bedrooms), de los atributos que no voy a utilizar mas.

```
new_properati_Capital1_All_sin_bedrooms_clean=pd.DataFrame(new_properati_Capital1_All_sin_bedrooms)
display(new_properati_Capital1_All_sin_bedrooms_clean.l1.unique())
display(new_properati_Capital1_All_sin_bedrooms_clean.l2.unique())

#confirmo que en atributo l1 y l2 solo hay una variable categorica. En l1 "Argentina" y en l2 "Capital
```

```
Federal".
#Tambien las elimino del df para no ocupar lugar.

new_properati_Capital1_All_sin_bedrooms_clean=new_properati_Capital1_All_sin_bedrooms_clean.drop(["l1","l2
","start_date", "end_date","created_on","lat","lon","currency",
                                                                    "title", "description",
"operation_type"], axis=1)

display(new_properati_Capital1_All_sin_bedrooms_clean.head(2))
new_properati_Capital1_All_sin_bedrooms_clean.shape
```

```
array(['Argentina'], dtype=object)
array(['Capital Federal'], dtype=object)
```

	l3	rooms	bedrooms	bathrooms	surface_total	surface_covered	price	property_type
0	San Cristobal	7.0	7.0	2.0	140.0	140.0	153000.0	Departamento
1	Boedo	2.0	1.0	2.0	70.0	58.0	159000.0	PH

```
(50112, 8)
```

```
#new_properati_Capital1_All_sin_bedrooms_clean.to_csv('new_properati_Capital1_All_sin_bedrooms_clean.csv')
```

- Grafico para ver como estan las correlaciones y la distribucion en un jointplot. Verificare luego si cambio luego de realizar el escalamiento de los datos.

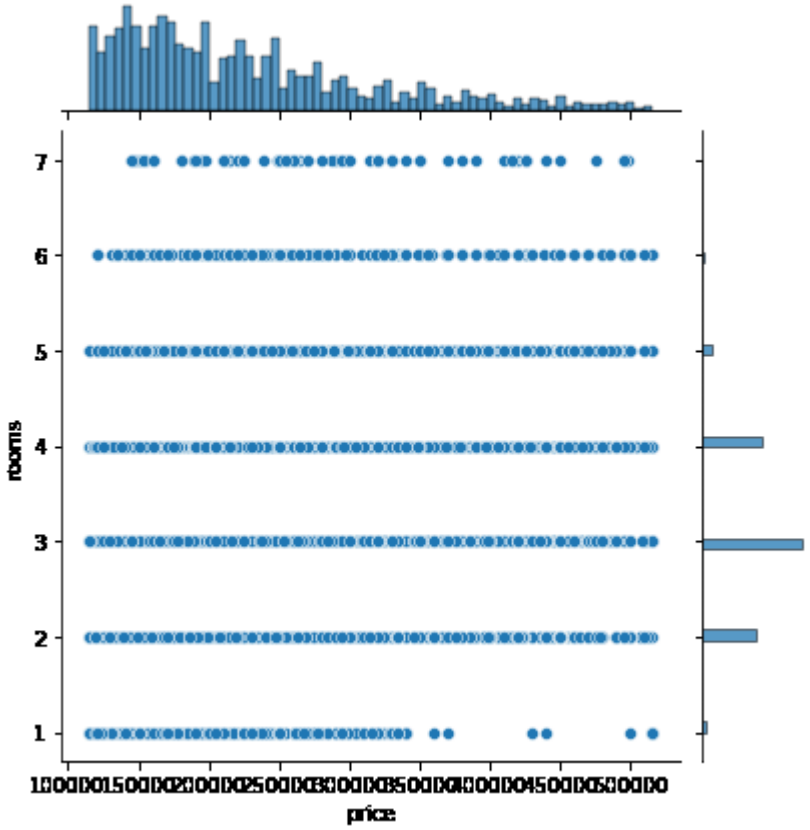
```
sns.jointplot(y="rooms", x="price",data=new_properati_Capital1_All_sin_bedrooms_clean)

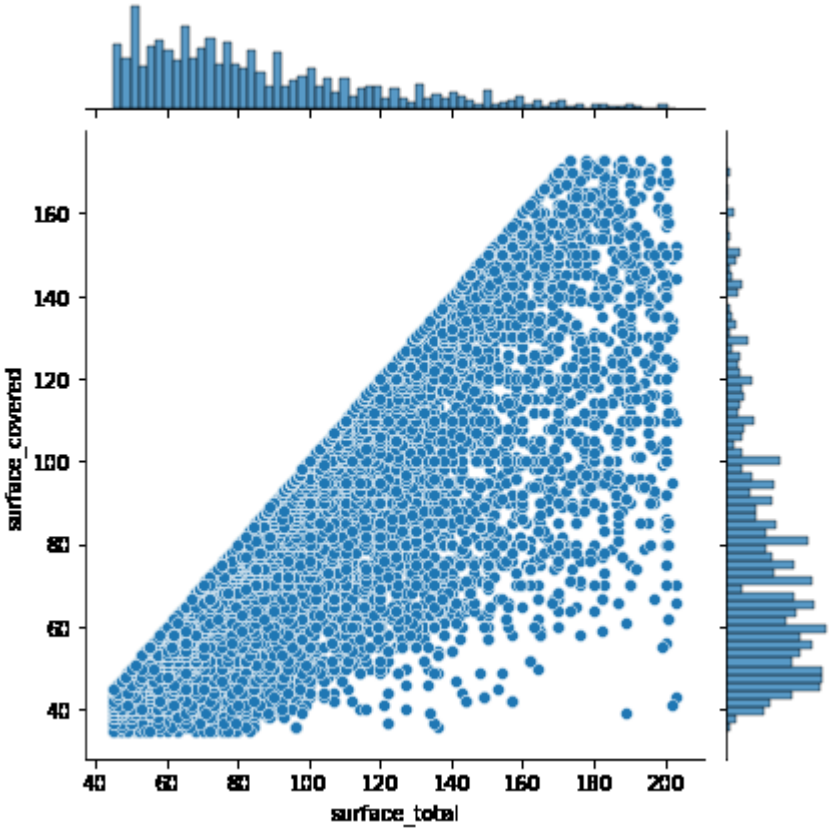
sns.jointplot(y="surface_covered", x="surface_total",data=new_properati_Capital1_All_sin_bedrooms_clean)

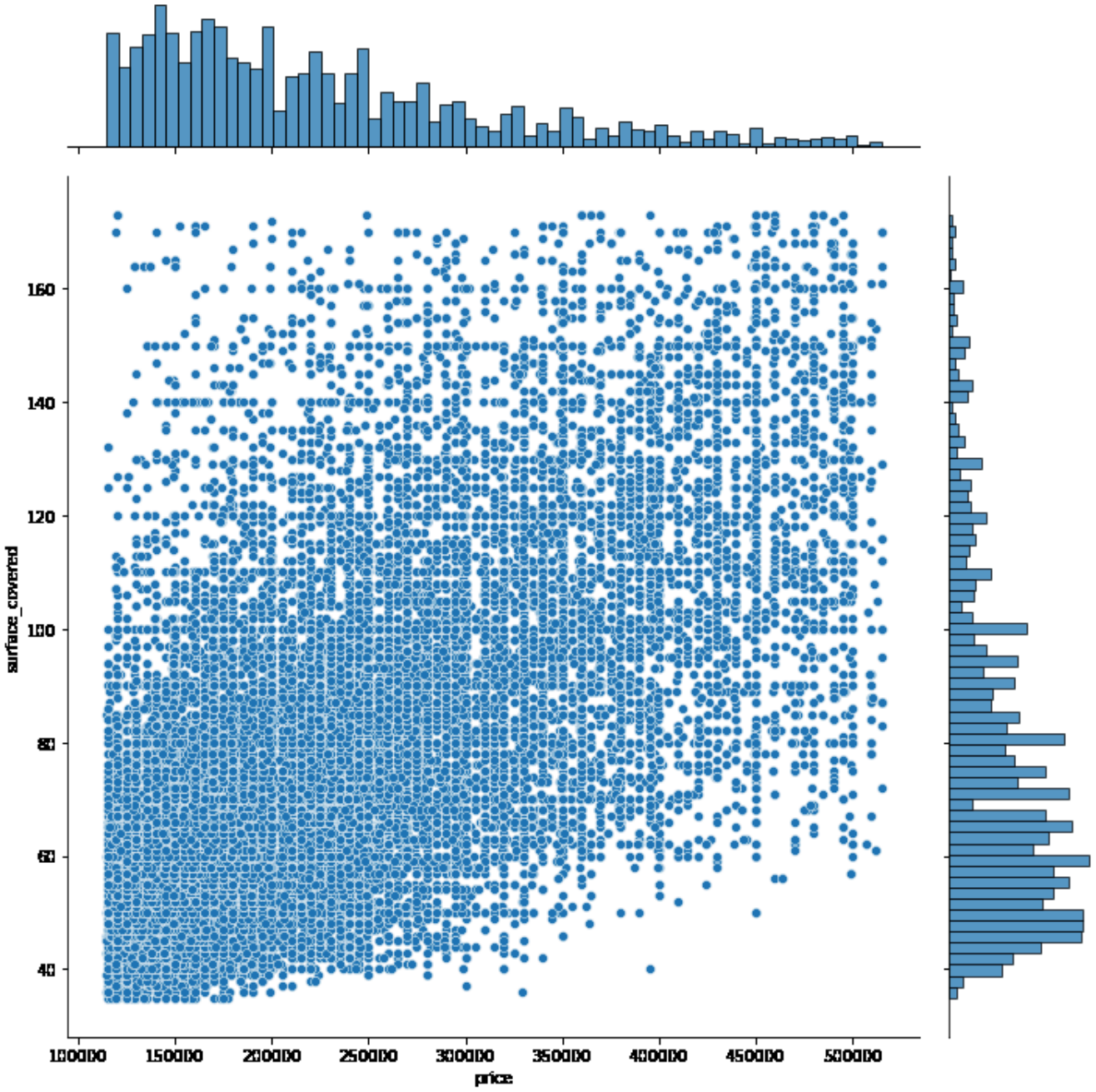
sns.jointplot(y="surface_covered", x="price",data=new_properati_Capital1_All_sin_bedrooms_clean,height=10)

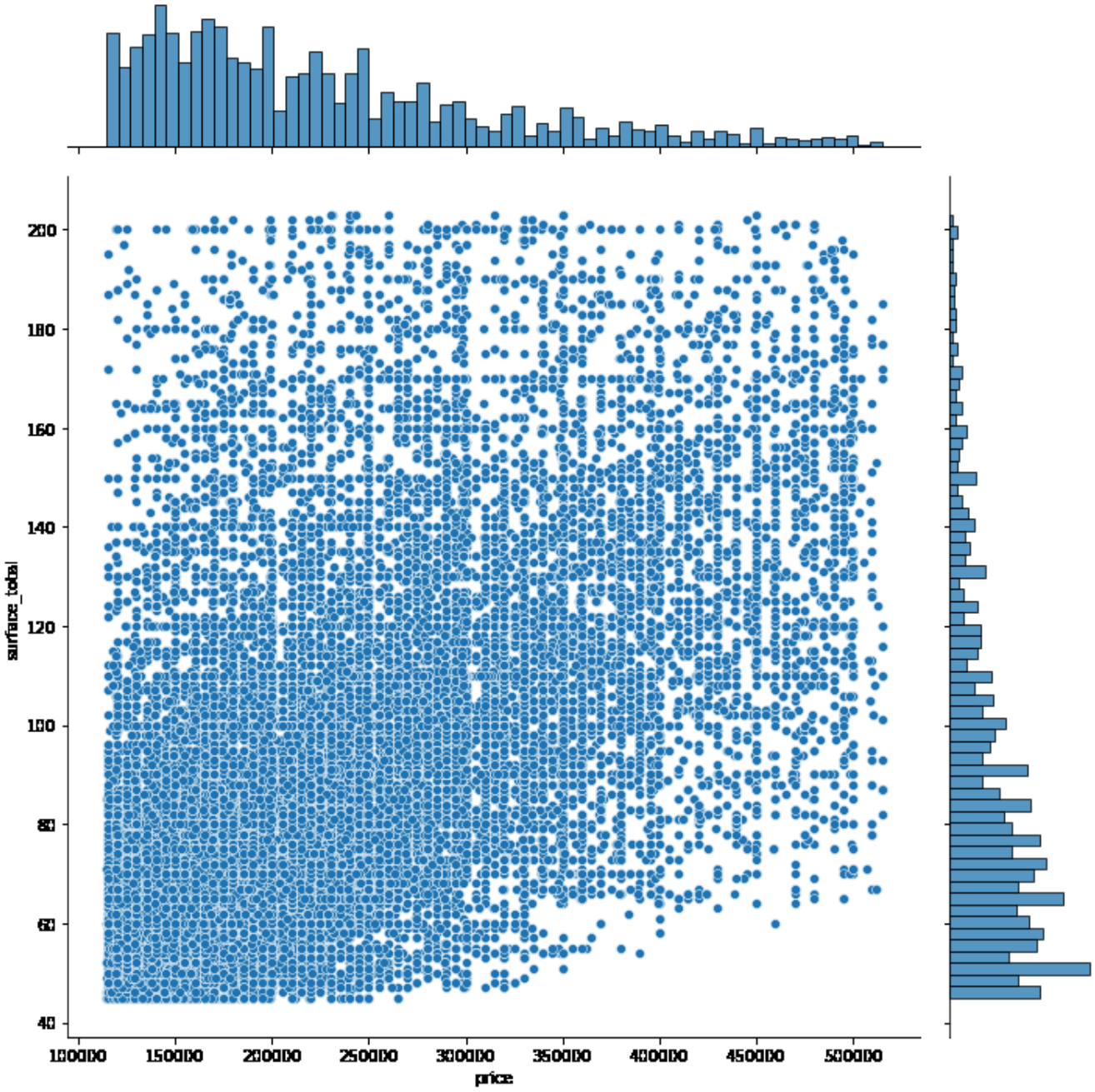
sns.jointplot(y="surface_total", x="price",data=new_properati_Capital1_All_sin_bedrooms_clean,height=10)
```

```
<seaborn.axisgrid.JointGrid at 0x18237aeb6d0>
```




surface_covered
surface_total
40
60
80
100
120
140
160
180
200


surface_covered
160
140
120
100
80
60
40
100000
150000
200000
250000
300000
350000
400000
450000
500000
price



-Voy a reescalar los valores numericos por Z-Core. Los voy a fit y transform con la Clase de scikit-learn llamada StandarScaler. Reescalo los que provienen de los atributos numericos, surface_covered, surface_total, price, rooms, bedrooms y bathrooms. Tambien podria usar la clase Robust_Scaler pero es para problemas de potencias o distribuciones muy asimetricas. Igual voy a hacer una prueba para ver cual se comporta mejor.

```
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import RobustScaler

scaler_Standard=StandardScaler()
#scaler_Robust=RobustScaler()

scaler_Standard.fit(new_properati_Capital1_All_sin_bedrooms_clean[["surface_total","surface_covered",
"bathrooms","price","rooms","bedrooms"]])
new_properati_Capital1_All_sin_bedrooms_clean_Scalet=
scaler_Standard.transform(new_properati_Capital1_All_sin_bedrooms_clean[["surface_total","surface_covered"
, "bathrooms",

"price","rooms","bedrooms"]])

df=pd.DataFrame(new_properati_Capital1_All_sin_bedrooms_clean)
```

```
display(df.shape)
display(df.head(3))

df2=pd.DataFrame(new_properati_Capital1_All_sin_bedrooms_clean_Scalet)
display(df2.shape)
df2.head(3)
```

```
(50112, 8)
```

	l3	rooms	bedrooms	bathrooms	surface_total	surface_covered	price	property_type
0	San Cristobal	7.0	7.0	2.0	140.0	140.0	153000.0	Departamento
1	Boedo	2.0	1.0	2.0	70.0	58.0	159000.0	PH
2	Palermo	2.0	1.0	1.0	45.0	45.0	125000.0	PH

```
(50112, 6)
```

	0	1	2	3	4	5
0	1.572325	2.266138	0.723356	-0.809003	4.252372	5.906773
1	-0.495685	-0.625787	0.723356	-0.741033	-1.209918	-1.300293
2	-1.234259	-1.084263	-0.769286	-1.126197	-1.209918	-1.300293

```
new_properati_Capital2_All_sin_bedrooms_clean_Scalet=pd.DataFrame(new_properati_Capital1_All_sin_bedrooms_
clean_Scalet)
new_properati_Capital2_All_sin_bedrooms_clean_Scalet=pd.DataFrame(new_properati_Capital1_All_sin_bedrooms_
clean_Scalet,index=new_properati_Capital1_All_sin_bedrooms_clean[["surface_total","surface_covered",

"bathrooms","price","rooms","bedrooms"]].index,
               columns=new_properati_Capital1_All_sin_bedrooms_clean[["surface_total","surface_covered",

"bathrooms","price","rooms","bedrooms"]].columns)
```

- Al df new_properati_Capital2_All_sin_bedrooms_clean_Scalet lo creo desde el df new_properati_Capital1_All_sin_bedrooms_clean_Scalet asignandoselo con los valores de este ultimo ya reescalado por el metodo

```
#new_properati_Capital2_All_sin_bedrooms_clean_Scalet.to_csv('new_properati_Capital2_All_sin_bedrooms_clea
n_normalizado.csv')
```

```
new_properati_Capital2_All_sin_bedrooms_clean_Scalet.head().round(5)
```

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms
0	1.57232	2.26614	0.72336	-0.80900	4.25237	5.90677
1	-0.49568	-0.62579	0.72336	-0.74103	-1.20992	-1.30029
2	-1.23426	-1.08426	-0.76929	-1.12620	-1.20992	-1.30029
3	-0.05254	-0.90793	-0.76929	0.79962	-1.20992	-1.30029
5	-0.90929	-0.69632	-0.76929	-0.84299	-1.20992	-1.30029

```
display(new_properati_Capital1_All_sin_bedrooms_clean.describe().round(5))
new_properati_Capital2_All_sin_bedrooms_clean_Scalet.describe().round(5)
```

	rooms	bedrooms	bathrooms	surface_total	surface_covered	price
count	50112.00000	50112.00000	50112.00000	50112.00000	50112.00000	50112.00000
mean	3.10752	2.08252	1.51539	86.77842	75.74407	224413.98114
std	0.91538	0.83252	0.66996	33.84932	28.35510	88274.94718
min	1.00000	0.00000	1.00000	45.00000	35.00000	114500.00000
25%	2.00000	1.00000	1.00000	60.00000	54.00000	155000.00000
50%	3.00000	2.00000	1.00000	78.00000	69.00000	200000.00000
75%	4.00000	3.00000	2.00000	104.00000	90.00000	270000.00000
max	7.00000	12.00000	4.00000	203.00000	173.00000	515000.00000

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms
count	50112.00000	50112.00000	50112.00000	50112.00000	50112.00000	50112.00000
mean	0.00000	-0.00000	-0.00000	-0.00000	0.00000	-0.00000
std	1.00001	1.00001	1.00001	1.00001	1.00001	1.00001
min	-1.23426	-1.43694	-0.76929	-1.24514	-2.30238	-2.50147
25%	-0.79111	-0.76686	-0.76929	-0.78635	-1.20992	-1.30029
50%	-0.25934	-0.23785	-0.76929	-0.27657	-0.11746	-0.09912
75%	0.50878	0.50277	0.72336	0.51641	0.97500	1.10206
max	3.43353	3.42996	3.70864	3.29186	4.25237	11.91266

- En todos los atributos sus medias y std se vieron mejoradas acercandose a 0 y 1 respectivamente.

- Vuelvo a graficar luego del reescalado.

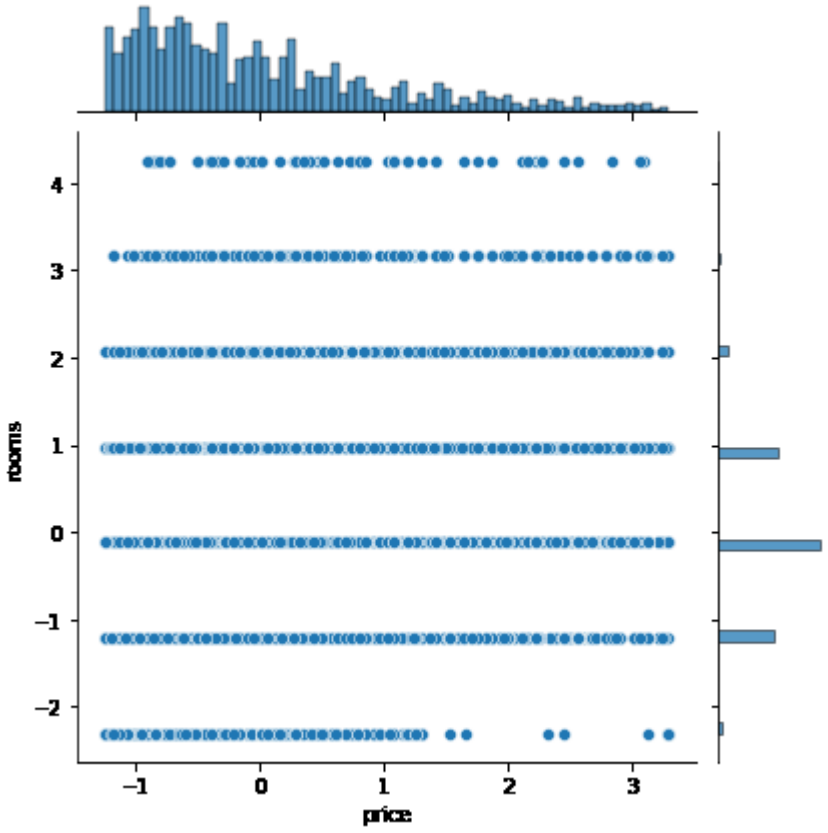
```
sns.jointplot(y="rooms", x="price",data=new_properati_Capital2_All_sin_bedrooms_clean_Scalet)

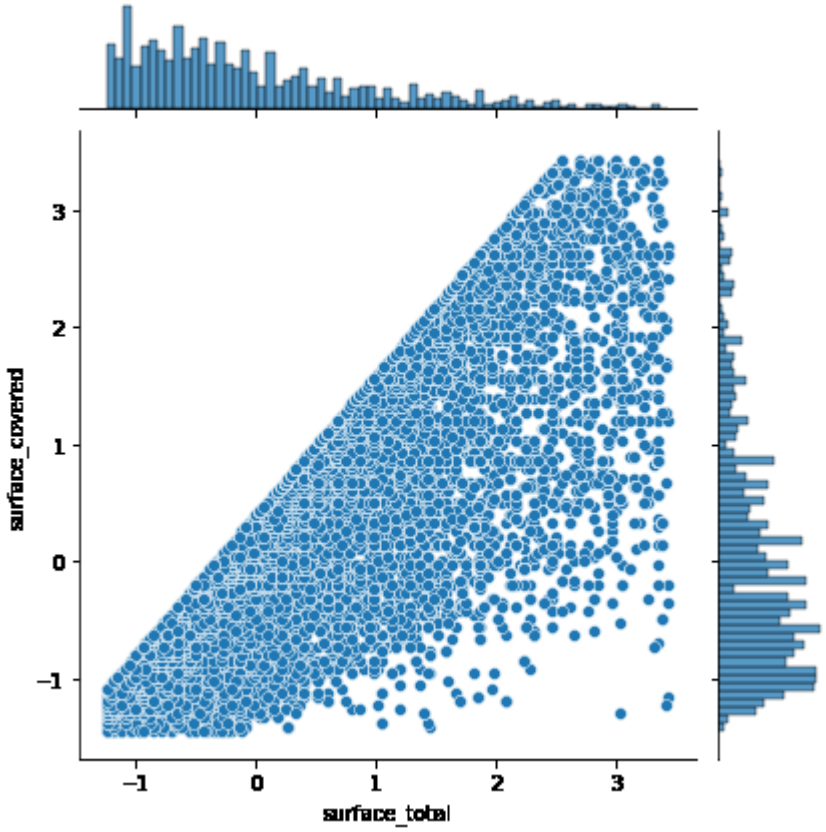
sns.jointplot(y="surface_covered",
x="surface_total",data=new_properati_Capital2_All_sin_bedrooms_clean_Scalet)

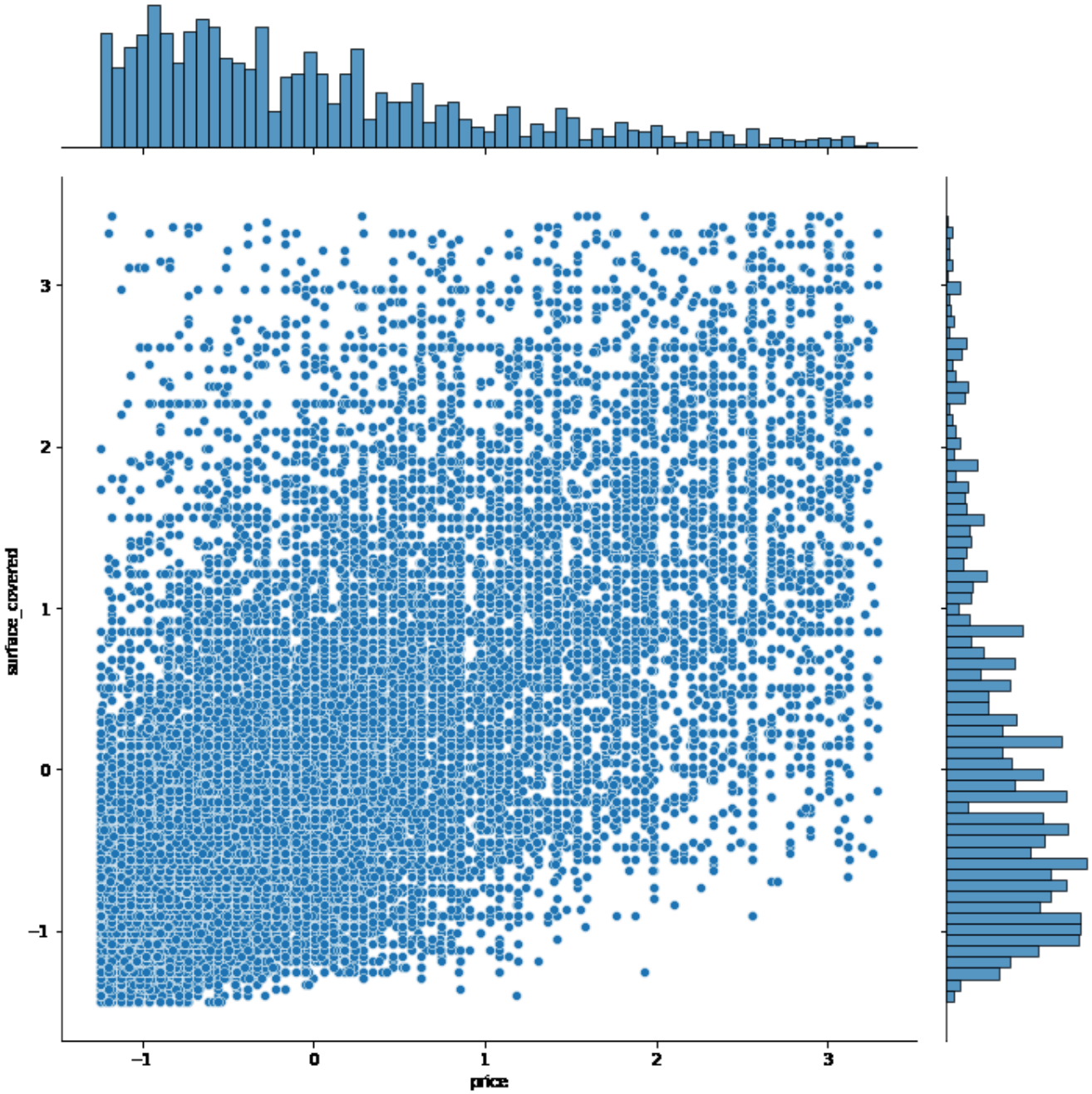
sns.jointplot(y="surface_covered",
x="price",data=new_properati_Capital2_All_sin_bedrooms_clean_Scalet,height=10)

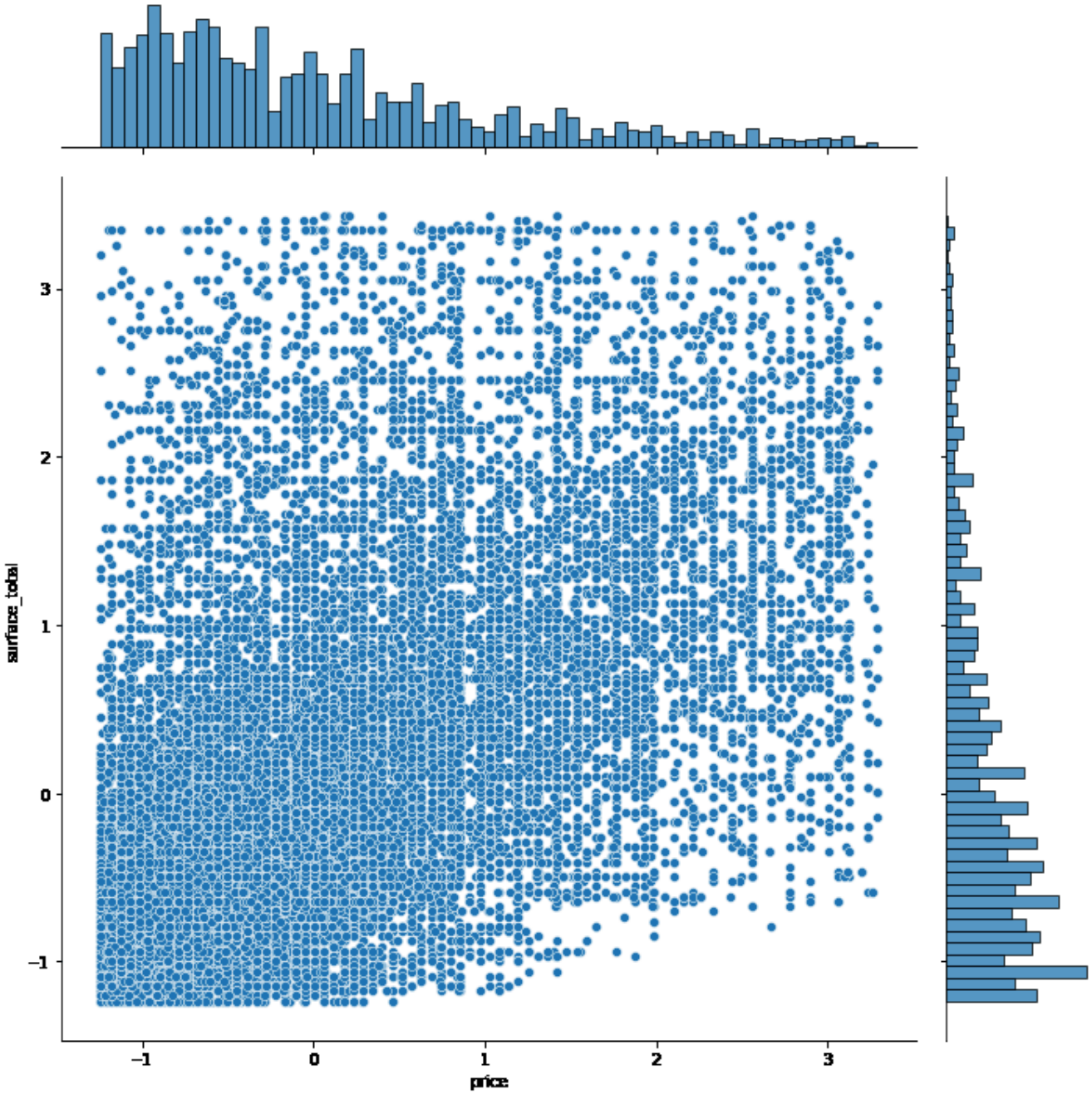
sns.jointplot(y="surface_total",
x="price",data=new_properati_Capital2_All_sin_bedrooms_clean_Scalet,height=10)
```

```
<seaborn.axisgrid.JointGrid at 0x182581ef6d0>
```




surface_covered
surface_total
3
2
1
0
−1
−1
0
1
2
3


surface_covered
price
3
2
1
0
−1
−1
0
1
2
3



- No han habido cambios significativos. Es probable que quizas no hubiera sido necesario reescalar estos atributos numericos. Igualmente los voy a utilizar porque usar datos reescalados es siempre una mejora y en el peor de los casos no tendria ningun efecto negativo.

2.5 · Encoding

- Voy a hacer un OneHotEncoder para la variable categorica property_type. Posee 3 unique_value que son "Casa", "Departamento" y "PH".

```
from sklearn.preprocessing import OneHotEncoder

onehot_encoder = OneHotEncoder(dtype=np.int, sparse=True)

transformed =
onehot_encoder.fit_transform(new_properati_Capital1_All_sin_bedrooms_clean[["property_type"]]).toarray()
print(transformed)
print(onehot_encoder.categories_)
```

```
nominales = pd.DataFrame(transformed, columns=["casa","departamento","PH"])

display(nominales)
```

```
[[0 1 0]
 [0 0 1]
 [0 0 1]
 ...
 [0 0 1]
 [0 0 1]
 [0 1 0]]
[array(['Casa', 'Departamento', 'PH'], dtype=object)]
```

	casa	departamento	PH
0	0	1	0
1	0	0	1
2	0	0	1
3	0	0	1
4	0	0	1
...	...	...	...
50107	0	1	0
50108	0	0	1
50109	0	0	1
50110	0	0	1
50111	0	1	0

50112 rows × 3 columns

```
dummies = pd.get_dummies(new_properati_Capital1_All_sin_bedrooms_clean[["property_type"]])
nominales2 = pd.concat([new_properati_Capital2_All_sin_bedrooms_clean_Scalet, dummies], axis=1)
nominales2
```

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms	property_type_Casa	property_type_Departamento	property_type_PH
0	1.572325	2.266138	0.723356	-0.809003	4.252372	5.906773	0	1	0
1	-0.495685	-0.625787	0.723356	-0.741033	-1.209918	-1.300293	0	0	1
2	-1.234259	-1.084263	-0.769286	-1.126197	-1.209918	-1.300293	0	0	1
3	-0.052540	-0.907926	-0.769286	0.799624	-1.209918	-1.300293	0	0	1
5	-0.909286	-0.696322	-0.769286	-0.842988	-1.209918	-1.300293	0	0	1
...	...	...	...	...	...	...	...	...	...
146548	-0.141169	0.220630	0.723356	2.838727	-0.117460	-0.099115	0	1	0
146549	-0.082083	0.291165	-0.769286	-0.525794	-0.117460	-0.099115	0	0	1
146550	0.804207	0.679106	0.723356	0.833609	2.067456	-0.099115	0	0	1
146555	1.690497	2.054534	0.723356	2.895369	-0.117460	-0.099115	0	0	1
146557	1.720040	2.442475	0.723356	2.215668	-0.117460	-0.099115	0	1	0

50112 rows × 9 columns

```
data_premiun= pd.DataFrame(nominales2)
display(data_premiun.head())
display(data_premiun.shape)
```

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms	property_type_Casa	property_type_Departamento	property_type_PH
0	1.572325	2.266138	0.723356	-0.809003	4.252372	5.906773	0	1	0
1	-0.495685	-0.625787	0.723356	-0.741033	-1.209918	-1.300293	0	0	1
2	-1.234259	-1.084263	-0.769286	-1.126197	-1.209918	-1.300293	0	0	1
3	-0.052540	-0.907926	-0.769286	0.799624	-1.209918	-1.300293	0	0	1
5	-0.909286	-0.696322	-0.769286	-0.842988	-1.209918	-1.300293	0	0	1

```
(50112, 9)
```

```
display(data_premiun.describe().round(3))
data_premiun.corr().round(3)
```

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms	property_type_Casa	property_type_Departamento	property_type_PH
count	50112.000	50112.000	50112.000	50112.000	50112.000	50112.000	50112.000	50112.000	50112.000
mean	0.000	-0.000	-0.000	-0.000	0.000	-0.000	0.014	0.854	0.132
std	1.000	1.000	1.000	1.000	1.000	1.000	0.115	0.353	0.338
min	-1.234	-1.437	-0.769	-1.245	-2.302	-2.501	0.000	0.000	0.000
25%	-0.791	-0.767	-0.769	-0.786	-1.210	-1.300	0.000	1.000	0.000
50%	-0.259	-0.238	-0.769	-0.277	-0.117	-0.099	0.000	1.000	0.000
75%	0.509	0.503	0.723	0.516	0.975	1.102	0.000	1.000	0.000
max	3.434	3.430	3.709	3.292	4.252	11.913	1.000	1.000	1.000

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms	property_type_Casa	property_type_Departamento	property_type_PH
surface_total	1.000	0.886	0.518	0.621	0.666	0.645	0.213	-0.305	0.245
surface_covered	0.886	1.000	0.571	0.661	0.740	0.726	0.169	-0.225	0.177
bathrooms	0.518	0.571	1.000	0.552	0.452	0.452	0.036	-0.005	-0.007
price	0.621	0.661	0.552	1.000	0.429	0.411	0.045	0.036	-0.053
rooms	0.666	0.740	0.452	0.429	1.000	0.941	0.117	-0.215	0.184
bedrooms	0.645	0.726	0.452	0.411	0.941	1.000	0.102	-0.204	0.178
property_type_Casa	0.213	0.169	0.036	0.045	0.117	0.102	1.000	-0.284	-0.046
property_type_Departamento	-0.305	-0.225	-0.005	0.036	-0.215	-0.204	-0.284	1.000	-0.945
property_type_PH	0.245	0.177	-0.007	-0.053	0.184	0.178	-0.046	-0.945	1.000

Importante: A partir de aca haremos el analisis y comparacion de modelos con el df data_premiun. Es el df que ya posee todo el trabajo de limpieza, reescalado de variable numericas y codificado de variable categorica.

2.7 · Comparacion con resultados de modelos del proyecto 1

- Genero el df (df_proyecto1_clean), igual al del proyecto 1.

```
properati.shape
```

```
(146660, 19)
```

```
df_proyecto1 = properati.query("l2 == 'Capital Federal' and property_type == ['Departamento', 'PH',
'Casa'] and 15 <= surface_total <= 1000 and price <= 4000000")
display(df_proyecto1.shape)
df_proyecto1.head(3)
```

```
(82373, 19)
```

	start_date	end_date	created_on	lat	lon	l1	l2	l3	rooms	bedrooms	...	description	property_type	operation_type
0	2019-10-17	2019-12-23	2019-10-17	-34.605880	-58.384949	Argentina	Capital Federal	San Cristobal	7.0	7.0	...	DESCRIPCION DE LA PROPIEDAD: Departamento de 1...	Departamento	Venta
1	2019-10-17	2019-11-21	2019-10-17	-34.624056	-58.412110	Argentina	Capital Federal	Boedo	2.0	1.0	...	PH reciclado en Boedo a una cuadra de la plaz...	PH	Venta
2	2019-10-17	2019-11-01	2019-10-17	-34.593569	-58.427474	Argentina	Capital Federal	Palermo	2.0	1.0	...	2 ambienets amplio , excelente estado , patio ...	PH	Venta

```
df_proyecto1_clean=pd.DataFrame(df_proyecto1)
df_proyecto1_clean= df_proyecto1[['rooms', 'bedrooms', 'bathrooms', 'surface_total', 'surface_covered',
'price']].dropna()

display(df_proyecto1_clean.shape)
df_proyecto1_clean.head(3)
```

```
(81019, 6)
```

	rooms	bedrooms	bathrooms	surface_total	surface_covered	price
0	7.0	7.0	2.0	140.0	140.0	153000.0
1	2.0	1.0	2.0	70.0	58.0	159000.0
2	2.0	1.0	1.0	45.0	45.0	125000.0

```
display(df_proyecto1_clean.describe().round())
df_proyecto1_clean.corr().round(2)
```

	rooms	bedrooms	bathrooms	surface_total	surface_covered	price
count	81019.0	81019.0	81019.0	81019.0	81019.0	81019.0
mean	3.0	2.0	2.0	95.0	85.0	263773.0
std	1.0	1.0	1.0	77.0	156.0	298950.0
min	1.0	0.0	1.0	15.0	1.0	6000.0
25%	2.0	1.0	1.0	49.0	44.0	125000.0
50%	3.0	2.0	1.0	71.0	63.0	180000.0
75%	4.0	3.0	2.0	111.0	96.0	280000.0
max	26.0	15.0	14.0	1000.0	17220.0	4000000.0

	rooms	bedrooms	bathrooms	surface_total	surface_covered	price
rooms	1.00	0.93	0.60	0.73	0.30	0.48
bedrooms	0.93	1.00	0.61	0.70	0.29	0.44
bathrooms	0.60	0.61	1.00	0.65	0.28	0.64
surface_total	0.73	0.70	0.65	1.00	0.38	0.69
surface_covered	0.30	0.29	0.28	0.38	1.00	0.30
price	0.48	0.44	0.64	0.69	0.30	1.00

- Modelo de Benchmark /Regresion lineal

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error
```

```
X = data_premiun[['bathrooms', 'surface_total', 'surface_covered']]
#X = new_properati_Capital1_All_sin_bedrooms_clean[['bathrooms', 'surface_total', 'surface_covered']]
#X = df_proyecto1_clean[['bathrooms', 'surface_total', 'surface_covered']]

y =data_premiun['price']
#y =new_properati_Capital1_All_sin_bedrooms_clean['price']
#y = df_proyecto1_clean['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

```
linear_model = LinearRegression()

display(linear_model.fit(X_train, y_train))
```

```
LinearRegression()
```

```
# Evaluación modelo benchmark

y_train_pred = linear_model.predict(X_train)
y_test_pred = linear_model.predict(X_test)

print('Modelo referencia: Regresión lineal')

rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
print(f'Raíz del error cuadrático medio en Test: {rmse_test}')

plt.figure(figsize = (18,8))

plt.subplot(1,2,1)
sns.distplot(y_train - y_train_pred, bins = 20, label = 'train')
sns.distplot(y_test - y_test_pred, bins = 20, label = 'test')
plt.xlabel('errores')
plt.legend()

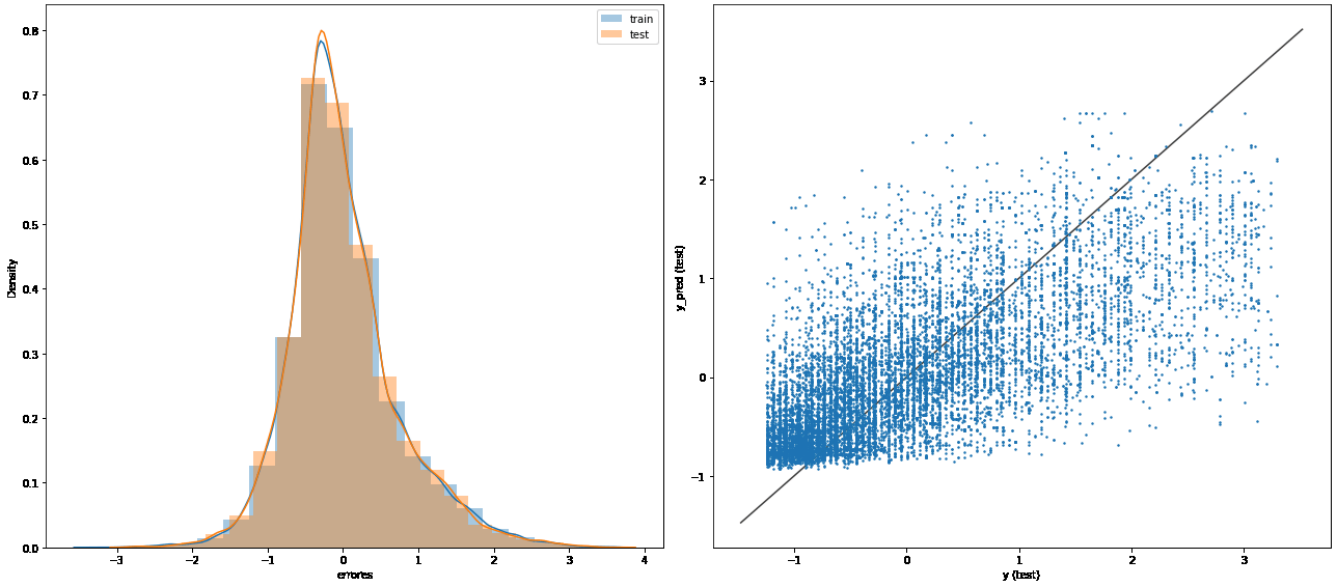
ax = plt.subplot(1,2,2)
ax.scatter(y_test,y_test_pred, s =2)

lims = [
np.min([ax.get_xlim(), ax.get_ylim()]),  # min of both axes
np.max([ax.get_xlim(), ax.get_ylim()]),  # max of both axes]
]
```

```
ax.plot(lims, lims, 'k-', alpha=0.75, zorder=0)
plt.xlabel('y (test)')
plt.ylabel('y_pred (test)')

plt.tight_layout()
plt.show()
```

```
Modelo referencia: Regresión lineal
Raíz del error cuadrático medio en Train: 0.7175818120067323
Raíz del error cuadrático medio en Test: 0.7126921178424601
```



2.7.1 · Proyecto 1 y 2 / Arbol de Regresion y K Vecinos

> - **df_proyecto1_clean** (el original del proyecto1).
> - **new_properati_Capital1_All_sin_bedrooms_clean** (es el df al que llegue Yo en este proyecto 2, luego de hacer todo el preprocesamiento de datos, **menos el reescalado**).

```
X = new_properati_Capital1_All_sin_bedrooms_clean[['bathrooms', 'surface_total', 'surface_covered']]

#X = df_proyecto1_clean[['bathrooms', 'surface_total', 'surface_covered']]


y =new_properati_Capital1_All_sin_bedrooms_clean['price']
#y = df_proyecto1_clean['price']

print(X.shape)
print(y.shape)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

X_train.head()
```

```
(50112, 3)
(50112,)
(35078, 3) (15034, 3) (35078,) (15034,)
```

	bathrooms	surface_total	surface_covered
36144	1.0	53.0	53.0
85042	2.0	160.0	155.0
10907	4.0	130.0	125.0
138198	1.0	63.0	58.0
47037	1.0	72.0	64.0

```
tree_regressor = DecisionTreeRegressor(max_depth=20, random_state=42)

knn_regressor = KNeighborsRegressor(n_neighbors=4)

display(tree_regressor.fit(X_train, y_train))
knn_regressor.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=20, random_state=42)
```

```
KNeighborsRegressor(n_neighbors=4)
```

```
modelos = ['Árbol de Decisión', 'Vecinos más cercanos']

for i, model in enumerate([tree_regressor, knn_regressor]):
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    print(f'Modelo: {modelos[i]}')

    rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
    rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
    print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
    print(f'Raíz del error cuadrático medio en Test: {rmse_test}')

    plt.figure(figsize = (18,8))

    plt.subplot(1,2,1)
    sns.distplot(y_train - y_train_pred, bins = 20, label = 'train')
    sns.distplot(y_test - y_test_pred, bins = 20, label = 'test')
    plt.xlabel('errores')
    plt.legend()

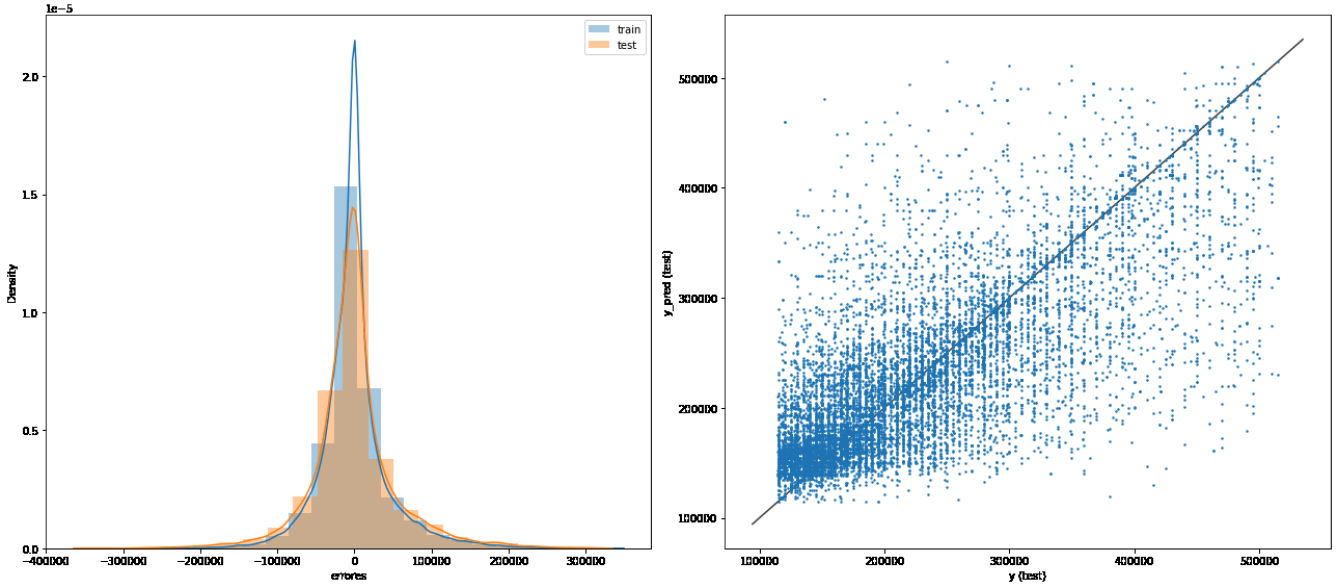

    ax = plt.subplot(1,2,2)
    ax.scatter(y_test,y_test_pred, s =2)

    lims = [
    np.min([ax.get_xlim(), ax.get_ylim()]),  # min of both axes
    np.max([ax.get_xlim(), ax.get_ylim()]),  # max of both axes]
    ]

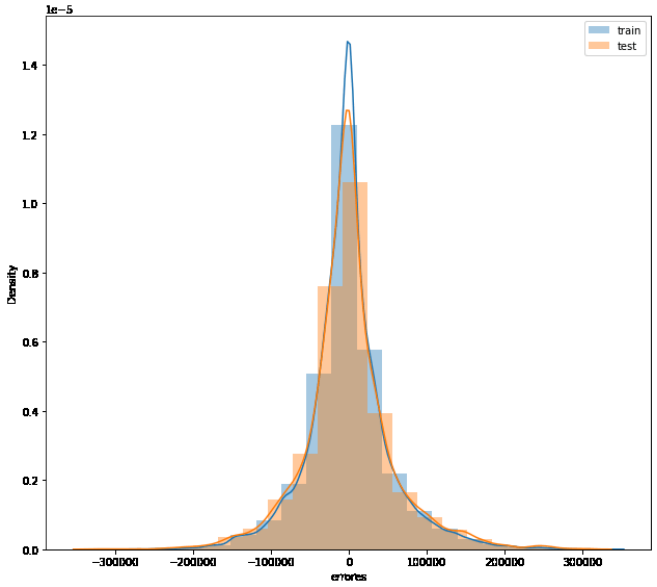
    ax.plot(lims, lims, 'k-', alpha=0.75, zorder=0)
    plt.xlabel('y (test)')
    plt.ylabel('y_pred (test)')

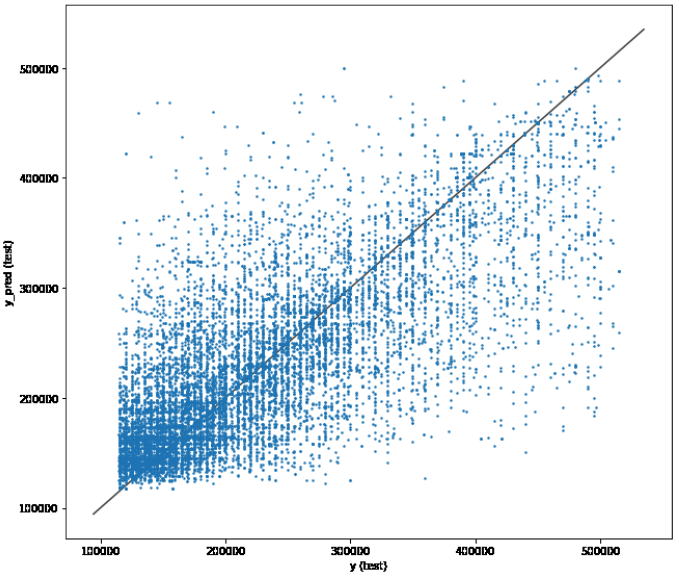
    plt.tight_layout()
    plt.show()
```

```
Modelo: Árbol de Decisión
Raíz del error cuadrático medio en Train: 44670.275581253154
Raíz del error cuadrático medio en Test: 56243.826394677686
```



```
Modelo: Vecinos más cercanos
Raíz del error cuadrático medio en Train: 52878.62782580318
Raíz del error cuadrático medio en Test: 58560.34405764339
```





- Curvas de Validacion para KNN ####

```
# Definimos las listas vacias para los valores de accuracy deseados
lista_rmse_train_knn = []
lista_rmse_test_knn = []

# Definimos la lista de valores de k que vamos a explorar
k_vecinos = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,20,25,30,40,50,80]

# Generamos un loop sobre los distintos valores de k
for k in k_vecinos:

    # Vamos a repetir el siguiente bloque de código

    # Definir el modelo con el valor de vecinos deseado
    knn_regressor = KNeighborsRegressor(n_neighbors= k)
    #reg_1 = LinearRegression()

    # Entrenar el modelo
    knn_regressor.fit(X_train, y_train)

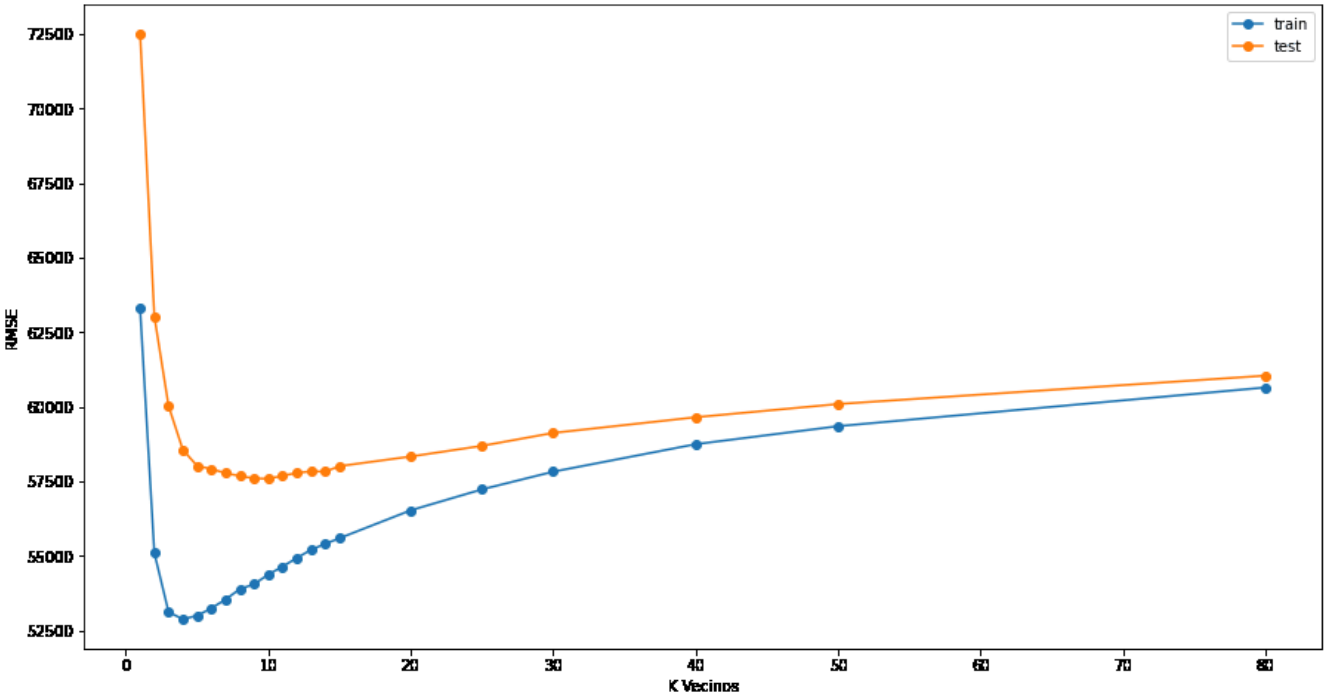
    # Predecir y evaluar sobre el set de entrenamiento
    y_train_pred = knn_regressor.predict(X_train)
    rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))

    # Predecir y evaluar sobre el set de evaluación
    y_test_pred = knn_regressor.predict(X_test)
    rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))

    # Agregar la información a las listas
    lista_rmse_train_knn.append(rmse_train)
    lista_rmse_test_knn.append(rmse_test)
```

```
plt.figure(figsize = (15,8))
plt.plot(k_vecinos, lista_rmse_train_knn,'o-',label='train' )
plt.plot(k_vecinos, lista_rmse_test_knn,'o-',label='test')
plt.legend()
plt.xlabel("K Vecinos")
plt.ylabel("RMSE")
```

```
Text(0, 0.5, 'RMSE')
```




```
print("RMSE de Train", lista_rmse_train_knn)
print()
print("RMSE de Test",lista_rmse_test_knn)
```

```
RMSE de Train [63300.26127020773, 55094.956722818606, 53114.69731553755, 52878.62782580318, 52993.855066560274,
53244.700492121774, 53539.76312941841, 53887.10535013954, 54051.25222371899, 54374.46123769596, 54642.9781324403
8, 54928.509321306316, 55206.00415789189, 55404.4689962666, 55593.539561018035, 56528.7293631606, 57230.14033969
7966, 57823.32411813781, 58745.79097283575, 59349.45822896145, 60647.839719017284]

RMSE de Test [72517.79570831775, 63031.85787780417, 60021.22189851369, 58560.34405764339, 58019.03635108905, 579
13.02762901825, 57774.59662379946, 57679.70609436522, 57599.78614237622, 57583.97228651106, 57687.5921538004, 57
780.87474857898, 57834.8203850392, 57825.0175134305, 58010.252843708346, 58333.34658813388, 58689.947351420225,
59123.7058774747, 59649.242922986676, 60092.55957026944, 61043.85644554942]
```

```
print("K vecinos con mínimo RMSE para
train:",k_vecinos[lista_rmse_train_knn.index(min(lista_rmse_train_knn))],"con un valor
de",lista_rmse_train_knn[lista_rmse_train_knn.index(min(lista_rmse_train_knn))])

print("K vecinos con mínimo RMSE para
test:",k_vecinos[lista_rmse_test_knn.index(min(lista_rmse_test_knn))],"con un valor
de",lista_rmse_test_knn[lista_rmse_test_knn.index(min(lista_rmse_test_knn))])
```

```
K vecinos con mínimo RMSE para train: 4 con un valor de 52878.62782580318
K vecinos con mínimo RMSE para test: 10 con un valor de 57583.97228651106
```

> En la curva de validacion de **test**, el RMSE mas bajo esta en 12 K vecinos; de 1 a 5 baja el RMSE, y de ahi en adelante se mantiene muy parejo siendo 12 el punto minimo. En la curva de validacion de **train**, el RMSE mas bajo esta en 5 K vecino, y de ahi en adelante no para de crecer hasta el 80.
>
> **Por lo tanto el K vecino que me parece mas adecuada para train es de 5.** #####

- Curvas de Validacion para Arbol de Decision ####

```
# Definimos las listas vacias para los valores de accuracy deseados
lista_rmse_train_dt = []
lista_rmse_test_dt = []

# Definimos la lista de valores de max_depth que vamos a explorar
max_depths = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,20,25,30,40,50,80]

# Generamos un loop sobre los distintos valores de k
for max_depth in max_depths:

    # Vamos a repetir el siguiente bloque de código
```

```
    # Definir el modelo con la profundidad deseada
    tree_regressor = DecisionTreeRegressor(max_depth = max_depth, random_state=42)

    # Entrenar el modelo
    tree_regressor.fit(X_train, y_train)

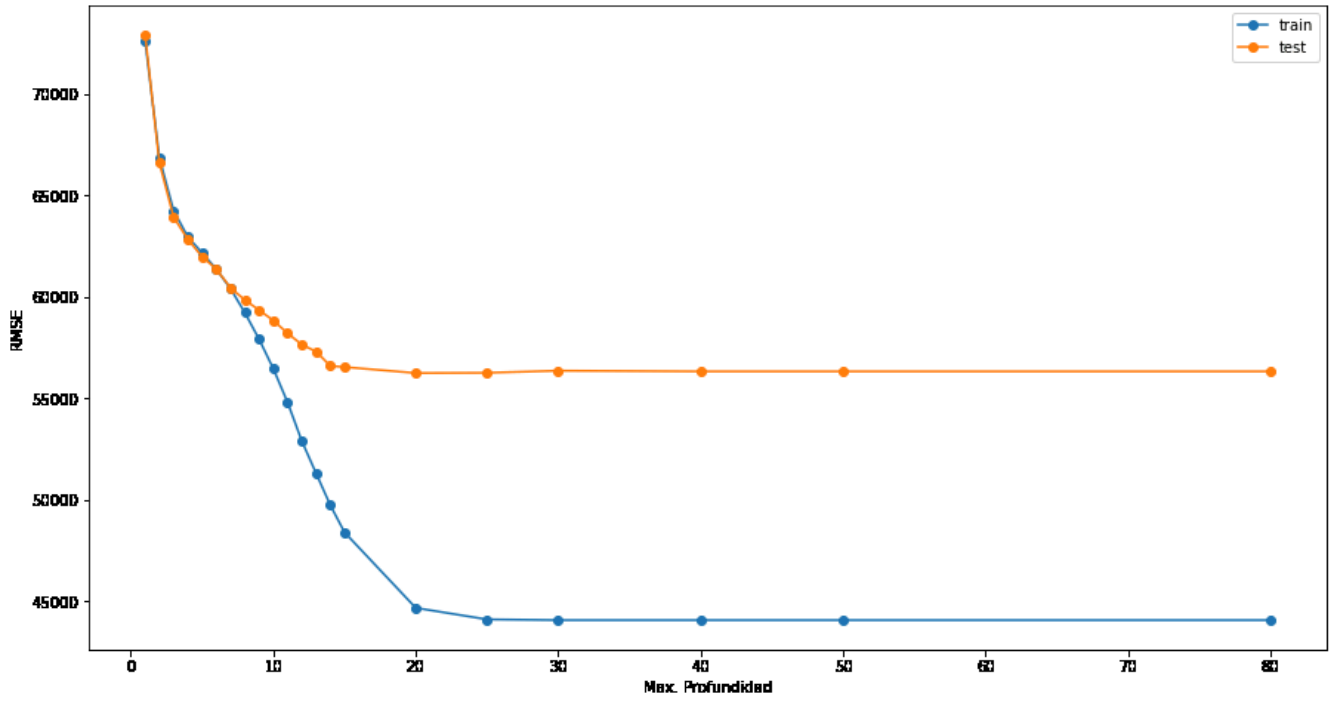
    # Predecir y evaluar sobre el set de entrenamiento
    y_train_pred = tree_regressor.predict(X_train)
    rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))

    # Predecir y evaluar sobre el set de evaluación
    y_test_pred = tree_regressor.predict(X_test)
    rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))

    # Agregar la información a las listas
    lista_rmse_train_dt.append(rmse_train)
    lista_rmse_test_dt.append(rmse_test)
```

```
plt.figure(figsize = (15,8))
plt.plot(max_depths, lista_rmse_train_dt,'o-',label='train' )
plt.plot(max_depths, lista_rmse_test_dt,'o-',label='test')
plt.legend()
plt.xlabel("Max. Profundidad")
plt.ylabel("RMSE")
```

```
Text(0, 0.5, 'RMSE')
```




```
print("RMSE de Train",lista_rmse_train_dt)
print()
print("RMSE de Test",lista_rmse_test_dt)
```

```
RMSE de Train [72630.35462837177, 66858.27962222022, 64247.91587571058, 62948.594066658574, 62168.15435799987, 6
1358.446854930255, 60399.35543006139, 59217.795988239945, 57911.12454247018, 56457.6812999613, 54800.0788317621
1, 52894.66174066441, 51306.19790989622, 49732.79197963539, 48393.13196182335, 44670.275581253154, 44104.7036044
0875, 44067.2617764599, 44067.24751731076, 44067.24751731076, 44067.24751731076]

RMSE de Test [72924.59643879658, 66612.74702688809, 63923.5201441798, 62830.19674869889, 61976.74887338525, 6136
3.061536726505, 60441.790690384216, 59842.46498676892, 59345.98519585529, 58833.419648461226, 58206.22949287053,
57642.91751036346, 57305.75466427445, 56595.65824449393, 56538.3980815748, 56243.826394677686, 56256.14742703830
4, 56358.1616974049, 56323.789798095764, 56323.789798095764, 56323.789798095764]
```

```
print("Profundidad con mínimo RMSE para
train:",max_depths[lista_rmse_train_dt.index(min(lista_rmse_train_dt))],"con un valor
de",lista_rmse_train_dt[lista_rmse_train_dt.index(min(lista_rmse_train_dt))])

print("Profundidad con mínimo RMSE para
```

```
test:",max_depths[lista_rmse_test_dt.index(min(lista_rmse_test_dt))],"con un valor
de",lista_rmse_test_dt[lista_rmse_test_dt.index(min(lista_rmse_test_dt))])
```

```
Profundidad con mínimo RMSE para train: 40 con un valor de 44067.24751731076
Profundidad con mínimo RMSE para test: 20 con un valor de 56243.826394677686
```

> En la curva de validacion de test, el RMSE mas bajo esta en 15; practicamente de ahi en adelante hasta 80 no tiene modificaciones significativas. En la curva de validacion de train, el RMSE mas bajo esta en 30 pero desde el 20 practicamente no se reduce significativamente hasta la prueba con 80.
>
> **Por lo tanto la profundidad que me parece mas adecuada para train es de 20.** #####

2.7.2 · DF Proyecto 2 Reescalado/ Arbol de Regresion y K Vecinos

- Ahora con el df del proyecto 2 cuyos datos han sido escalados (**data_premiun**).

```
X = data_premiun[['bathrooms', 'surface_total', 'surface_covered']]

y = data_premiun['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

```
tree_regressor = DecisionTreeRegressor(max_depth=20, random_state=42)

knn_regressor = KNeighborsRegressor(n_neighbors=2)

tree_regressor.fit(X_train, y_train)
knn_regressor.fit(X_train, y_train)
```

```
KNeighborsRegressor(n_neighbors=2)
```

```
y_train_pred_tree =tree_regressor.predict(X_train)
y_test_pred_tree = tree_regressor.predict(X_test)

y_train_pred_knn = knn_regressor.predict(X_train)
y_test_pred_knn = knn_regressor.predict(X_test)
```

```
    rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_tree))
    rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_tree))
    print(f'Raíz del error cuadrático medio en Train por TreeRegressor: {rmse_train}')
    print(f'Raíz del error cuadrático medio en Test por TreRegressor: {rmse_test}')

    rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_knn))
    rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_knn))
    print(f'Raíz del error cuadrático medio en Train por Knn: {rmse_train}')
    print(f'Raíz del error cuadrático medio en Test por Knn: {rmse_test}')
```

```
Raíz del error cuadrático medio en Train por TreeRegressor: 0.5060407592237395
Raíz del error cuadrático medio en Test por TreRegressor: 0.6392006683030012
Raíz del error cuadrático medio en Train por Knn: 0.6168907632602438
Raíz del error cuadrático medio en Test por Knn: 0.7022933319151048
```

2.7.3 · Conclusiones/Resultados

> - **df_proyecto1_clean**

- Modelo de **Benchmark**/Regresion Lineal.

Raíz del error cuadrático medio en Train: **202687.150**

Raíz del error cuadrático medio en Test: **206082.712**

- Modelo: Árbol de Decisión

Raíz del error cuadrático medio en Train: **80976.866**

Raíz del error cuadrático medio en Test: **156077.545**
- Modelo: Vecinos más cercanos
Raíz del error cuadrático medio en Train: **111439.282**
Raíz del error cuadrático medio en Test: **160835.375**

> - **new_properati_Capital1_All_sin_bedrooms_clean**

- Modelo de **Benchmark**/Regresion Lineal.

Raíz del error cuadrático medio en Train: **63343.864**
Raíz del error cuadrático medio en Test: **62912.231**
- Modelo: Árbol de Decisión
Raíz del error cuadrático medio en Train: **44670.275**
Raíz del error cuadrático medio en Test: **56243.826**
- Modelo: Vecinos más cercanos
Raíz del error cuadrático medio en Train: **55094.956**
Raíz del error cuadrático medio en Test: **63031.857**

> - **data_premiun**

- Modelo de **Benchmark**/Regresion Lineal.

Raíz del error cuadrático medio en Train: **0.717**
Raíz del error cuadrático medio en Test: **0.712**
- Modelo: Árbol de Decisión
Raíz del error cuadrático medio en Train por TreeRegressor: **0.5060**
Raíz del error cuadrático medio en Test por TreRegressor: **0.6376**
- Modelo: Vecinos más cercanos
Raíz del error cuadrático medio en Train por Knn: **0.6168**
Raíz del error cuadrático medio en Test por Knn: **0.7022**

- Probe con distintas combinaciones de atributos. Hasta aca los mejores resultados fueron con la combinacion de surface_total, surface_covered y bathrooms.

- En los 3 df el mejor valor de **profundidad de arbol fue de 20** y en el kvecinos fue **n-vecinos de 5.** Los elegi con las curvas de validacion en forma manual.

- En los 3 df, en los dos modelos utilizados (K-vecinos y arboles de decision), el modelo que tiene una menor diferencia entre el resultado de train y test, es siempre el modelo de **K-vecinos**.

- Los 3 df mejoran su RMSE con Kvecino y Arbol de decision, contra el benchmark, que es un modelo de regresion lineal.

-Para (**data_premiun**) el modelo que tiene un RMSE de test un poco mas bajo, es el Arbol de Decision.

- Los dos df del proyecto 2 (**new_properati_Capital1_All_sin_bedrooms_clean**) y **(data_premiun) mejoran arriba de un 65% su RMSE** contra el df del proyecto 1 (**df_proyecto1_clean**). Tanto en Kvecinos como en Arbol de decision.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import make_pipeline
```

3 · Modelos Avanzados

3.1 · Modelo Avanzado 1 - Regresión lineal con atributos polinómicos y regularización

- Con GridSearch y CroosValidation

3.1.1 · Regresion Lineal

```
data_premiun.head()
```

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms	property_type_Casa	property_type_Departamento	property_type_PH
0	1.572325	2.266138	0.723356	-0.809003	4.252372	5.906773	0	1	0
1	-0.495685	-0.625787	0.723356	-0.741033	-1.209918	-1.300293	0	0	1
2	-1.234259	-1.084263	-0.769286	-1.126197	-1.209918	-1.300293	0	0	1
3	-0.052540	-0.907926	-0.769286	0.799624	-1.209918	-1.300293	0	0	1
5	-0.909286	-0.696322	-0.769286	-0.842988	-1.209918	-1.300293	0	0	1

```
from sklearn.model_selection import train_test_split

X = data_premiun[[ "bathrooms","surface_covered","surface_total"]]

y =data_premiun[['price']]

print(X.shape)
print(y.shape)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

display(X_train.head())
y_train.head()
```

```
(50112, 3)
(50112, 1)
(35078, 3) (15034, 3) (35078, 1) (15034, 1)
```

	bathrooms	surface_covered	surface_total
36144	-0.769286	-0.802124	-0.997915
85042	0.723356	2.795148	2.163185
10907	3.708639	1.737127	1.276895
138198	-0.769286	-0.625787	-0.702486
47037	-0.769286	-0.414183	-0.436599

	price
36144	-0.978928
85042	0.686340
10907	1.989101
138198	-0.842999
47037	0.085937

```
from sklearn.linear_model import LinearRegression

reg_1 = LinearRegression()

reg_1.get_params().keys()

param_grid_RL={"copy_X": ["True","False"],
"fit_intercept":["True","False"],
"normalize":["True","False"]}

model2 = GridSearchCV(reg_1, param_grid=param_grid_RL,cv=15)
```

```
model2.fit(X_train, y_train)
```

```
GridSearchCV(cv=15, estimator=LinearRegression(),
             param_grid={'copy_X': ['True', 'False'],
                         'fit_intercept': ['True', 'False'],
                         'normalize': ['True', 'False']})
```

```
y_train_pred_1 = model2.predict(X_train)
y_test_pred_1 = model2.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error

rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_1))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_1))
print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
print(f'Raíz del error cuadrático medio en Test: {rmse_test}')

plt.figure(figsize = (18,8))

plt.subplot(1,2,1)
sns.distplot(y_train - y_train_pred_1, bins = 20, label = 'train')
sns.distplot(y_test - y_test_pred_1, bins = 20, label = 'test')
plt.xlabel('errores')
plt.legend()

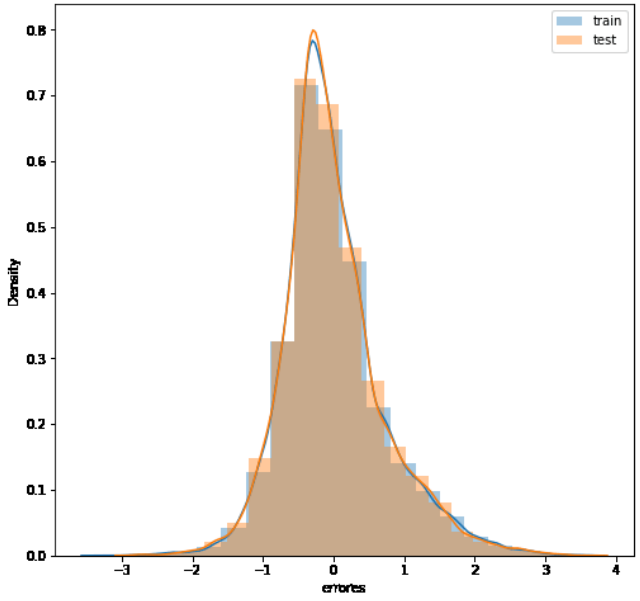

ax = plt.subplot(1,2,2)
ax.scatter(y_test,y_test_pred_1, s =2)

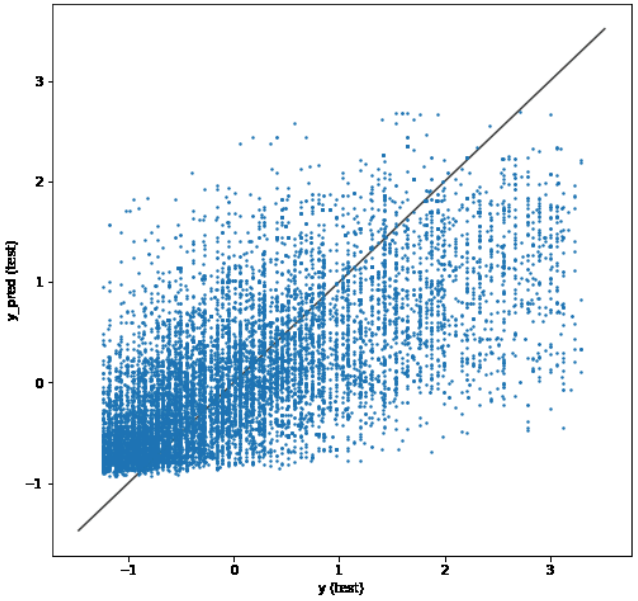
lims = [
np.min([ax.get_xlim(), ax.get_ylim()]),  # min of both axes
np.max([ax.get_xlim(), ax.get_ylim()]),  # max of both axes]
]

ax.plot(lims, lims, 'k-', alpha=0.75, zorder=0)
plt.xlabel('y (test)')
plt.ylabel('y_pred (test)')
```

```
Raíz del error cuadrático medio en Train: 0.7175818120067323
Raíz del error cuadrático medio en Test: 0.7126921178424601
```

```
Text(0, 0.5, 'y_pred (test)')
```





3.1.1.2 · Regresion lineal con atributos polinomicos

- Ahora voy a probar agregando atributos polinomicos con la utilizacion de la clases PolynomialFeatures de Scikit-Learn.

```
from sklearn.model_selection import train_test_split

X = data_premiun[[ "bathrooms","surface_covered","surface_total"]]

y =data_premiun[['price']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

```
from sklearn.preprocessing import PolynomialFeatures

poly1 = PolynomialFeatures(4,interaction_only=True)

poly1.get_params().keys()
```

```
dict_keys(['degree', 'include_bias', 'interaction_only', 'order'])
```

```
X_train_new =poly1.fit_transform(X_train)
X_test_new = poly1.fit_transform(X_test)
```

```
poly1.get_feature_names
```

```
<bound method PolynomialFeatures.get_feature_names of PolynomialFeatures(degree=4, interaction_only=True)>
```

```
from sklearn.linear_model import LinearRegression

reg_2 = LinearRegression()

param_grid_RL={"copy_X": ["True","False"],
"fit_intercept":["True","False"],
"normalize":["True","False"]}

model3 = GridSearchCV(reg_2, param_grid=param_grid_RL,cv=15)
```

```
model3.fit(X_train_new, y_train)
```

```
GridSearchCV(cv=15, estimator=LinearRegression(),
             param_grid={'copy_X': ['True', 'False'],
                         'fit_intercept': ['True', 'False'],
                         'normalize': ['True', 'False']})
```

```
y_train_pred_2 = model3.predict(X_train_new)
y_test_pred_2 = model3.predict(X_test_new)
```

```
from sklearn.metrics import mean_squared_error

rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_2))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_2))
print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
print(f'Raíz del error cuadrático medio en Test: {rmse_test}')

plt.figure(figsize = (18,8))

plt.subplot(1,2,1)
sns.distplot(y_train - y_train_pred_2, bins = 20, label = 'train')
sns.distplot(y_test - y_test_pred_2, bins = 20, label = 'test')
plt.xlabel('errores')
plt.legend()


ax = plt.subplot(1,2,2)
ax.scatter(y_test,y_test_pred_2, s =2)

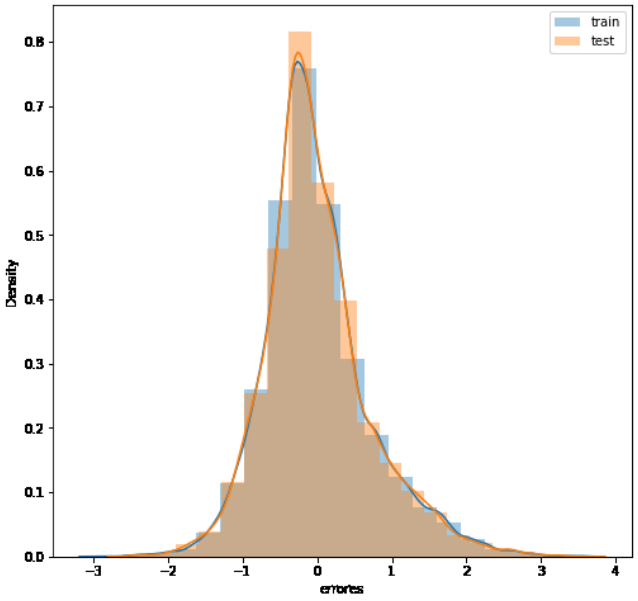
lims = [
np.min([ax.get_xlim(), ax.get_ylim()]),  # min of both axes
np.max([ax.get_xlim(), ax.get_ylim()]),  # max of both axes]
]

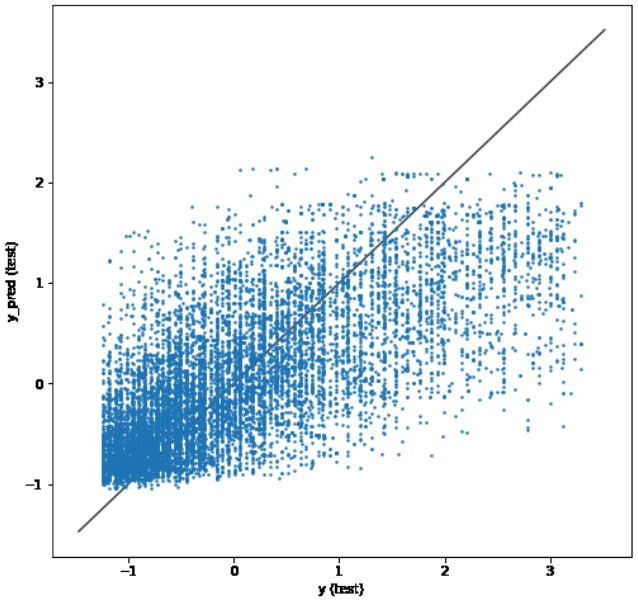
ax.plot(lims, lims, 'k-', alpha=0.75, zorder=0)
```

```
plt.xlabel('y (test)')
plt.ylabel('y_pred (test)')
```

```
Raíz del error cuadrático medio en Train: 0.7137075396545709
Raíz del error cuadrático medio en Test: 0.7087867668526242
```

```
Text(0, 0.5, 'y_pred (test)')
```





- El modelo de regresion lineal con atributos polinomicos es muy sensible al sobreajuste. En este caso el rmse dio casi igual que sin atributos polinomicos; pero igualmente voy a probar regularizar. Tenemos principalmente dos modelos; Ridge, Lasso

3.1.1.3 · Regresion lineal con Regularizacion Ridge

```
from sklearn.linear_model import Ridge, Lasso

reg_ridge = Ridge(alpha= 0.1)

reg_ridge_scores = cross_val_score(reg_ridge, X, y, cv=15)
reg_ridge.fit(X_train_new,y_train)

y_train_pred_3 = reg_ridge.predict(X_train_new)
y_test_pred_3 = reg_ridge.predict(X_test_new)
```

```
from sklearn.metrics import mean_squared_error

rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_3))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_3))
print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
print(f'Raíz del error cuadrático medio en Test: {rmse_test}')

plt.figure(figsize = (18,8))

plt.subplot(1,2,1)
sns.distplot(y_train - y_train_pred_3, bins = 20, label = 'train')
sns.distplot(y_test - y_test_pred_3, bins = 20, label = 'test')
plt.xlabel('errores')
plt.legend()

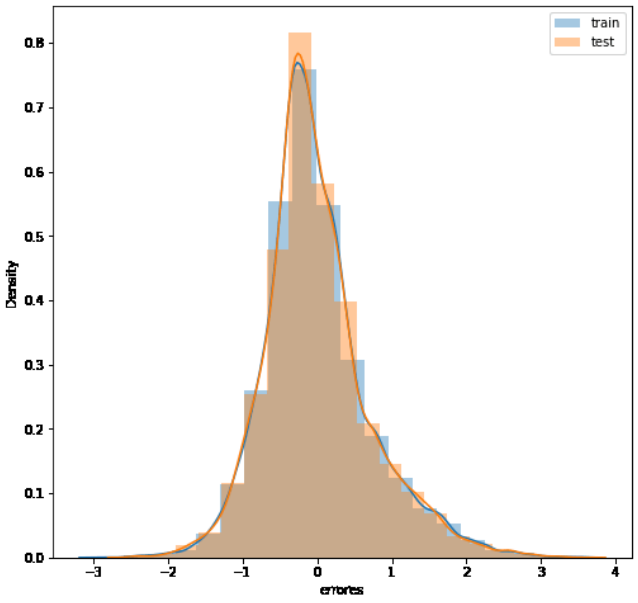

ax = plt.subplot(1,2,2)
ax.scatter(y_test,y_test_pred_3, s =2)

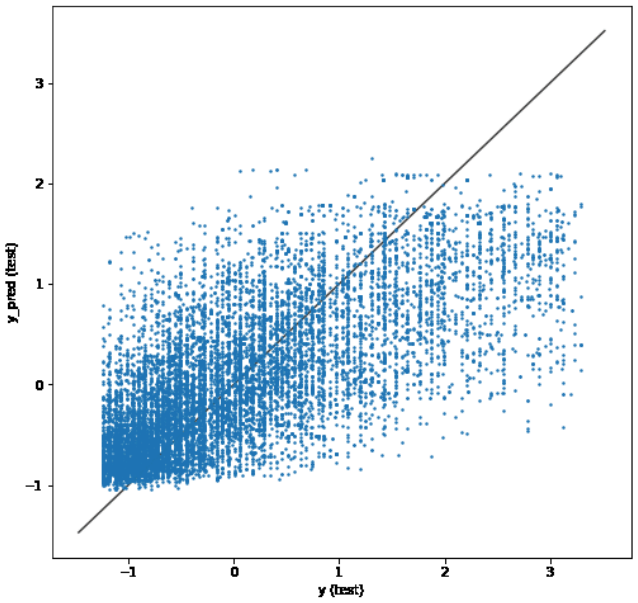
lims = [
np.min([ax.get_xlim(), ax.get_ylim()]),  # min of both axes
np.max([ax.get_xlim(), ax.get_ylim()]),  # max of both axes]
]
```

```
ax.plot(lims, lims, 'k-', alpha=0.75, zorder=0)
plt.xlabel('y (test)')
plt.ylabel('y_pred (test)')
```

```
Raíz del error cuadrático medio en Train: 0.713707539658679
Raíz del error cuadrático medio en Test: 0.708786769765957
```

```
Text(0, 0.5, 'y_pred (test)')
```





3.1.1.4 · Regresion lineal con Regularizacion Lasso

```
from sklearn.linear_model import Ridge, Lasso

reg_lasso = Lasso(alpha= 0.1)
reg_lasso_scores = cross_val_score(reg_lasso, X, y, cv=20)

reg_lasso.fit(X_train_new,y_train)

y_train_pred_4 = reg_lasso.predict(X_train_new)
y_test_pred_4= reg_lasso.predict(X_test_new)
```

```
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_4))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_4))
print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
print(f'Raíz del error cuadrático medio en Test: {rmse_test}')
```

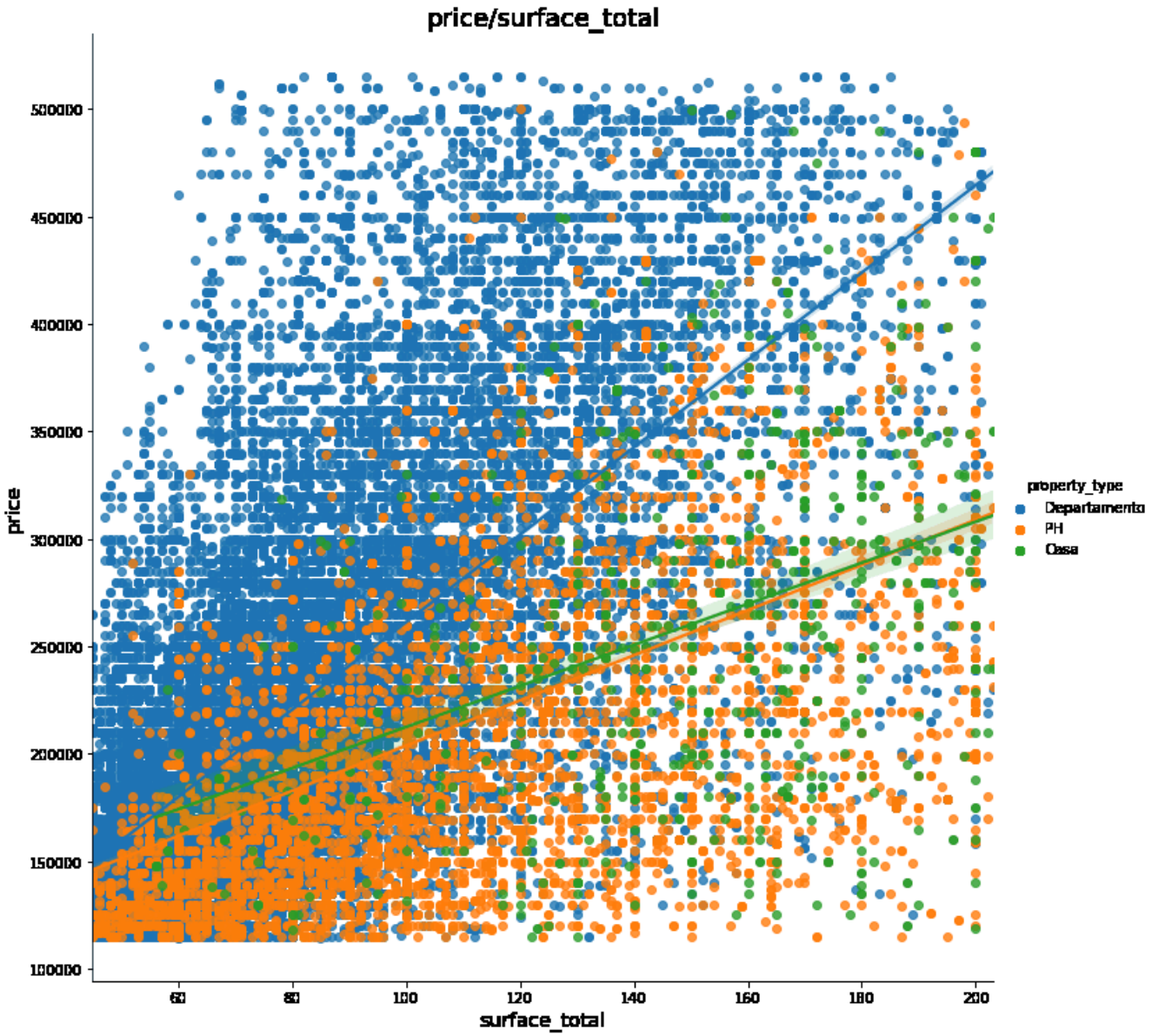
```
Raíz del error cuadrático medio en Train: 0.7278254275935023
Raíz del error cuadrático medio en Test: 0.7242213878201413
```

3.1.1.5 · Regresion lineal con atributos categoricos "property_type" diferenciados (departamento/ph y Casa)

- Apoyandome en este grafico del preprocesamiento voy a analizar regresion lineal diferenciando en dos grupos del atributo property_type (Departamento/Casa y PH)

```
g=sns.lmplot(x="surface_total", y="price",
data=new_properati_Capital1_All_sin_bedrooms,height=10,hue="property_type")
plt.ylabel("price",size=14)
plt.xlabel('surface_total',size=14)
plt.title("price/surface_total" ,size=18)
```

```
Text(0.5, 1.0, 'price/surface_total')
```




```
data_premiun.head(3)
```

	surface_total	surface_covered	bathrooms	price	rooms	bedrooms	property_type_Casa	property_type_Departamento	property_type_PH
0	1.572325	2.266138	0.723356	-0.809003	4.252372	5.906773	0	1	0
1	-0.495685	-0.625787	0.723356	-0.741033	-1.209918	-1.300293	0	0	1
2	-1.234259	-1.084263	-0.769286	-1.126197	-1.209918	-1.300293	0	0	1

3.1.1.5.1 · Departamento

```
X = data_premiun[[ "bathrooms","surface_covered","surface_total","property_type_Departamento"]]

y =data_premiun[['price']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

```
from sklearn.linear_model import LinearRegression

reg_3 = LinearRegression()

reg_3.get_params().keys()

param_grid_RL={"copy_X": ["True","False"],
"fit_intercept":["True","False"],
"normalize":["True","False"]}

model4 = GridSearchCV(reg_3, param_grid=param_grid_RL,cv=15)
```

```
model4.fit(X_train, y_train)
```

```
GridSearchCV(cv=15, estimator=LinearRegression(),
             param_grid={'copy_X': ['True', 'False'],
                         'fit_intercept': ['True', 'False'],
                         'normalize': ['True', 'False']})
```

```
y_train_pred_5 = model4.predict(X_train)
y_test_pred_5 = model4.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error

rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_5))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_5))
print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
print(f'Raíz del error cuadrático medio en Test: {rmse_test}')

plt.figure(figsize = (18,8))

plt.subplot(1,2,1)
sns.distplot(y_train - y_train_pred_5, bins = 20, label = 'train')
sns.distplot(y_test - y_test_pred_5, bins = 20, label = 'test')
plt.xlabel('errores')
plt.legend()

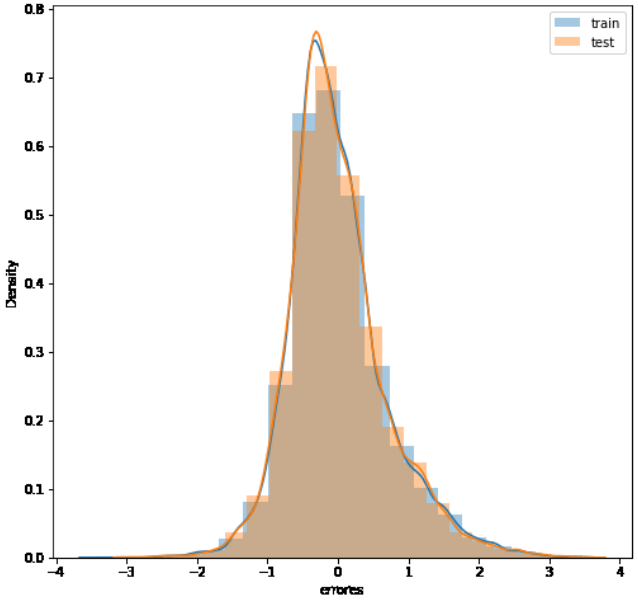

ax = plt.subplot(1,2,2)
ax.scatter(y_test,y_test_pred_5, s =2)

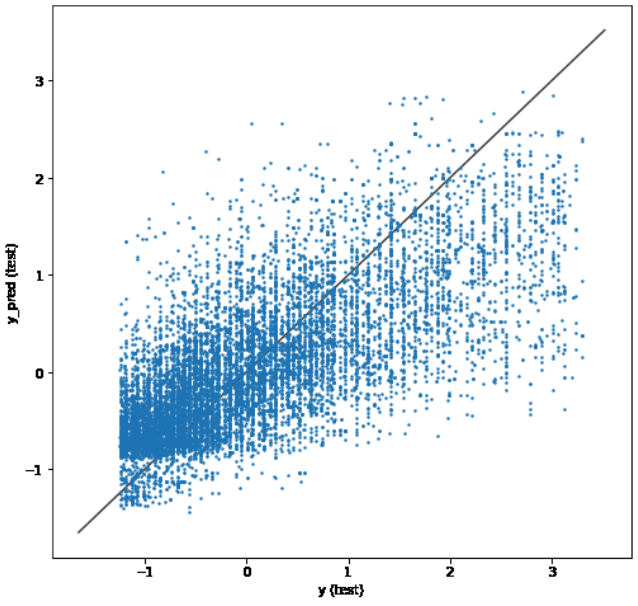
lims = [
np.min([ax.get_xlim(), ax.get_ylim()]),  # min of both axes
np.max([ax.get_xlim(), ax.get_ylim()]),  # max of both axes]
]

ax.plot(lims, lims, 'k-', alpha=0.75, zorder=0)
plt.xlabel('y (test)')
plt.ylabel('y_pred (test)')
```

```
Raíz del error cuadrático medio en Train: 0.6944128699155601
Raíz del error cuadrático medio en Test: 0.6887085448108949
```

```
Text(0, 0.5, 'y_pred (test)')
```





3.1.1.5.2 · Casa y PH

```
X = data_premiun[[ "bathrooms","surface_covered","surface_total","property_type_Casa","property_type_PH"]]

y =data_premiun[['price']]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

```
from sklearn.linear_model import LinearRegression

reg_4 = LinearRegression()

reg_4.get_params().keys()

param_grid_RL={"copy_X": ["True","False"],
"fit_intercept":["True","False"],
"normalize":["True","False"]}

model5 = GridSearchCV(reg_4, param_grid=param_grid_RL,cv=15)
```

```
model5.fit(X_train, y_train)
```

```
GridSearchCV(cv=15, estimator=LinearRegression(),
             param_grid={'copy_X': ['True', 'False'],
                         'fit_intercept': ['True', 'False'],
                         'normalize': ['True', 'False']})
```

```
y_train_pred_6 = model5.predict(X_train)
y_test_pred_6= model5.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error

rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_6))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_6))
print(f'Raíz del error cuadrático medio en Train: {rmse_train}')
print(f'Raíz del error cuadrático medio en Test: {rmse_test}')

plt.figure(figsize = (18,8))

plt.subplot(1,2,1)
sns.distplot(y_train - y_train_pred_6, bins = 20, label = 'train')
sns.distplot(y_test - y_test_pred_6, bins = 20, label = 'test')
plt.xlabel('errores')
plt.legend()

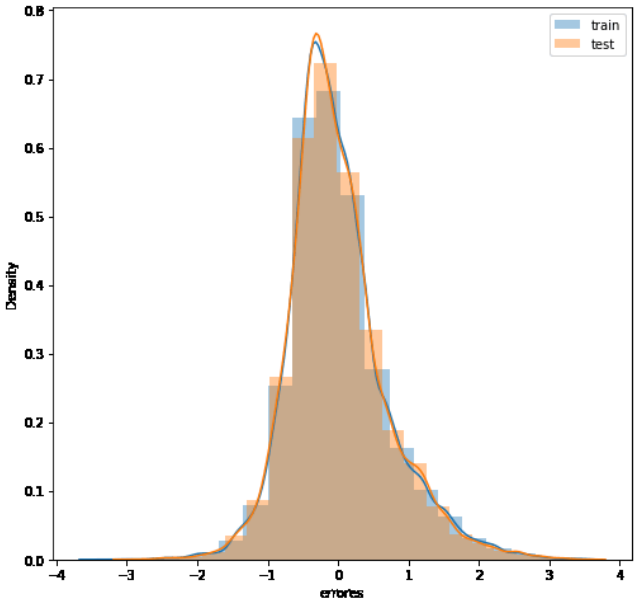

ax = plt.subplot(1,2,2)
ax.scatter(y_test,y_test_pred_6, s =2)

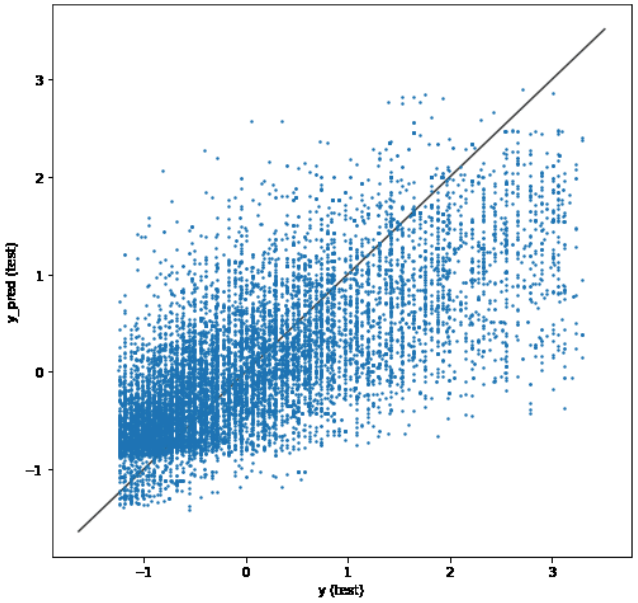
lims = [
np.min([ax.get_xlim(), ax.get_ylim()]),  # min of both axes
np.max([ax.get_xlim(), ax.get_ylim()]),  # max of both axes]
]

ax.plot(lims, lims, 'k-', alpha=0.75, zorder=0)
plt.xlabel('y (test)')
plt.ylabel('y_pred (test)')
```

```
Raíz del error cuadrático medio en Train: 0.6941859879176381
Raíz del error cuadrático medio en Test: 0.6880988089766211
```

```
Text(0, 0.5, 'y_pred (test)')
```





3.1.2 · Conclusiones/Resultados

- **data_premiun**

1- LinearRegression()

Raíz del error cuadrático medio en Train: **0.7175**

Raíz del error cuadrático medio en Test: **0.7126**

2- poly = PolynomialFeatures(6,interaction_only=True)/LinearRegression()

Raíz del error cuadrático medio en Train: **0.7137**

Raíz del error cuadrático medio en Test: **0.7087**

3- reg_ridge = Ridge(alpha= 0.1)

Raíz del error cuadrático medio en Train: **0.7137**

Raíz del error cuadrático medio en Test: **0.7087**

4- reg_lasso = Lasso(alpha= 0.1)

Raíz del error cuadrático medio en Train: **0.7278**

Raíz del error cuadrático medio en Test: **0.7242**

- Regresion lineal con atributos categoricos "property_type" diferenciados/Departamento

Raíz del error cuadrático medio en Train: **0.6944**

Raíz del error cuadrático medio en Test: **0.6887**

- Regresion lineal con atributos categoricos "property_type" diferenciados/PH, Casa

Raíz del error cuadrático medio en Train: **0.6941**

Raíz del error cuadrático medio en Test: **0.6880**

- Adjunto tambien estos resultados de Kvecinos y Arboles para tener toda la info visible para comparar resultados.

- **data_premiun**

- Modelo: Árbol de Decisión

Raíz del error cuadrático medio en Train por TreeRegressor: **0.5060**

Raíz del error cuadrático medio en Test por TreRegressor: **0.6376**

- Modelo: Vecinos más cercanos

Raíz del error cuadrático medio en Train por Knn: **0.6168**

Raíz del error cuadrático medio en Test por Knn: **0.7022**

El modelo regresion lineal; ya sea simple, con atributos polinomicos o con regularizacion Lazzo o Ridge, dio muy malos resultados teniendo en cuenta que es un modelo avanzado. Mas alla de esto es importante remarcar que cuando diferenciamos las variables categoricas Casa, PH y Departamentos del atributo "property_type; los resultado mejoraron 0.7126 a 0.6880. Voy a utilizar esta informacion para utilizarla en el siguiente modelo avanzado que voy a utilizar (XGboost).

Hasta aca el mejor resultado lo obtuvimos con Arbol de Decision. RMSE_Test= 0.6376.__ (sin Gridsearch o crossvalidation).

3.2 · Modelo Avanzado 2 - XGBoostRegression

3.2.1 · XGBoostRegression con Gredsearch y CV

```
import sklearn.metrics as metrics

from sklearn.model_selection import learning_curve
```

```
import xgboost as xgb

X = data_premiun[[ "bathrooms","surface_covered","surface_total"]]

y =data_premiun[['price']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model_xgb_r=xgb.XGBRegressor(booster="gbtree",silent=True,randomstate=42)

parameters_xgb={"n_estimators":[100,200,400,600],
                "gamma":[0.2,0.4,0.6,0.8],
                "learning_rate":[0.05,0.1],
                "n_estimators":[100,200,400,600],
                "max_depth":[4,8,10.15]}


grid = GridSearchCV(estimator=model_xgb_r, param_grid=parameters_xgb,cv=7,n_jobs=-5)
```

```
grid.fit(X_train, y_train)
```

```
GridSearchCV(cv=7,
             estimator=XGBRegressor(base_score=None, booster='gbtree',
                                    colsample_bylevel=None,
                                    colsample_bynode=None,
                                    colsample_bytree=None, gamma=None,
                                    gpu_id=None, importance_type='gain',
                                    interaction_constraints=None,
                                    learning_rate=None, max_delta_step=None,
                                    max_depth=None, min_child_weight=None,
                                    missing=nan, monotone_constraints=None,
                                    n_estimators=100, n_jobs=None,
                                    num_parallel_tree=None, random_state=None,
                                    randomstate=42, reg_alpha=None,
                                    reg_lambda=None, scale_pos_weight=None,
                                    silent=True, subsample=None,
                                    tree_method=None, validate_parameters=None,
                                    verbosity=None),
             n_jobs=-5,
             param_grid={'gamma': [0.2, 0.4, 0.6, 0.8],
                         'learning_rate': [0.05, 0.1],
                         'max_depth': [4, 8, 10.15],
                         'n_estimators': [100, 200, 400, 600]})
```

```
y_train_pred_8= grid.predict(X_train)
y_test_pred_8= grid.predict(X_test)
```

```
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_8))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_8))

print('RMSE sobre el train set: ', rmse_train)
print('RMSE sobre el test set: ', rmse_test)
```

```
RMSE sobre el train set:  0.5605450367778209
RMSE sobre el test set:  0.6166470911124178
```

```
importancias_features=grid.best_estimator_.feature_importances_
print(importancias_features)
#Orden X = data_premiun1[[ "bathrooms","surface_covered","surface_total"]]
```

```
[0.24624148 0.56510425 0.18865424]
```

3.2.1.1 · XGBoostRegression con Gredsearch con atributos categoricos "property_type" diferenciados (departamento/ph y Casa)

3.2.1.1.1 · Departamento

```
X = data_premiun[[ "bathrooms","surface_covered","surface_total","property_type_Departamento"]]

y =data_premiun[['price']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model_xgb_r=xgb.XGBRegressor(silent=True,randomstate=42)

parameters_xgb={"n_estimators":[100,200,400,600],
                "gamma":[0.2,0.4,0.6,0.8],
                "learning_rate":[0.05,0.1],
                "n_estimators":[100,200,400,600],
                "max_depth":[4,8,10.15]}


grid = GridSearchCV(estimator=model_xgb_r, param_grid=parameters_xgb,cv=7,n_jobs=-5)

grid.fit(X_train, y_train)
```

```
GridSearchCV(cv=7,
             estimator=XGBRegressor(base_score=None, booster=None,
                                    colsample_bylevel=None,
                                    colsample_bynode=None,
                                    colsample_bytree=None, gamma=None,
                                    gpu_id=None, importance_type='gain',
                                    interaction_constraints=None,
                                    learning_rate=None, max_delta_step=None,
                                    max_depth=None, min_child_weight=None,
                                    missing=nan, monotone_constraints=None,
                                    n_estimators=100, n_jobs=None,
                                    num_parallel_tree=None, random_state=None,
                                    randomstate=42, reg_alpha=None,
                                    reg_lambda=None, scale_pos_weight=None,
                                    silent=True, subsample=None,
                                    tree_method=None, validate_parameters=None,
                                    verbosity=None),
             n_jobs=-5,
             param_grid={'gamma': [0.2, 0.4, 0.6, 0.8],
                         'learning_rate': [0.05, 0.1],
                         'max_depth': [4, 8, 10.15],
                         'n_estimators': [100, 200, 400, 600]})
```

```
y_train_pred_9= grid.predict(X_train)
y_test_pred_9= grid.predict(X_test)
```

```
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_9))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_9))

print('RMSE sobre el train set: ', rmse_train)
print('RMSE sobre el test set: ', rmse_test)
```

```
RMSE sobre el train set:  0.5115429566160498
RMSE sobre el test set:  0.580993278517465
```

```
importancias_features=grid.best_estimator_.feature_importances_
print(importancias_features)
# Orden X = data_premiun1[[ "bathrooms","surface_covered","surface_total","property_type_Departamento"]]
```

```
[0.16380732 0.4036619  0.14024761 0.29228324]
```

3.2.1.1.2 · Casa y PH

> **Nota (2026):** las salidas de las secciones 3.2.1.1.2 y 3.2.1.1.3 se habían perdido en el notebook original (el kernel se reinició antes de guardar). Se re-ejecutaron con el mismo código y los mismos datos (`new_properati_Capital2_All_sin_bedrooms_clean_normalizado.csv`), con versiones actuales de las librerías. Por eso los valores pueden diferir levemente de los citados en las conclusiones, que se escribieron con la corrida original.

```
X = data_premiun[[ "bathrooms","surface_covered","surface_total","property_type_Casa","property_type_PH"]]

y =data_premiun[['price']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model_xgb_r=xgb.XGBRegressor(silent=True,randomstate=42)

parameters_xgb={"n_estimators":[100,200,400,600],
                "gamma":[0.2,0.4,0.6,0.8],
                "learning_rate":[0.05,0.1],
                "n_estimators":[100,200,400,600],
                "max_depth":[4,8,10.15]}


grid = GridSearchCV(estimator=model_xgb_r, param_grid=parameters_xgb,cv=7,n_jobs=-5)

grid.fit(X_train, y_train)
```

```
GridSearchCV(cv=7,
             estimator=XGBRegressor(base_score=None, booster=None,
                                    callbacks=None, colsample_bylevel=None,
                                    colsample_bynode=None,
                                    colsample_bytree=None, device=None,
                                    early_stopping_rounds=None,
                                    enable_categorical=True, eval_metric=None,
                                    feature_types=None, feature_weights=None,
                                    gamma=None, grow_policy=None,
                                    importance_type=None,
                                    interaction_constraints=None,...
                                    max_cat_threshold=None,
                                    max_cat_to_onehot=None, max_delta_step=None,
                                    max_depth=None, max_leaves=None,
                                    min_child_weight=None, missing=nan,
                                    monotone_constraints=None,
                                    multi_strategy=None, n_estimators=None,
                                    n_jobs=None, num_parallel_tree=None, ...),
             n_jobs=-5,
             param_grid={'gamma': [0.2, 0.4, 0.6, 0.8],
                         'learning_rate': [0.05, 0.1],
                         'max_depth': [4, 8, 10.15],
                         'n_estimators': [100, 200, 400, 600]})
```

```
y_train_pred_10= grid.predict(X_train)
y_test_pred_10= grid.predict(X_test)
```

```
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_10))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_10))

print('RMSE sobre el train set: ', rmse_train)
print('RMSE sobre el test set: ', rmse_test)
```

```
RMSE sobre el train set:  0.5769045293406351
RMSE sobre el test set:  0.6042547052189969
```

```
importancias_features=grid.best_estimator_.feature_importances_
print(importancias_features)
#Orden X = data_premiun1[[
"bathrooms","surface_covered","surface_total","property_type_Casa","property_type_PH"]]
```

```
[0.19649845 0.36126304 0.10049497 0.11515126 0.22659232]
```

3.2.1.1.3 · Pruebo con todos los Atributos

```
X = data_premiun[[
"bathrooms","surface_covered","surface_total","bedrooms","rooms","property_type_Casa","property_type_Depar
tamento",
                   "property_type_PH"]]

y =data_premiun[['price']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model_xgb_r=xgb.XGBRegressor(booster="gbtree",silent=True,randomstate=42)

parameters_xgb={"n_estimators":[100,200,400,600],
                "gamma":[0.2,0.4,0.6,0.8],
                "learning_rate":[0.05,0.1],
                "n_estimators":[100,200,400,600],
                "max_depth":[4,8,10.15]}


grid = GridSearchCV(estimator=model_xgb_r, param_grid=parameters_xgb,cv=7,n_jobs=-5)
```

```
grid.fit(X_train, y_train)
```

```
GridSearchCV(cv=7,
             estimator=XGBRegressor(base_score=None, booster='gbtree',
                                    callbacks=None, colsample_bylevel=None,
                                    colsample_bynode=None,
                                    colsample_bytree=None, device=None,
                                    early_stopping_rounds=None,
                                    enable_categorical=True, eval_metric=None,
                                    feature_types=None, feature_weights=None,
                                    gamma=None, grow_policy=None,
                                    importance_type=None,
                                    interaction_constraints=N...
                                    max_cat_threshold=None,
                                    max_cat_to_onehot=None, max_delta_step=None,
                                    max_depth=None, max_leaves=None,
                                    min_child_weight=None, missing=nan,
                                    monotone_constraints=None,
                                    multi_strategy=None, n_estimators=None,
                                    n_jobs=None, num_parallel_tree=None, ...),
             n_jobs=-5,
             param_grid={'gamma': [0.2, 0.4, 0.6, 0.8],
                         'learning_rate': [0.05, 0.1],
                         'max_depth': [4, 8, 10.15],
                         'n_estimators': [100, 200, 400, 600]})
```

```
y_train_pred_11= grid.predict(X_train)
y_test_pred_11= grid.predict(X_test)
```

```
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred_11))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred_11))

print('RMSE sobre el train set: ', rmse_train)
print('RMSE sobre el test set: ', rmse_test)
```

```
RMSE sobre el train set:  0.5375530560500715
RMSE sobre el test set:  0.5722529959993942
```

```
importancias_features=grid.best_estimator_.feature_importances_
print(importancias_features)
#Orden X = data_premiun1[[
```

```
"bathrooms","surface_covered","surface_total","bedrooms","rooms","property_type_Casa","property_type_Depar
tamento",
#                   "property_type_PH"]]
```

```
[0.15904433 0.273773   0.10201913 0.05263152 0.04592119 0.01909026
 0.25058296 0.09693757]
```

3.2.2 · Conclusiones/Resultados

- El mejor RMSE (0.53858) lo obtengo con todos los atributos incluidos los categoricos (casa, departamento y PH).

- **feature_importances:**

> - El peso de los atributos en este modelo estuvo siempre, en todas las combinaciones que realice, en **surface_covered.**
> - Cuando solo utilice los 3 atributos surface_covered, surface_total y bathrooms, el peso era de 0.565.
> - Cuando utilice ademas el atributo property_type_Departamento fue de 0.36685666.
> - Con todos los atributos incluidos, su peso fue de 0.2719. En este ultimo caso, el peso de property_type_Departamento y Casa estuvo en valores muy cercano de 0.25 y 0.24.

4 · Conclusiones Generales

- Entre el df del proyecto 1 y los df del proyecto 2, el mejor desempeño se da por la mejora en el **preprocesamiento de los datos.** Esa mejora es **superior a un 65%.**

- En el df de este proyecto 2 **(data_premiun)** luego de haber probado con:

> 1-Modelos Simples (Kvecinos, Arboles de desicion, Regresion lineal).
>
> 2-Modelos Avanzados (Regresion lineal con Atributos Polinomicos y Regularizacion, XGboost)
>
> 3-Distintas combinaciones de atributos (incluso el categorico "property_type).
>
> 4- Distintos hiperparametros (Gridsearch) y Cross validation.
>
> -**El mejor desempeño se logro con el Modelo de XGBoost** con todos los atributos incluidos (incluso el categorico "property_type), con un Gridserach y cv.
>
> - **El mejor RMSE_Test conseguido es de RMSE (0.53858).**

5 · Para seguir investigando

- Probar el desempeño de XGboost con el df del proyecto1 **(df_proyecto1_clean)**. Voy a tener que codificar los atributos de property_type (Casa, Departamento y PH).

- Probar el desempeño de XGboost con alguno de los df del preprocesamiento de este proyecto 2, pero que no tenga eliminados los outliers. Voy a tener que codificar los atributos de property_type y tendre que trabajar los nulos. Este modelo XGboost tiene un mejor desempeño con los outliers. Haciendo esta prueba podria verificar esto.

- Probar el df **data_premiun1** con todos los **unique** del atributo **"l3"**, siendo todas las "Ciudades" dentro de la "Region Capital Federal".

- Probar esto ultimo con el modelo Light GBM; que permitiria hacerlo sin codificar los atributos categoricos con dummies. Ahorro tiempo y pruebo su desempeño.